

STMIK KAPUTAMA BINJAI
GOOGLE PROMPT ENGINEERING COURSE



FINAL PROJECT REPORT

NAZSYA AI TUTOR

AI Smart Tutor Using Advanced Prompt Engineering Techniques

Submitted by:

Nazwa Salsabila

NPM : 23441066

STMIK KAPUTAMA BINJAI

2026

TABLE OF CONTENTS

TABLE OF CONTENTS	i
LIST OF FIGURES	iv
PROJECT INTRODUCTION	1
M1. Project Setup and Initial Prompt Exploration	3
1.1 Team Formation and Project Planning	3
1.2 Initial LLM Selection and API Setup	3
1.2.1 Initial Installation Command for Gemini API	4
1.2.2 Final API Migration Preparation	4
1.2.3 Additional System Configuration Improvement	6
1.3 Basic Prompt Exploration	7
1.3.1 Initial Prompt Testing Using Gemini API	8
1.3.2 Model Configuration Error and Resolution	9
1.3.3 Final Basic Prompt Implementation in OpenRouter Version	12
M2. Structured Answer Generator and Multi-Level Explanation Generator	15
2.1 Structured Answer Generator Design	15
2.1.1 Initial Structured Prompt Design Using Gemini API	15
2.1.2 Final Structured Prompt Implementation in OpenRouter Version	17
2.2 Multi-Level Explanation Generator Design	19
2.2.1 Initial Multi-Level Prompting Implementation Using Gemini API	19
2.2.2 Final Multi-Level Prompting Implementation in OpenRouter Version	22
2.2.3 Multi-Level Response Comparison	26
2.3 Testing and Refinement	27
2.3.1 Testing Summary	28
M3. Study Plan Generator and Quiz Generator	30
3.1 Study Plan Generator Development	30
3.1.1 Initial Study Plan Prompt Integration Using Gemini API	30
3.1.2 Final Study Plan Generator Implementation in OpenRouter Version	31
3.2 Quiz Generator Development	33
3.2.1 Initial Quiz Generation Prompt Integration Using Gemini API	33
3.2.2 Final Interactive Quiz Generator Implementation in OpenRouter Version	35

3.3 Few-Shot Prompting Integration	38
3.3.1 Initial Few-Shot Prompting Implementation	38
3.3.2 Final Few-Shot Prompting Integration in OpenRouter Version.....	40
3.3.3 Few-Shot Prompting Effect	42
M4. Answer Evaluator (Examiner AI).....	43
4.1 Answer Evaluator Prompt Design.....	43
4.1.1 Initial Answer Evaluation Prompt Design.....	43
4.1.2 Final Answer Evaluator Implementation in OpenRouter Version	45
4.2 Rubric-Based Prompting Implementation.....	48
4.2.1 Initial Rubric-Based Evaluation Prompt Design	48
4.2.2 Final Rubric-Based Evaluator Implementation in OpenRouter Version.....	50
4.3 Role-Based Prompting Experimentation.....	53
4.3.1 Initial Role-Based Prompting Implementation.....	53
4.3.2 Final Role-Based Evaluator Implementation in OpenRouter Version	56
4.3.3 Role Comparison Summary	59
M5. Prompt Comparison Engine and Optimization	60
5.1 Prompt Comparison Engine Development.....	60
5.1.1 Initial Multiple Prompt Generation Implementation	60
5.1.2 Prompt Comparison Display Implementation	60
5.1.3 Final Prompt Comparison Engine Implementation in OpenRouter Version.....	61
5.1.4 Prompt Comparison Analysis	64
5.2 Prompt Optimization	65
5.2.1 Initial Optimized Prompt Implementation.....	65
5.2.2 Final Optimized Prompt Implementation in OpenRouter Version.....	67
5.2.3 Prompt Optimization Analysis	68
5.3 Performance Evaluation	69
5.3.1 Performance Evaluation Implementation	69
5.3.2 Performance Evaluation Analysis	72
M6. Finalization, Documentation, and Presentation	74
6.1 Final Project Report Writing	74
6.1.1 Final System Overview	74
6.1.2 Development Challenges and Improvements	76
6.1.3 System Workflow	78

6.1.4 Code Structure Improvement.....	79
6.1.5 UI Design Planning and Figma Prototype	80
6.1.6 Final Application Interface Documentation	83
6.2 Prompt Template Library Creation	86
6.2.1 Basic Prompt Template	87
6.2.2 Optimized Prompt Template	87
6.2.3 Quiz Generation Prompt Template.....	90
6.2.4 Evaluator Prompt Template.....	91
6.3 Presentation Preparation.....	92
6.3.1 Presentation Slide Structure.....	93
6.3.2 Final Demonstration Preparation.....	94
6.3.3 Final Project Conclusion	94

LIST OF FIGURES

Figure 1. 1 Project Folder Structure.....	3
Figure 1. 2 Project Folder Structure.....	3
Figure 1. 3 VSCode Workspace Setup.....	3
Figure 1. 4 Initial Gemini package installation process.....	4
Figure 1. 5 Initial Google AI Studio API setup	4
Figure 1. 6 Final OpenRouter API configuration in app.py.....	5
Figure 1. 7 OpenRouter API key configuration using .env file	6
Figure 1. 8 Successful final application execution using OpenRouter API.....	6
Figure 1. 9 Environment variable integration using .env configuration	7
Figure 1. 10 Install python-dotenv	7
Figure 1. 11 gitignore configuration for API security	7
Figure 1. 12 Initial Streamlit and Gemini API Integration Code.....	8
Figure 1. 13 Initial Prompt Testing.....	9
Figure 1. 14 API model configuration error during initial prompt testing	9
Figure 1. 15 Successful AI-generated educational response after updating the Gemini model configuration	10
Figure 1. 16 Comparison of AI-generated responses using different prompt variations and instructional styles	11
Figure 1. 17 Comparison of AI-generated responses using different prompt variations and instructional styles	11
Figure 1. 18 Final basic prompt implementation using OpenRouter API	13
Figure 1. 19 Basic prompt response result in the final Nazsya AI Tutor application.....	13
Figure 2. 1 Structured prompt implementation for educational response generation..	16
Figure 2. 2 Structured AI-generated educational response using prompt engineering techniques.	16
Figure 2. 3 Final structured prompt format implementation using OpenRouter API.....	17
Figure 2. 4 Structured educational response in the final Nazsya AI Tutor application	18
Figure 2. 5 Initial multi-level explanation prompt implementation using Streamlit and Gemini API	20
Figure 2. 6 Beginner-level educational explanation generated by Nazsya AI Tutor	20
Figure 2. 7 Intermediate-level educational explanation generated by Nazsya AI Tutor	21
Figure 2. 8 Advanced-level educational explanation generated by Nazsya AI Tutor	21
Figure 2. 9 Learner level adaptation rules in the final optimized prompt.....	22
Figure 2. 10 Final explanation level selection implementation in Streamlit	23
Figure 2. 11 Explanation level selection in the final Nazsya AI Tutor interface.....	23
Figure 2. 12 Final Multi-Level Prompting Implementation in OpenRouter Version	24
Figure 2. 13 Intermediate-level response generated in the final OpenRouter-based application	25
Figure 2. 14 Advanced-level response generated in the final OpenRouter-based application	26
Figure 2. 15 Testing and comparison of multi-level educational responses generated by Nazsya AI Tutor.....	28

Figure 2. 16 Final interface of the multi-level explanation system in the OpenRouter-based applicationN.....	28
Figure 3. 1 Study plan generator prompt integration	30
Figure 3. 2 AI-generated personalized study plan based on learner level and study duration	31
Figure 3. 3 AI-generated personalized study plan based on learner level and study duration	31
Figure 3. 4 Final study duration selection implementation in Streamlit.....	32
Figure 3. 5 Final study plan prompt implementation using OpenRouter API	32
Figure 3. 6 AI-generated study plan in the final OpenRouter-based application	32
Figure 3. 7 Quiz generation prompt integration.....	34
Figure 3. 8 AI-generated quiz and answer key for educational assessment	34
Figure 3. 9 AI-generated quiz and answer key for educational assessment	35
Figure 3. 10 Final quiz generation prompt format using OpenRouter API	36
Figure 3. 11 Quiz extraction implementation using QUIZ_START and QUIZ_END delimiters.....	36
Figure 3. 12 Quiz parsing implementation for interactive multiple-choice questions.....	36
Figure 3. 13 Interactive quiz display in the final Nazsya AI Tutor application.....	37
Figure 3. 14 Quiz score result after user submission	37
Figure 3. 15 Few-shot prompting implementation using example-based prompt engineering	39
Figure 3. 16 AI-generated educational response using Few-Shot Prompting techniques	39
Figure 3. 17 AI-generated educational response using Few-Shot Prompting techniques	40
Figure 3. 18 Final few-shot prompting implementation in the optimized prompt.....	41
Figure 3. 19 AI-generated educational response using final few-shot prompting integration	41
Figure 4. 1 Answer evaluator prompt implementation	44
Figure 4. 2 AI-generated evaluation feedback including score, strengths, weaknesses, and improvement suggestions.....	44
Figure 4. 3 AI-generated evaluation feedback including score, strengths, weaknesses, and improvement suggestions.....	45
Figure 4. 4 AI-generated evaluation feedback including score, strengths, weaknesses, and improvement suggestions.....	45
Figure 4. 5 Final answer evaluator prompt implementation using OpenRouter API	46
Figure 4. 6 Answer evaluator input field in the final Nazsya AI Tutor application	46
Figure 4. 7 AI-generated answer evaluation feedback in the final application.....	47
Figure 4. 8 AI-generated answer evaluation feedback in the final application.....	47
Figure 4. 9 Rubric-based evaluation prompt implementation.....	49
Figure 4. 10 AI-generated rubric-based evaluation using Accuracy, Completeness, and Clarity criteria	49
Figure 4. 11 AI-generated rubric-based evaluation using Accuracy, Completeness, and Clarity criteria	50
Figure 4. 12 Final rubric-based evaluator prompt implementation using OpenRouter API...	51
Figure 4. 13 Rubric-based answer evaluator input field in the final application	51
Figure 4. 14 AI-generated rubric-based evaluation feedback in the final application.....	52
Figure 4. 15 Role-based prompting implementation for AI evaluator behavior customization	54

Figure 4. 16	Friendly Tutor evaluation style generated	54
Figure 4. 17	Friendly Tutor evaluation style generated	55
Figure 4. 18	Strict Lecturer evaluation style generated	55
Figure 4. 19	Strict Lecturer evaluation style generated	56
Figure 4. 20	Final evaluator role selection implementation in Streamlit.....	57
Figure 4. 21	Final role-based evaluator prompt implementation using OpenRouter API	57
Figure 4. 22	Evaluator role selection in the final Nazsya AI Tutor interface	57
Figure 4. 23	Role-based answer evaluation feedback in the final application.....	58
Figure 5. 1	Multiple prompt generation implementation using several prompt engineering techniques	60
Figure 5. 2	Comparison of multiple AI-generated responses using different prompt engineering techniques.....	61
Figure 5. 3	Basic prompt response implementation using OpenRouter API	62
Figure 5. 4	Optimized prompt response implementation using OpenRouter API	63
Figure 5. 5	Final comparison display between Basic Prompt Response and Optimized Prompt Response	64
Figure 5. 6	Optimized prompt implementation for improving educational response quality	66
Figure 5. 7	AI-generated educational response using optimized prompt engineering techniques	66
Figure 5. 8	Final optimized prompt implementation using OpenRouter API.....	68
Figure 5. 9	Prompt engineering analysis of techniques used in the final Nazsya AI Tutor application.....	69
Figure 5. 10	Performance evaluation implementation for prompt engineering comparison ..	70
Figure 5. 11	Performance evaluation results and final analysis of multiple prompt engineering techniques.....	71
Figure 5. 12	Performance evaluation results of multiple prompt engineering techniques	71
Figure 6. 1	Final interface overview of Nazsya AI Tutor	75
Figure 6. 2	Main features of Nazsya AI Tutor	76
Figure 6. 3	Final OpenRouter API integration in Nazsya AI Tutor.....	77
Figure 6. 4	Interactive quiz parsing improvement using structured quiz delimiters.....	77
Figure 6. 5	User input and AI response workflow in Nazsya AI Tutor	79
Figure 6. 6	Final modular code structure of Nazsya AI Tutor in Visual Studio Code	80
Figure 6. 7	Figma prototype overview of Nazsya AI Tutor.....	83
Figure 6. 8	Learning session interface with basic and optimized prompt response sections..	83
Figure 6. 9	Basic prompt response result in the final Nazsya AI Tutor application	84
Figure 6. 10	Optimized prompt response result using advanced prompt engineering techniques	84
Figure 6. 11	Prompt engineering analysis section in Nazsya AI Tutor	85
Figure 6. 12	Quiz submission interface in Nazsya AI Tutor	85
Figure 6. 13	Performance evaluation results in the final application.....	86
Figure 6. 14	AI answer evaluator interface and rubric-based feedback.....	86
Figure 6. 15	Basic prompt template implementation in Nazsya AI Tutor	87
Figure 6. 16	Optimized prompt template implementation in Nazsya AI Tutor	89
Figure 6. 17	Quiz generation prompt template and parsing implementation.....	91

Figure 6. 18	Rubric-based and role-based evaluator prompt implementation.....	92
---------------------	--	----

PROJECT INTRODUCTION

1. Background

The rapid development of Artificial Intelligence has created new opportunities in the education sector. AI-powered learning systems can help students understand learning materials more easily by providing explanations, examples, quizzes, study plans, and feedback.

However, general AI responses are often inconsistent, too broad, or not adapted to the learner's level. Without proper prompt design, AI-generated answers may lack structure, clarity, and educational focus.

Therefore, this project was developed to design Nazsya AI Tutor, an AI-powered educational assistant that uses advanced prompt engineering techniques to generate structured, personalized, and interactive learning responses. The system applies several prompt engineering techniques, including structured prompting, multi-level explanation, few-shot prompting, role-based prompting, rubric-based evaluation, and prompt optimization.

2. Problem Statement

Based on the background, several problems were identified:

1. General AI responses are often not structured enough for educational use.
2. Learners may need different explanation levels depending on their understanding.
3. Students need not only explanations, but also study plans, quizzes, and feedback.
4. AI-generated evaluation needs clear criteria to provide useful learning feedback.
5. Prompt engineering techniques need to be tested and compared to improve AI response quality.

3. Project Objectives

The objectives of this project are:

1. To develop an AI-powered tutor application using Streamlit and Large Language Model API integration.
2. To implement structured prompting for generating organized educational explanations.
3. To provide multi-level explanations for Beginner, Intermediate, and Advanced learners.
4. To generate personalized study plans based on selected study duration.
5. To create interactive quizzes for learner self-assessment.
6. To implement an AI Answer Evaluator using rubric-based and role-based prompting.

7. To compare basic and optimized prompts to evaluate the impact of prompt engineering techniques.

4. Project Scope

The scope of this project includes the development of Nazsya AI Tutor as a web-based educational assistant using Streamlit. The system focuses on generating educational explanations, study plans, quizzes, and answer evaluations based on user input.

The application initially used Google Gemini API during the early prototype and prompt exploration stage. However, the final implementation was migrated to OpenRouter API using the DeepSeek Chat model to improve testing flexibility and response stability.

This project focuses on prompt engineering implementation and evaluation. It does not cover user account management, database storage, automatic grading for complex essays, or deployment to a public production server.

5. Project Benefits

This project provides several benefits:

1. Helping learners understand educational topics through structured and personalized explanations.
2. Supporting students with study plans and interactive quizzes.
3. Providing feedback on user-submitted answers through AI evaluation.
4. Demonstrating how prompt engineering techniques can improve AI-generated educational responses.
5. Providing a practical example of integrating Streamlit, API-based LLMs, and prompt engineering techniques in an educational application.

M1. Project Setup and Initial Prompt Exploration

1.1 Team Formation and Project Planning

In this initial stage, the project idea was selected, the project title was determined, and the overall development process was planned. The project chosen for development was Nazsya AI Tutor, an AI-powered smart tutor designed to provide personalized educational assistance using advanced prompt engineering techniques.

The main objective of this project is to improve learning experiences by generating structured explanations, adaptive responses, quizzes, study plans, and answer evaluations using Large Language Models (LLMs).

The project also identified the tools and technologies required for the project development, including Python, Streamlit, Google Gemini API for initial experimentation, and OpenRouter API with DeepSeek Chat for the final implementation.

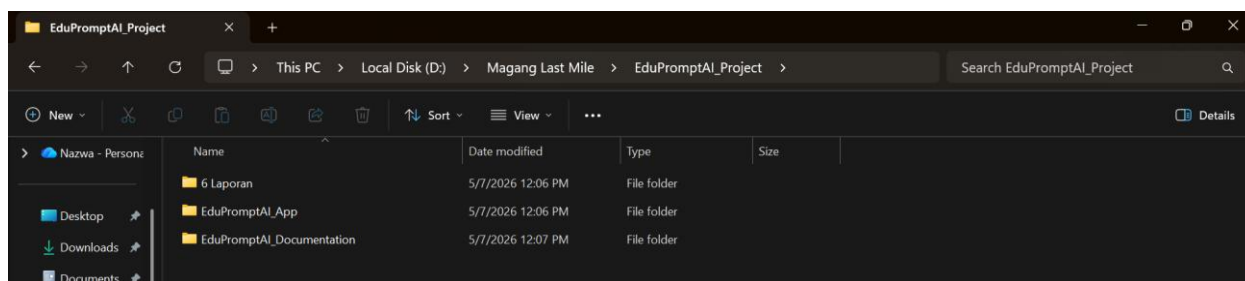


Figure 1. 1 Project Folder Structure

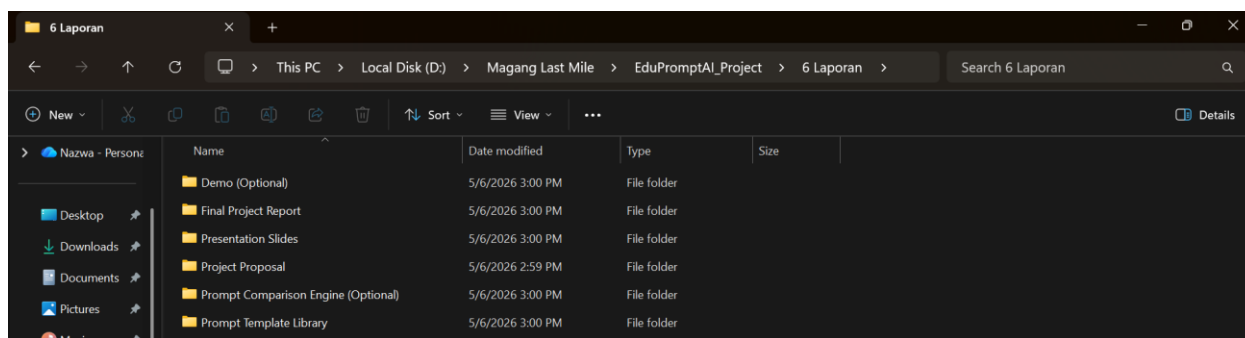


Figure 1. 2 Project Folder Structure

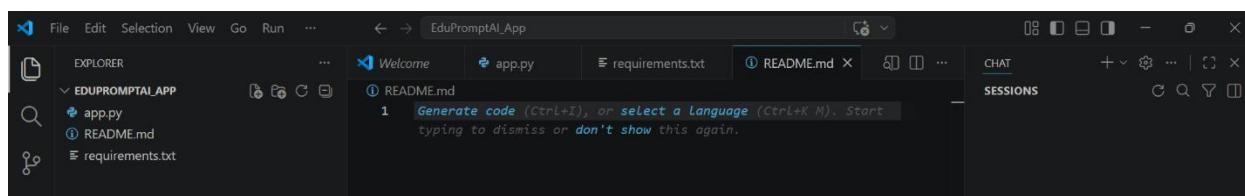


Figure 1. 3 VSCode Workspace Setup

1.2 Initial LLM Selection and API Setup

In the initial development phase, the project selected Google Gemini as the first Large Language Model used for prompt exploration and early prototype testing. Gemini was chosen

because it provides accessible API integration, fast response generation, and strong support for prompt engineering experiments.

The development environment was prepared by installing the required Python libraries such as Streamlit and Google Generative AI SDK. These tools were used to build the early prototype of Nazsya AI Tutor and test basic prompt generation, structured responses, and multi-level explanations.

The initial Gemini API key was generated through Google AI Studio and configured within the project environment. This setup was used during the early experimentation stage before the final implementation was later migrated to OpenRouter API due to API quota limitations and testing stability needs.

1.2.1 Initial Installation Command for Gemini API

`pip install streamlit google-generativeai`

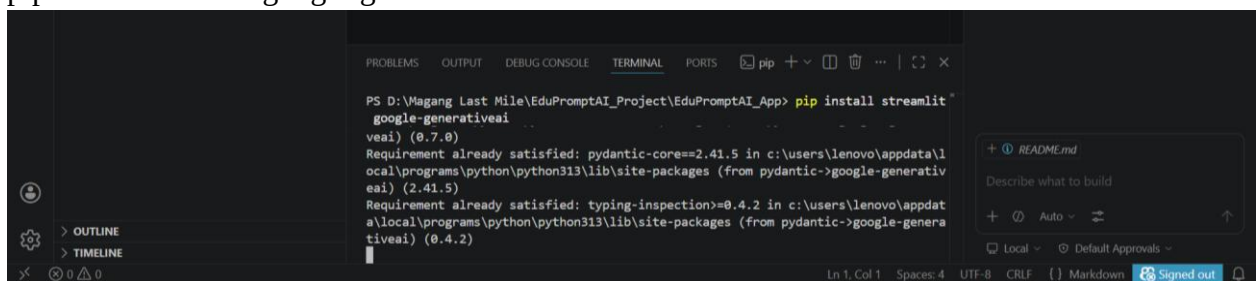


Figure 1. 4 Initial Gemini package installation process

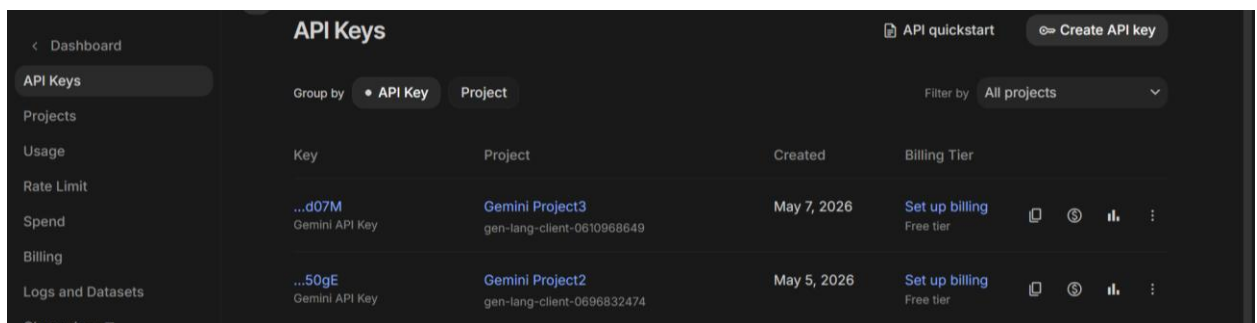


Figure 1. 5 Initial Google AI Studio API setup

1.2.2 Final API Migration Preparation

After several rounds of testing, the project encountered API quota limitations while using Google Gemini API. Since the application required repeated testing for structured explanations, study plans, quizzes, evaluator feedback, and prompt comparison, the project decided to migrate the final implementation to OpenRouter API.

OpenRouter was selected because it provides flexible access to multiple Large Language Models through one API endpoint. In the final version of Nazsya AI Tutor, the system uses the DeepSeek Chat model through OpenRouter while maintaining the same prompt engineering workflow developed during the Gemini experimentation phase.

The migration did not change the main purpose of the project. Instead, it improved the stability of the final implementation and allowed the system to continue generating structured educational outputs consistently.

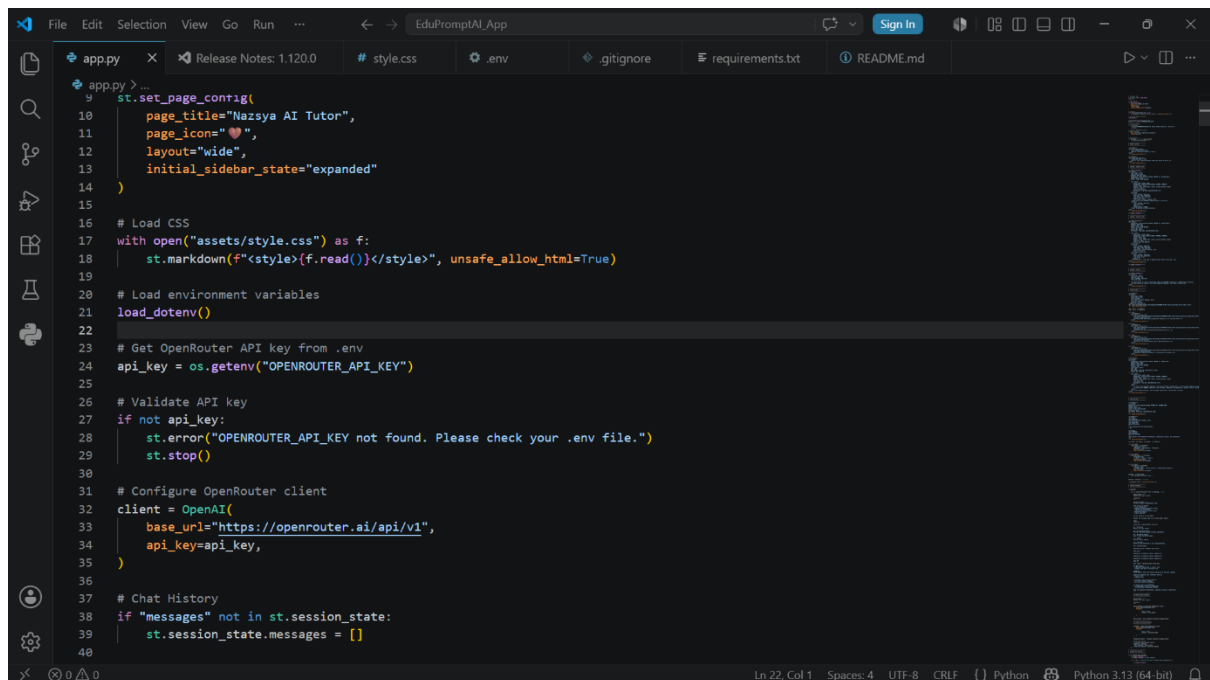
The following table summarizes the difference between the initial development setup and the final implementation setup used in Nazsya AI Tutor.

Component	Initial Development	Final Implementation
AI Provider	Google AI Studio	OpenRouter
AI Model	Gemini 2.5 Flash Lite	DeepSeek Chat
API Library	Google Generative AI SDK	OpenAI-Compatible SDK
Frontend	Streamlit	Streamlit
Backend	Python	Python
API Key Management	.env	.env
Purpose	Prompt exploration and prototype testing	Final application implementation

This migration allowed the project to continue using the same prompt engineering workflow while improving testing flexibility and final system stability.

Final API Installation Command

pip install openai python-dotenv



```

1  app.py > ...
2  st.set_page_config(
3      page_title="Nazsya AI Tutor",
4      page_icon="❤️",
5      layout="wide",
6      initial_sidebar_state="expanded"
7  )
8
9  # Load CSS
10 with open("assets/style.css") as f:
11     st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
12
13 # Load environment variables
14 load_dotenv()
15
16 # Get OpenRouter API key from .env
17 api_key = os.getenv("OPENROUTER_API_KEY")
18
19 # Validate API key
20 if not api_key:
21     st.error("OPENROUTER_API_KEY not found. Please check your .env file.")
22     st.stop()
23
24 # Configure OpenRouter client
25 client = OpenAI(
26     base_url="https://openrouter.ai/api/v1",
27     api_key=api_key,
28 )
29
30 # Chat History
31 if "messages" not in st.session_state:
32     st.session_state.messages = []
33

```

Figure 1. 6 Final OpenRouter API configuration in app.py

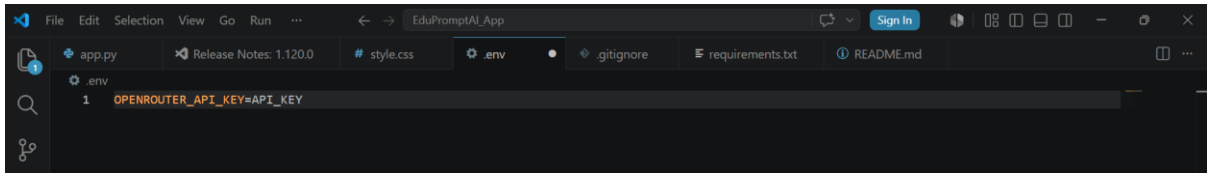


Figure 1. 7 OpenRouter API key configuration using .env file

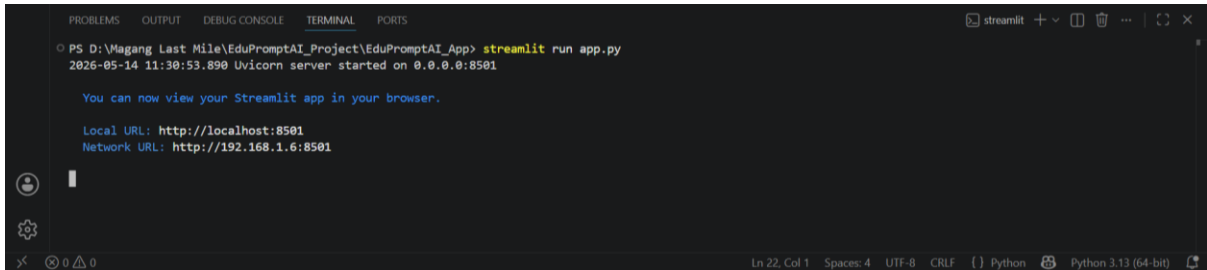


Figure 1. 8 Successful final application execution using OpenRouter API

1.2.3 Additional System Configuration Improvement

- **Environment Variable Integration for API Security**

To improve system security and follow software development best practices, the API key configuration was stored in environment variables using a .env file.

The python-dotenv library was implemented to securely load API credentials into the Streamlit application environment.

This improvement reduced the risk of API key exposure and improved project maintainability and deployment readiness.

- **Implementation**

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("OPENROUTER_API_KEY")

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
)
```

- **.env Configuration**

```
OPENROUTER_API_KEY=YOUR_API_KEY
```

- **.gitignore Configuration**

```
# Environment variables
.env

# Python cache
__pycache__/
*.pyc
```

```

# Virtual environment
venv/
.env/

# VS Code settings
.vscode/

# Streamlit cache
.streamlit/

```



Figure 1.9 Environment variable integration using .env configuration

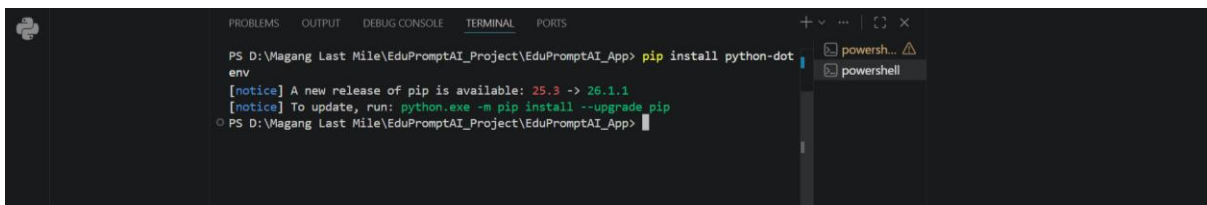


Figure 1.10 Install python-dotenv

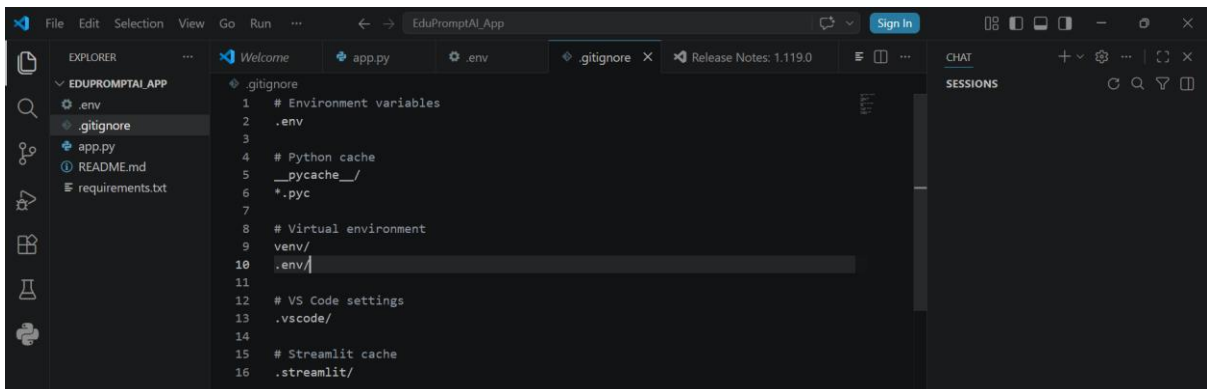


Figure 1.11 gitignore configuration for API security

1.3 Basic Prompt Exploration

After completing the initial API setup using Google Gemini, the project conducted prompt exploration experiments to understand how different prompt phrasing affects AI-generated responses. This stage was part of the early prototype development before the final implementation was migrated to OpenRouter API.

Several prompt variations were tested, including basic prompts, beginner-friendly prompts, and step-by-step explanation prompts.

The purpose of this experiment was to evaluate how prompt engineering techniques can improve response clarity, structure, and educational quality.

Example Prompts

1. Explain photosynthesis.
2. Explain photosynthesis for beginners.
3. Explain photosynthesis step-by-step for a 10-year-old student.

1.3.1 Initial Prompt Testing Using Gemini API

The following code snippet demonstrates the initial implementation of Streamlit and Gemini API integration for the Nazsya AI Tutor prototype.

```
import streamlit as st
import google.generativeai as genai

genai.configure(api_key="YOUR_API_KEY")

model = genai.GenerativeModel("gemini-1.5-flash")

st.title("EduPrompt AI")

question = st.text_input("Ask a question")

if st.button("Generate Answer"):
    response = model.generate_content(question)
    st.write(response.text)
```

```
app.py > ...
1  import streamlit as st
2  import google.generativeai as genai
3
4  # Configure API
5  genai.configure(api_key="API_KEY")
6
7  # Load Gemini model
8  model = genai.GenerativeModel("gemini-1.5-flash")
9
10 # Streamlit UI
11 st.title("EduPrompt AI")
12 st.subheader("AI Smart Tutor using Advanced Prompt Engineering Techniques")
13
14 question = st.text_input("Ask a question")
15
16 if st.button("Generate Answer"):
17     response = model.generate_content(question)
18     st.write(response.text)
```

Figure 1. 12 Initial Streamlit and Gemini API Integration Code

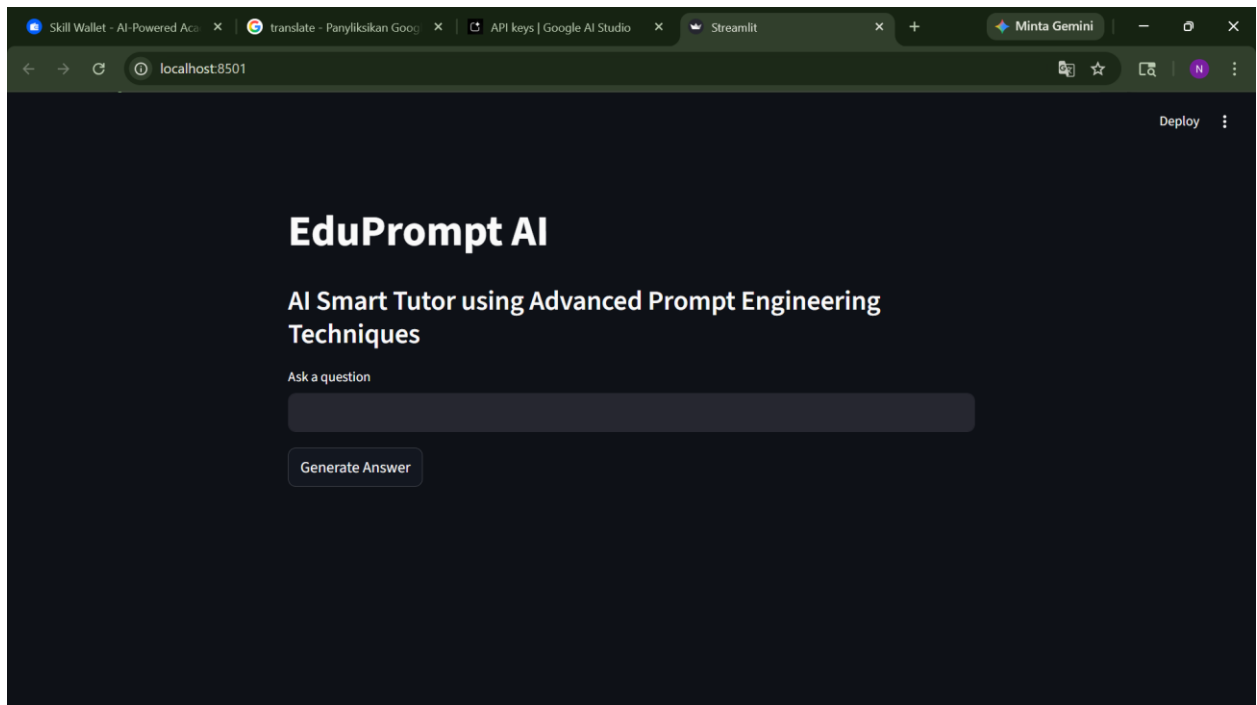


Figure 1. 13 *Initial Prompt Testing*

1.3.2 Model Configuration Error and Resolution

During the initial prompt testing phase, the project encountered an API model configuration error while using the Gemini 1.5 Flash model. The error occurred because the selected model version was not supported for the current API version.

To resolve the issue, the model configuration was updated from Gemini 1.5 Flash to Gemini 2.5 Flash Lite, which successfully restored the functionality of the AI response generation system.

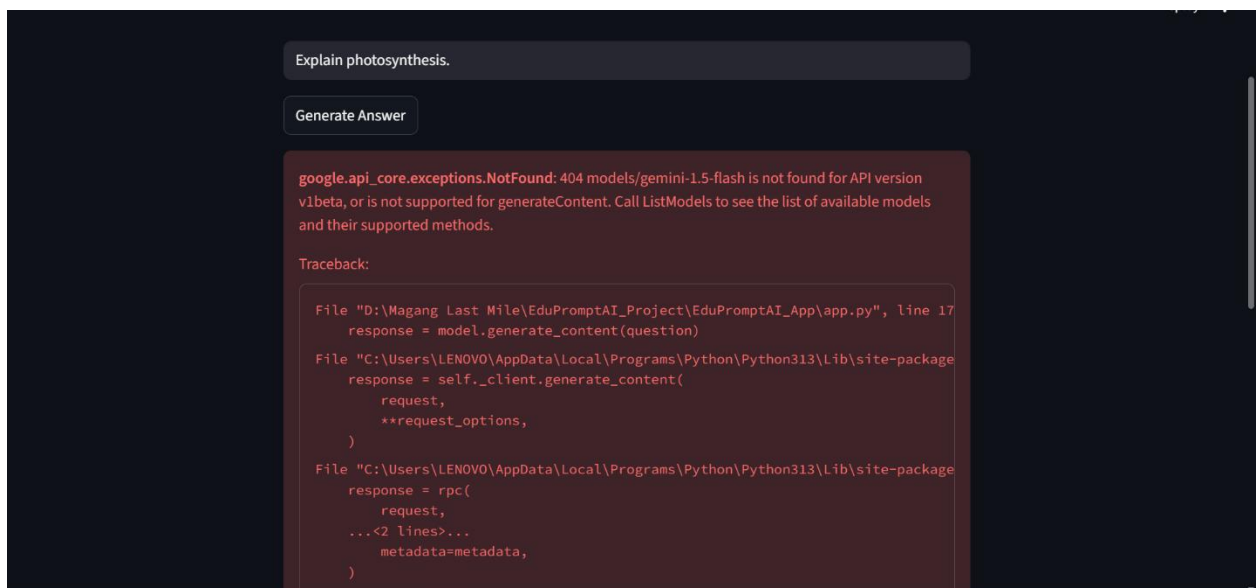


Figure 1. 14 *API model configuration error during initial prompt testing*

1. Explain photosynthesis.

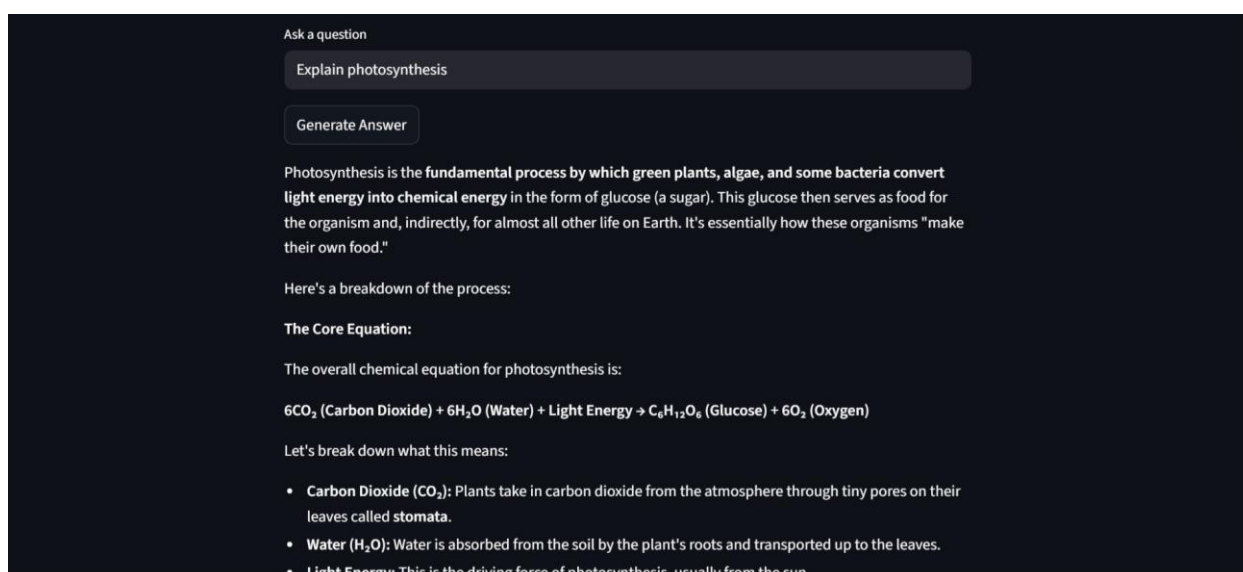
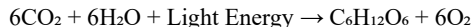


Figure 1. 15 Successful AI-generated educational response after updating the Gemini model configuration

Summary of AI-generated response:

Photosynthesis is the process by which green plants, algae, and some bacteria convert light energy into chemical energy in the form of glucose. This process uses carbon dioxide, water, and sunlight, while oxygen is released as a byproduct.

The overall equation of photosynthesis is:



Photosynthesis mainly occurs in the chloroplasts, which contain chlorophyll to absorb sunlight. The process happens in two main stages: the light-dependent reactions and the Calvin Cycle. In the light-dependent reactions, sunlight is captured and converted into energy molecules such as ATP and NADPH, while oxygen is released from water splitting. In the Calvin Cycle, ATP and NADPH are used to convert carbon dioxide into glucose.

Photosynthesis is important because it produces oxygen, provides food and energy for plants, supports food chains, and helps regulate carbon dioxide levels in the atmosphere. Therefore, photosynthesis is one of the most essential biological processes for life on Earth.

The initial prompt variation experiment demonstrated that different prompt formulations significantly influenced the structure, tone, and complexity of the AI-generated responses.

Basic prompts produced more formal and technical explanations, while beginner-oriented prompts generated simpler and more user-friendly educational content.

2. Explain photosynthesis for beginners.

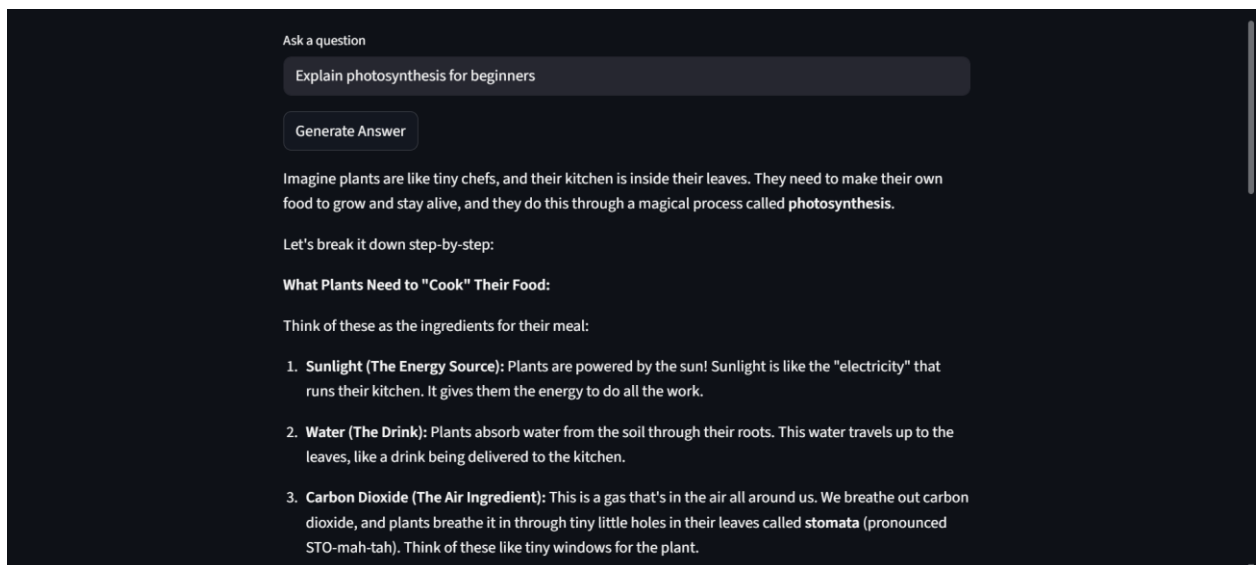


Figure 1. 16 Comparison of AI-generated responses using different prompt variations and instructional styles

Summary of AI-generated response:

The beginner prompt generated a simple and friendly explanation of photosynthesis using an analogy of plants as tiny chefs. The response explained that plants need sunlight, water, and carbon dioxide to make their own food.

The explanation described chlorophyll as the green pigment that captures sunlight inside the leaves. With the help of sunlight energy, plants combine water and carbon dioxide to produce glucose as food and release oxygen into the air.

This response was easier to understand because it used simple language, everyday comparisons, and a friendly learning style. It showed that beginner-oriented prompting can make scientific concepts more accessible for younger or beginner learners.

3. Explain photosynthesis step-by-step for a 10-year-old student.

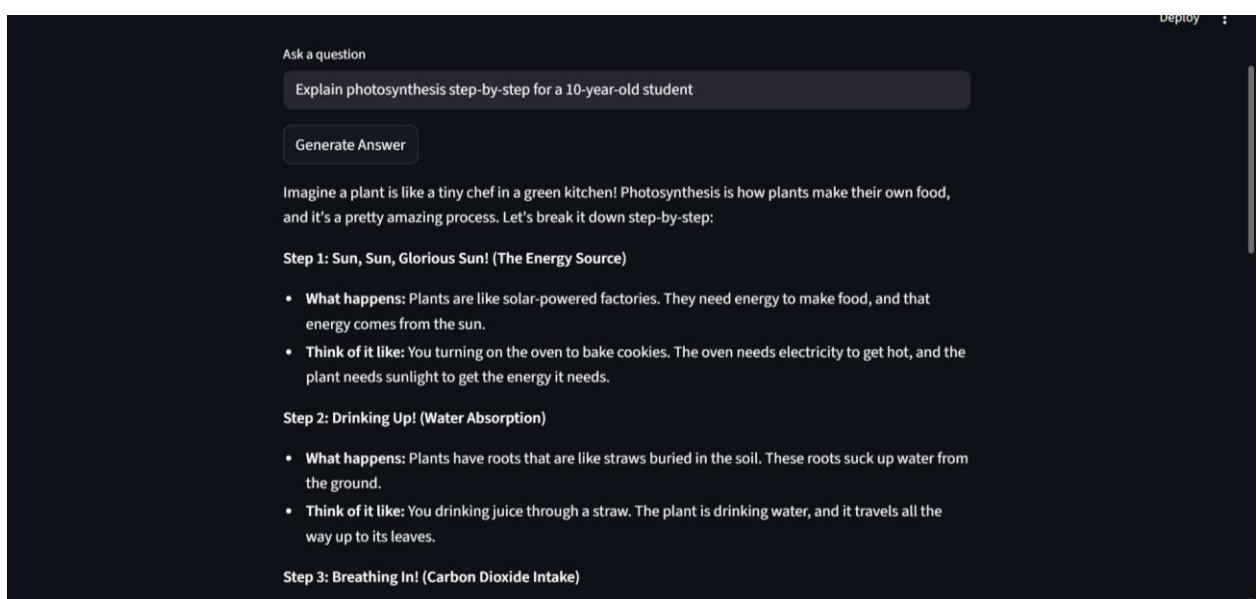


Figure 1. 17 Comparison of AI-generated responses using different prompt variations and instructional styles

Summary of AI-generated response:
The step-by-step prompt generated a beginner-friendly explanation of photosynthesis for a 10-year-old student. The response compared plants to tiny chefs in a green kitchen and explained the process in simple stages.
The explanation described how plants use sunlight as an energy source, absorb water through their roots, take in carbon dioxide through stomata, and use chlorophyll to capture sunlight. These ingredients are then used to produce glucose as food, while oxygen is released back into the air.
This response was more structured than the beginner prompt because it explained photosynthesis step by step. It also used playful analogies and simple comparisons, making the concept easier for young learners to understand.

Prompt Type	Prompt Example	Response Characteristics
Basic Prompt	Explain photosynthesis.	Formal and technical explanation
Beginner Prompt	Explain photosynthesis for beginners.	Simpler and more user-friendly explanation
10-Year-Old Prompt	Explain photosynthesis step-by-step for a 10-year-old student.	Step-by-step and playful explanation style

Based on the prompt comparison experiment, the project observed that prompt phrasing significantly influenced the tone, complexity, and instructional style of the AI-generated responses.

More specific prompts produced more structured, beginner-friendly, and engaging educational explanations, demonstrating the effectiveness of prompt engineering techniques in educational AI systems.

1.3.3 Final Basic Prompt Implementation in OpenRouter Version

After the initial prompt exploration using Gemini, the same basic prompting concept was later integrated into the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final application, the basic prompt was used to generate a simple response without additional structure, role instruction, learner level adaptation, study plan generation, or quiz generation. This basic response was displayed alongside the optimized response to show the difference between simple prompting and advanced prompt engineering.

The final basic prompt used in the application was:

Explain this topic briefly:

{question}

This implementation helped demonstrate that basic prompts can generate useful responses, but they are generally less structured and less personalized than optimized prompts.


```

392
393
394 # =====
395 # GENERATE BASIC RESPONSE
396 # =====
397
398 basic_prompt = f"""
399 Explain this topic briefly:
400
401 {question}
402 """
403
404 basic_response = client.chat.completions.create(
405     model="deepseek/deepseek-chat",
406     messages=[
407         {
408             "role": "user",
409             "content": basic_prompt
410         }
411     ]
412 )
413
414 basic_output = basic_response.choices[0].message.content
415
416 # =====
417 # GENERATE OPTIMIZED RESPONSE

```

PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
Network URL: http://192.168.1.6:8501

Figure 1. 18 Final basic prompt implementation using OpenRouter API

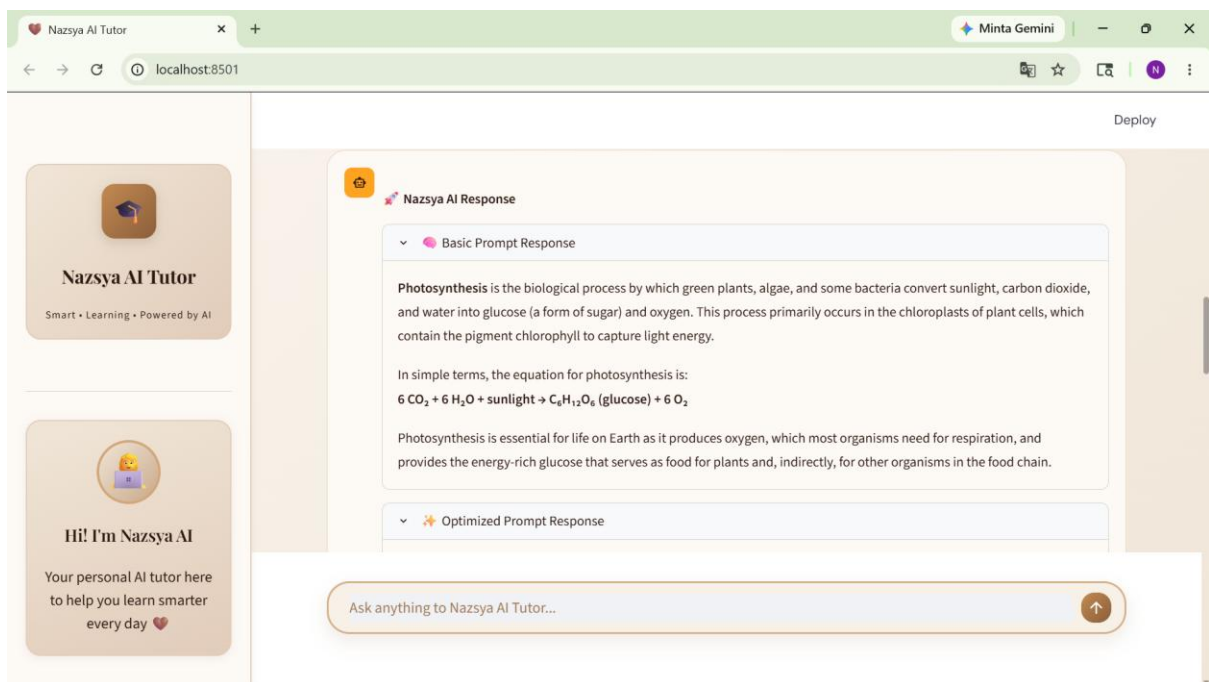


Figure 1. 19 Basic prompt response result in the final Nazsya AI Tutor application

The basic prompt generated a short and general explanation of photosynthesis. The response included the definition, the chemical equation, and the importance of photosynthesis for life on Earth. However, the response did not include structured sections, learner-level adaptation, study plan, quiz, or evaluator feedback. This shows that basic prompting can generate useful information, but it is still limited compared to optimized prompting.

Aspect	Observation
Prompt Type	Basic Prompt

Instruction	Explain this topic briefly
Topic Tested	Photosynthesis
Output Style	Short and general explanation
Strength	Clear, simple, and direct
Limitation	No structured format, no study plan, no quiz, no learner-level adaptation
Next Improvement	Structured and optimized prompting

Based on the final basic prompt testing, the project concluded that basic prompting can generate useful educational responses. However, the output is still limited because it does not automatically provide structured sections, learner-level adjustment, study plans, quizzes, or evaluator feedback. Therefore, the next development stage focused on designing structured and optimized prompts, which are discussed in M2.

M2. Structured Answer Generator and Multi-Level Explanation Generator

2.1 Structured Answer Generator Design

In this phase, the project enhanced the response generation mechanism by implementing a structured answer format within the prompt design. The structured answer generator was first tested during the initial Gemini-based prototype and later integrated into the final OpenRouter-based version of Nazsya AI Tutor.

The purpose of this enhancement was to improve the clarity, consistency, and educational quality of AI-generated responses. Instead of generating free-form answers, the AI was instructed to organize its response into predefined sections.

In the initial prototype, the structured format consisted of Definition, Detailed Explanation, Example, and Summary sections. In the final implementation, the structure was expanded to include Definition, Detailed Explanation, Real-World Example, Summary, Study Plan, and Interactive Quiz.

This structured prompting approach allowed the system to provide more organized and learner-friendly educational content.

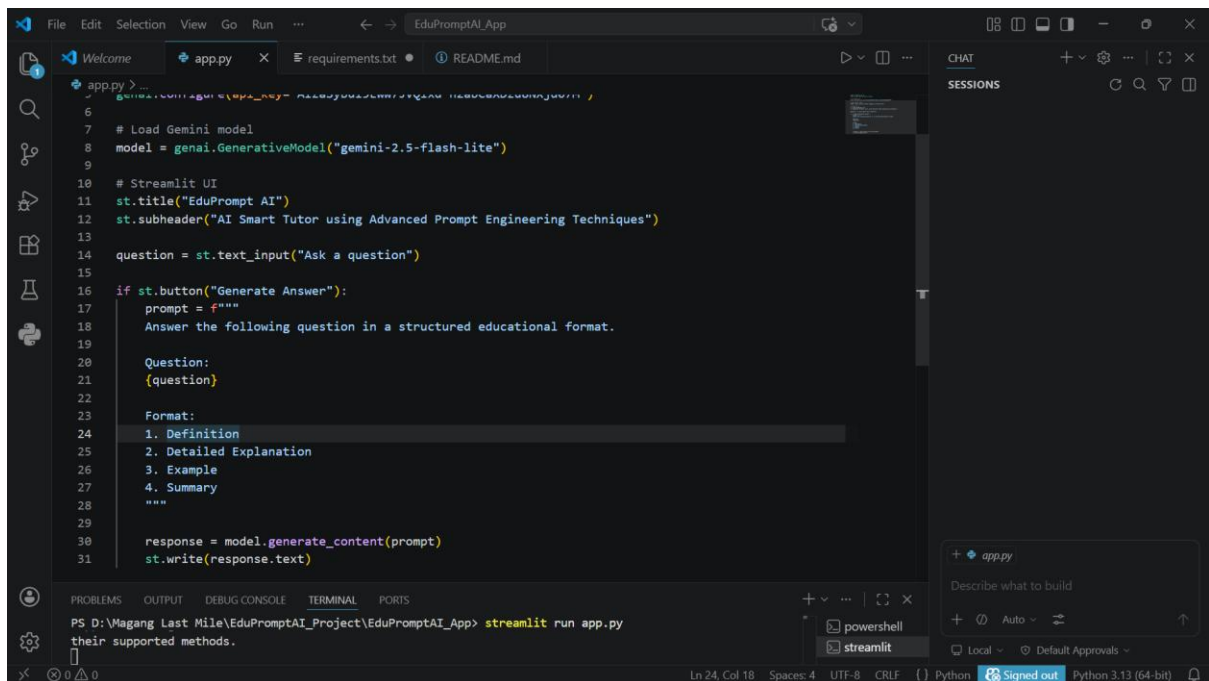
2.1.1 Initial Structured Prompt Design Using Gemini API

The following code shows the initial structured prompt implementation during the Gemini-based prototype stage.

```
prompt = f"""
Answer the following question in a structured educational format.

Question:
{question}

Format:
1. Definition
2. Detailed Explanation
3. Example
4. Summary
"""
response = model.generate_content(prompt)
```



```
app.py> ...
6
7 # Load Gemini model
8 model = genai.GenerativeModel("gemini-2.5-flash-lite")
9
10 # Streamlit UI
11 st.title("EduPrompt AI")
12 st.subheader("AI Smart Tutor using Advanced Prompt Engineering Techniques")
13
14 question = st.text_input("Ask a question")
15
16 if st.button("Generate Answer"):
17     prompt = f"""
18     Answer the following question in a structured educational format.
19
20     Question:
21     {question}
22
23     Format:
24     1. Definition
25     2. Detailed Explanation
26     3. Example
27     4. Summary
28     """
29
30     response = model.generate_content(prompt)
31     st.write(response.text)
```

PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
their supported methods.

Figure 2. 1 Structured prompt implementation for educational response generation..

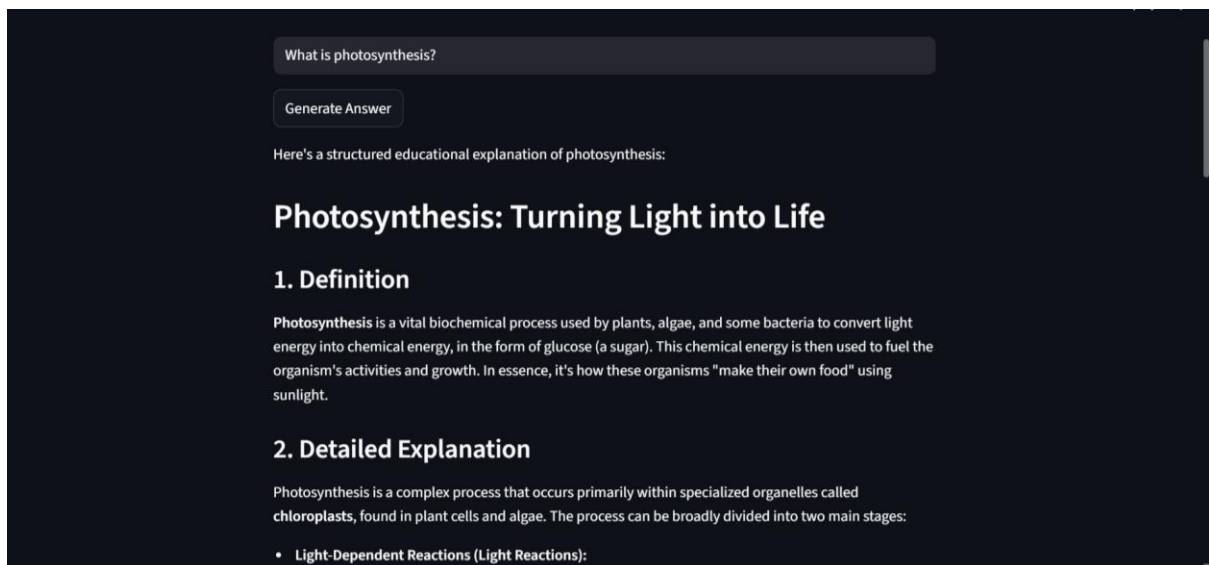


Figure 2. 2 Structured AI-generated educational response using prompt engineering techniques.

The structured prompt generated an educational response about photosynthesis using four main sections: Definition, Detailed Explanation, Example, and Summary. The response was more organized than the basic prompt because it separated the explanation into clear learning sections. This made the content easier to read and more suitable for educational use.

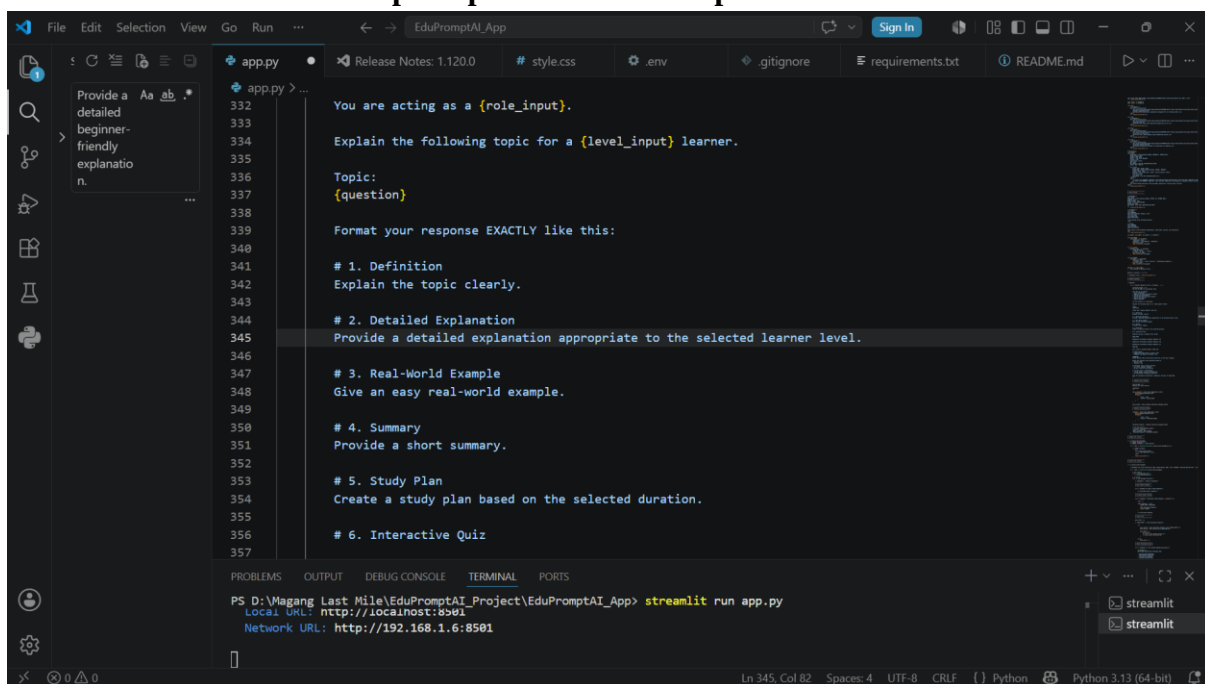
After the structured prompt was tested in the initial Gemini-based prototype, the same structured prompting concept was implemented in the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final implementation, the structured answer format was expanded to support more complete educational outputs. The AI was instructed to generate responses using the following sections:

1. Definition
2. Detailed Explanation
3. Real-World Example
4. Summary
5. Study Plan
6. Interactive Quiz

This final structured format improved the educational usefulness of the application because the response did not only explain the topic, but also provided learning guidance and practice questions.

2.1.2 Final Structured Prompt Implementation in OpenRouter Version



```
app.py
332 You are acting as a {role_input}.
333
334 Explain the following topic for a {level_input} learner.
335
336 Topic:
337 {question}
338
339 Format your response EXACTLY like this:
340
341 # 1. Definition
342 Explain the topic clearly.
343
344 # 2. Detailed Explanation
345 Provide a detailed explanation appropriate to the selected learner level.
346
347 # 3. Real-World Example
348 Give an easy real-world example.
349
350 # 4. Summary
351 Provide a short summary.
352
353 # 5. Study Plan
354 Create a study plan based on the selected duration.
355
356 # 6. Interactive Quiz
357
```

```
PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
Local URL: http://localhost:8501
Network URL: http://192.168.1.6:8501
```

Figure 2. 3 Final structured prompt format implementation using OpenRouter API

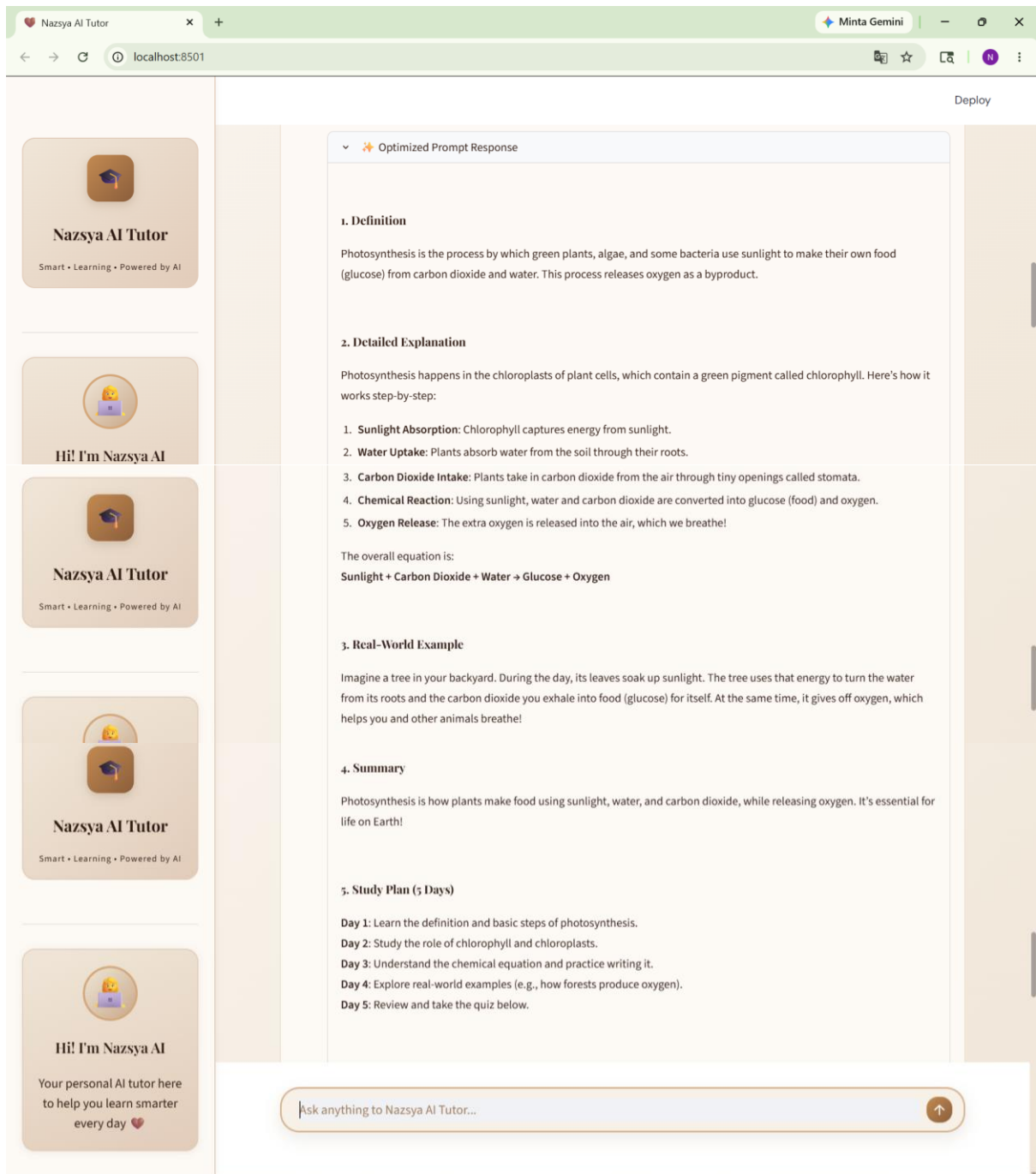


Figure 2. 4 Structured educational response in the final Nazsya AI Tutor application

Aspect	Initial Structured Prompt	Final Structured Prompt
API Provider	Gemini API	OpenRouter API
Model	Gemini 2.5 Flash Lite	DeepSeek Chat
Output Sections	Definition, Detailed Explanation, Example, Summary	Definition, Detailed Explanation, Real-World Example, Summary, Study Plan, Interactive Quiz
Purpose	Test structured response format	Generate complete educational learning output
Improvement	More organized than basic prompt	More complete and suitable for AI tutor system

Based on the structured answer generator testing, the project concluded that structured prompting significantly improved the organization and readability of AI-generated educational content. The final implementation expanded the structure by adding study plan and interactive quiz sections, making the system more suitable as an AI-powered learning assistant.

2.2 Multi-Level Explanation Generator Design

In this phase, the project implemented a multi-level explanation mechanism to adapt AI-generated educational content based on the learner's knowledge level. This feature was first tested in the initial Gemini-based prototype and later integrated into the final OpenRouter-based version of Nazsya AI Tutor.

The system was designed to provide explanations for Beginner, Intermediate, and Advanced learners by dynamically modifying the prompt instructions according to the selected learning level.

This approach improved personalization and enabled the AI tutor to generate explanations with varying levels of complexity, tone, terminology, and instructional style.

2.2.1 Initial Multi-Level Prompting Implementation Using Gemini API

```
level = st.selectbox(
    "Choose explanation level",
    ["Beginner", "Intermediate", "Advanced"]
)

prompt = f"""
You are an AI tutor.

Explain the following question for a {level} learner.

Question:
{question}

Format:
1. Definition
2. Detailed Explanation
3. Example
4. Summary

Make the explanation appropriate for the selected learning level.
"""
```

```

14 level = st.selectbox(
15     "Choose explanation level",
16     ["Beginner", "Intermediate", "Advanced"]
17 )
18
19 question = st.text_input("Ask a question")
20
21 if st.button("Generate Answer"):
22     prompt = f"""
23     You are an AI tutor.
24
25     Explain the following question for a {level} learner.
26
27     Question:
28     {question}
29
30     Format:
31     1. Definition
32     2. Detailed Explanation
33     3. Example
34     4. Summary
35
36     Make the explanation appropriate for the selected learning level.
37     """
38
39     response = model.generate_content(prompt)
40     st.write(response.text)

```

Figure 2. 5 Initial multi-level explanation prompt implementation using Streamlit and Gemini API

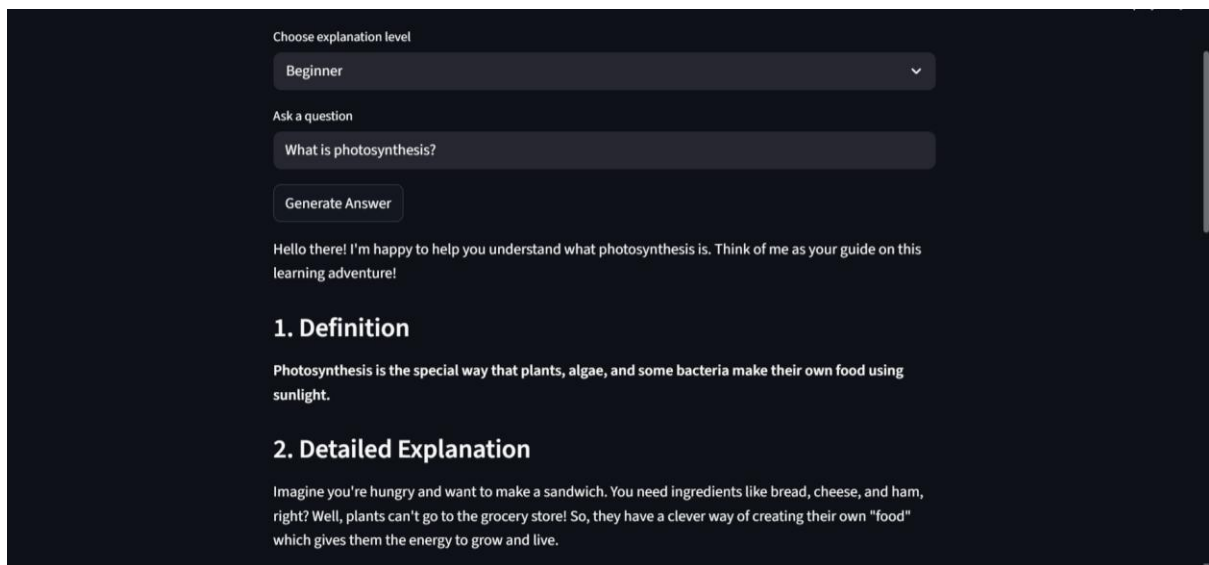


Figure 2. 6 Beginner-level educational explanation generated by Nazsya AI Tutor

Summary of AI-generated response:

The Beginner-level response explained photosynthesis using simple and friendly language. It described photosynthesis as the way plants, algae, and some bacteria make their own food using sunlight.

The response used easy analogies, such as plants making food like a person making a sandwich and chloroplasts acting like tiny kitchens inside plant cells. It explained that plants need sunlight, water, and carbon dioxide to produce glucose as food and release oxygen into the air.

This output showed that the Beginner level successfully generated a simple, friendly, and analogy-based explanation that is easier for early learners to understand.

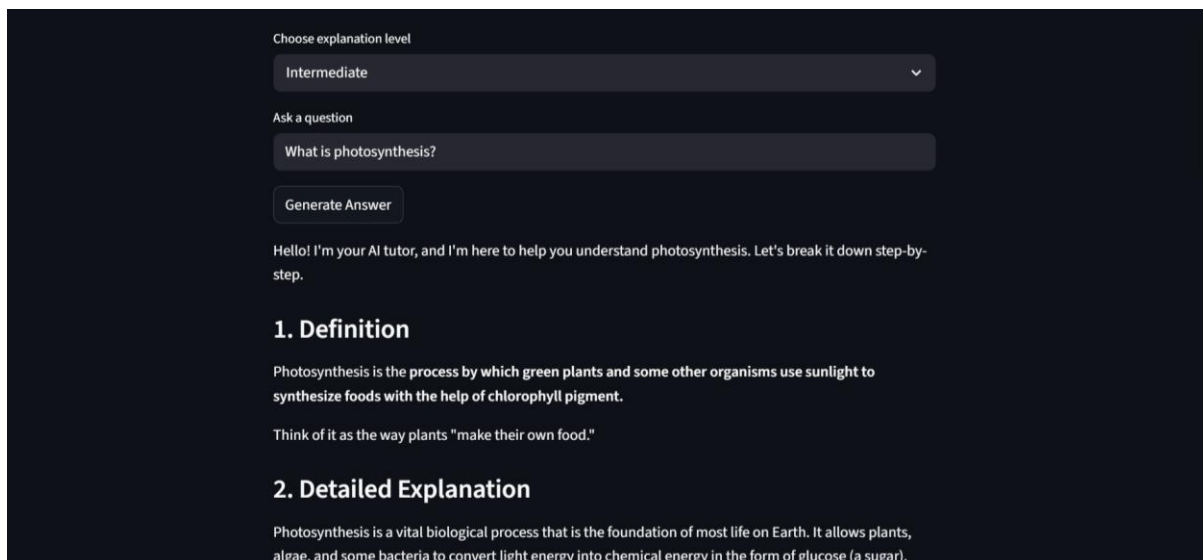


Figure 2. 7 Intermediate-level educational explanation generated by Nazsya AI Tutor

Summary of AI-generated response:

The Intermediate-level response explained photosynthesis with a more detailed and semi-academic style. It described photosynthesis as the process by which plants use sunlight, water, and carbon dioxide to produce glucose and oxygen with the help of chlorophyll.

The response included more scientific terms such as chloroplasts, stomata, light-dependent reactions, Calvin Cycle, ATP, NADPH, and the chemical equation of photosynthesis. It also explained how sunlight energy is converted into chemical energy and how glucose is produced.

This output showed that Intermediate-level prompting generated a deeper explanation than the Beginner level, while still remaining understandable for learners with basic scientific knowledge.

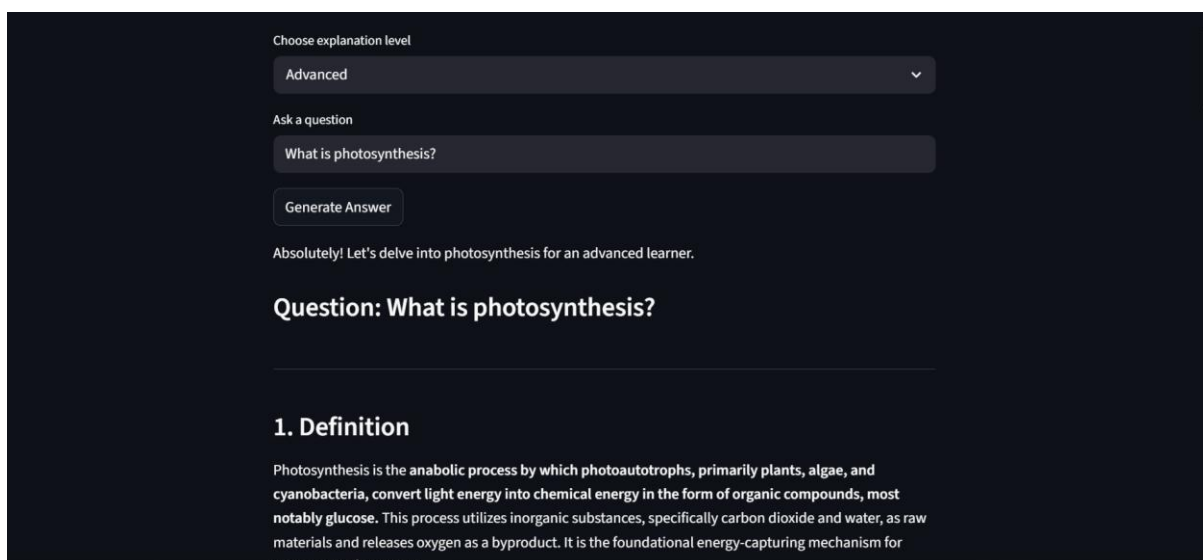


Figure 2. 8 Advanced-level educational explanation generated by Nazsya AI Tutor

Summary of AI-generated response:

The Advanced-level response explained photosynthesis in a technical and scientific way. It described photosynthesis as an anabolic process where photoautotrophs convert light energy into chemical energy by producing glucose from carbon dioxide and water.

The response explained the two main stages of photosynthesis in detail: light-dependent reactions and the Calvin Cycle. It included advanced scientific concepts such as photosystems, electron transport chain, photolysis, ATP synthesis, NADPH production, carbon fixation, RuBisCO, RuBP regeneration, and photorespiration.

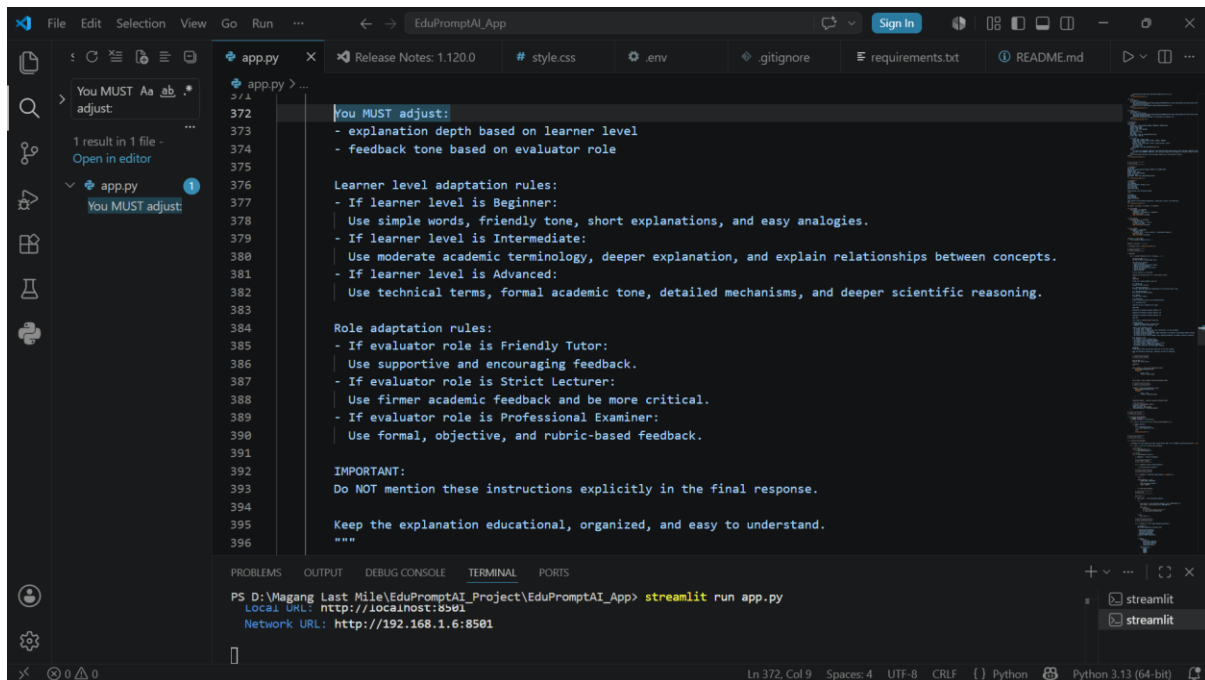
This output showed that Advanced-level prompting generated a more comprehensive and academic explanation. The response was suitable for learners who already have stronger biological knowledge and need deeper scientific reasoning.

2.2.2 Final Multi-Level Prompting Implementation in OpenRouter Version

After the multi-level explanation mechanism was tested in the Gemini-based prototype, the feature was implemented in the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final application, users can select one of three explanation levels: Beginner, Intermediate, or Advanced. The selected level is inserted dynamically into the optimized prompt so that the AI can adjust the explanation depth, terminology, examples, and instructional style.

The final implementation also expands the response format by combining multi-level explanation with structured answer generation, study plan generation, and interactive quiz generation.



```
File Edit Selection View Go Run ... EduPromptAI_App Sign In
app.py x Release Notes: 1.120.0 # style.css .env .gitignore requirements.txt README.md
You MUST adjust:
1 result in 1 file - Open in editor
app.py 1
You MUST adjust:

372 You MUST adjust:
373 - explanation depth based on learner level
374 - feedback tone based on evaluator role
375
376 Learner level adaptation rules:
377 - If learner level is Beginner:
378   Use simple words, friendly tone, short explanations, and easy analogies.
379 - If learner level is Intermediate:
380   Use moderate academic terminology, deeper explanation, and explain relationships between concepts.
381 - If learner level is Advanced:
382   Use technical terms, formal academic tone, detailed mechanisms, and deeper scientific reasoning.
383
384 Role adaptation rules:
385 - If evaluator role is Friendly Tutor:
386   Use supportive and encouraging feedback.
387 - If evaluator role is Strict Lecturer:
388   Use firmer academic feedback and be more critical.
389 - If evaluator role is Professional Examiner:
390   Use formal, objective, and rubric-based feedback.
391
392 IMPORTANT:
393 Do NOT mention these instructions explicitly in the final response.
394
395 Keep the explanation educational, organized, and easy to understand.
396 """

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
Local URL: http://localhost:8501
Network URL: http://192.168.1.6:8501

Ln 372, Col 9 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit)
```

Figure 2. 9 Learner level adaptation rules in the final optimized prompt

```

with col_input1:
    level_input = st.selectbox(
        "Explanation Level",
        ["Beginner", "Intermediate", "Advanced"],
        key="level_input",
        label_visibility="collapsed"
    )

```

Figure 2. 10 Final explanation level selection implementation in Streamlit

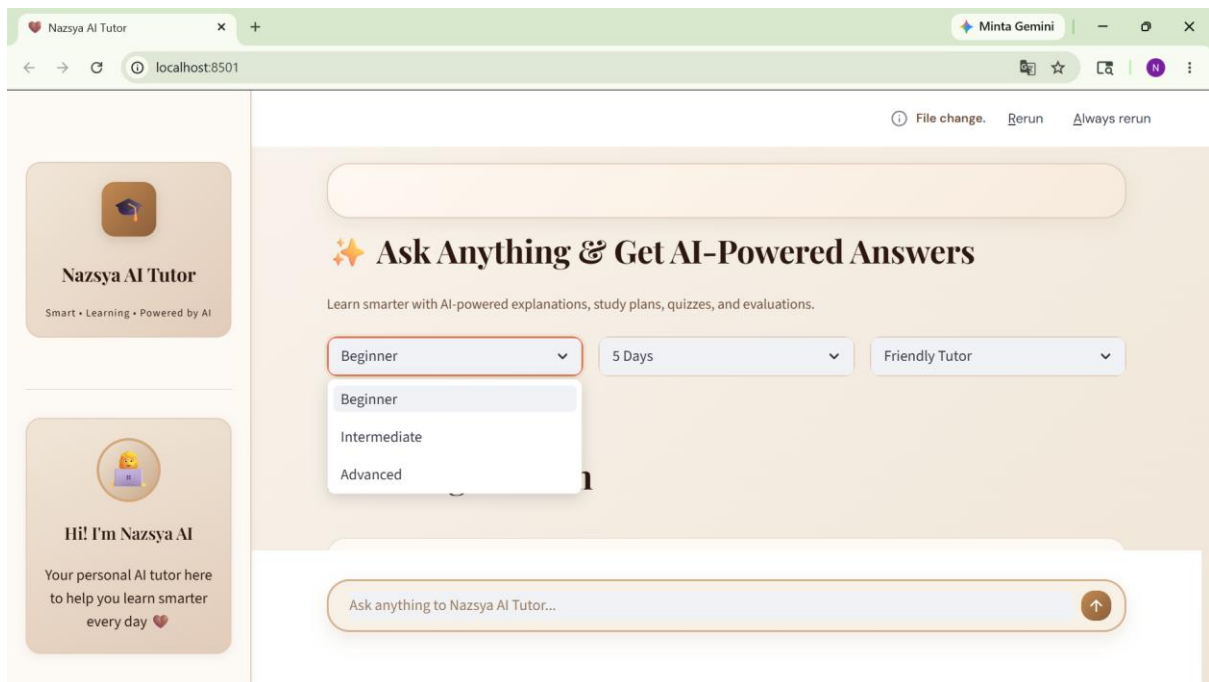


Figure 2. 11 Explanation level selection in the final Nazsya AI Tutor interface

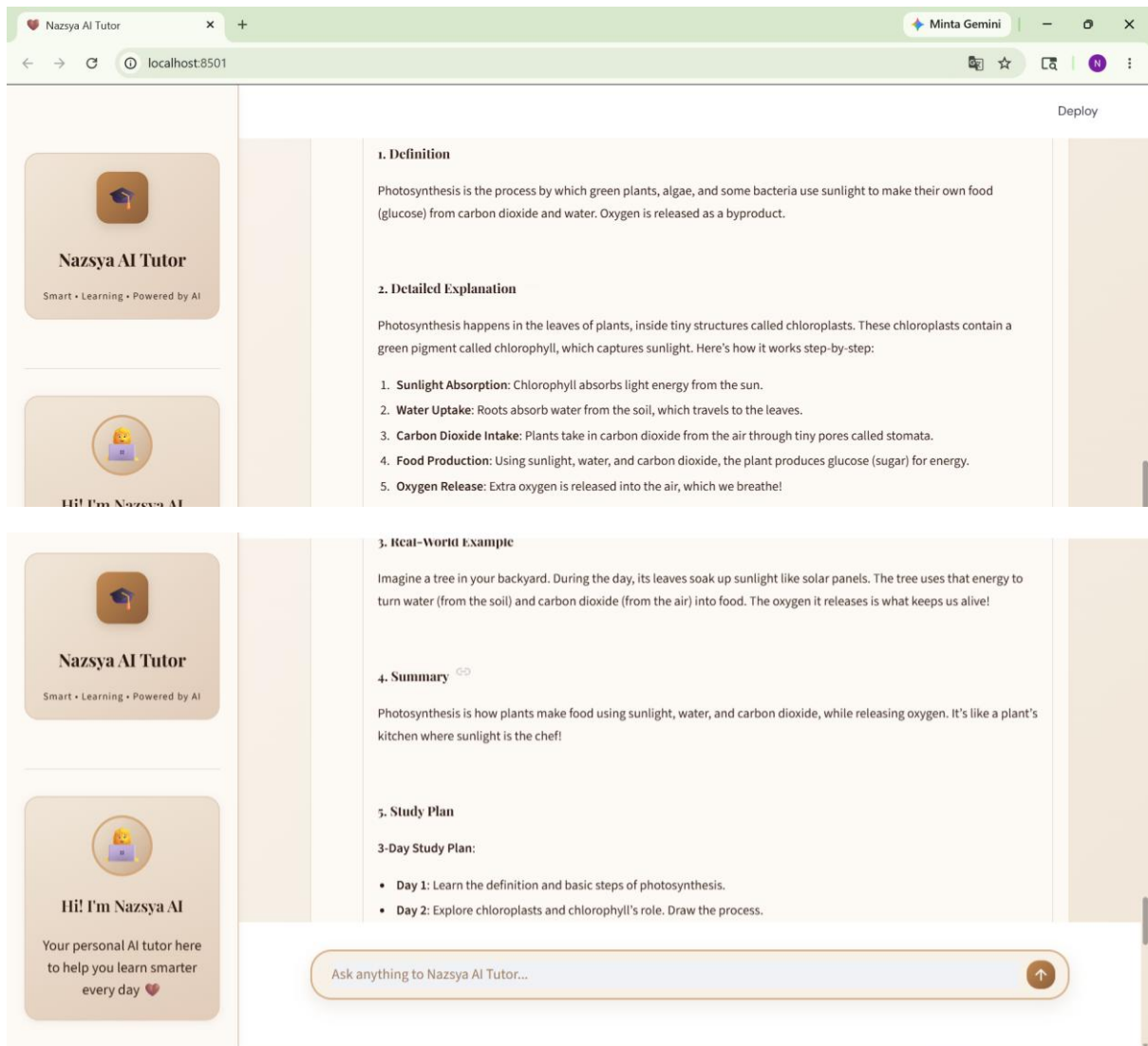


Figure 2. 12 Final Multi-Level Prompting Implementation in OpenRouter Version

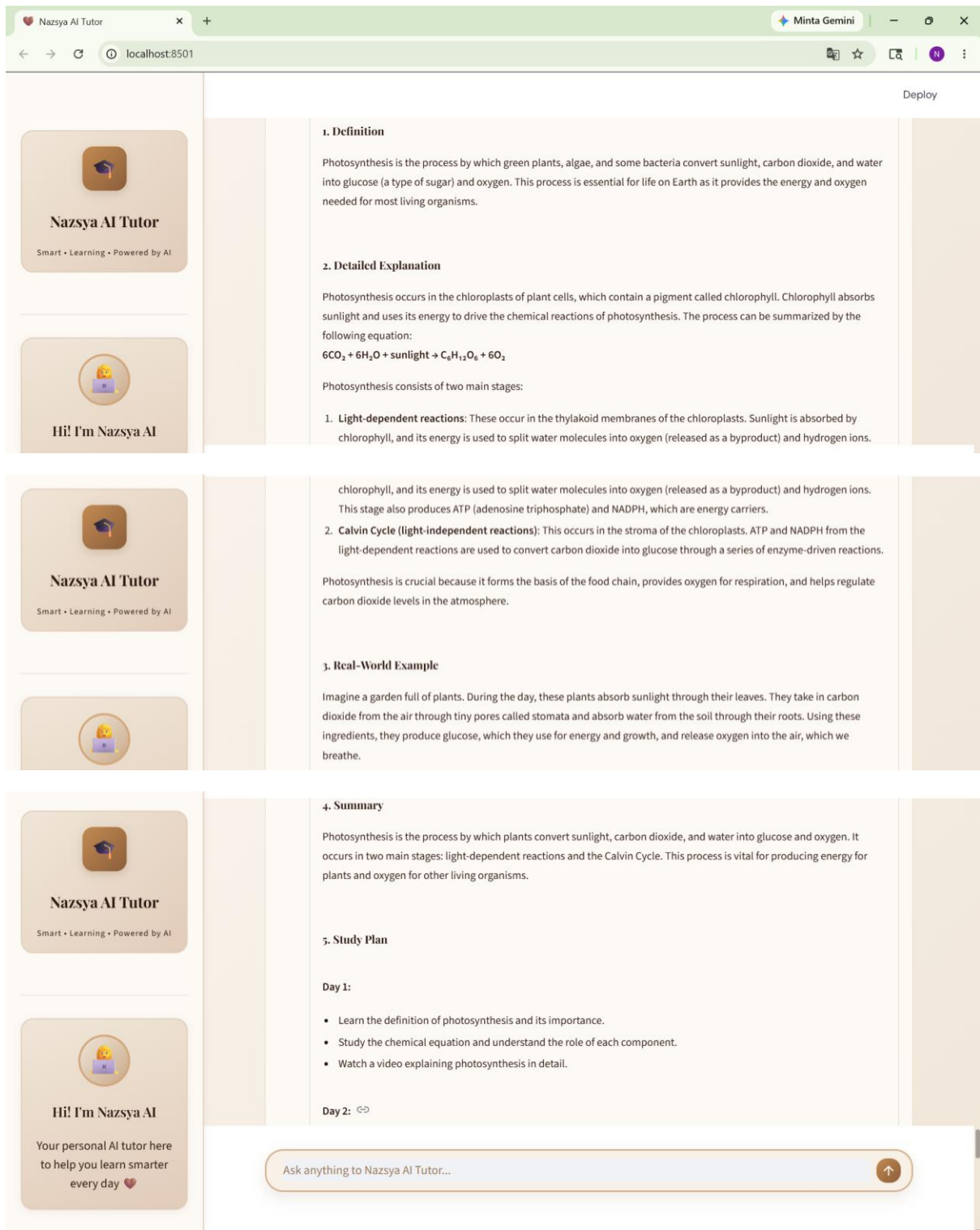


Figure 2. 13 Intermediate-level response generated in the final OpenRouter-based application

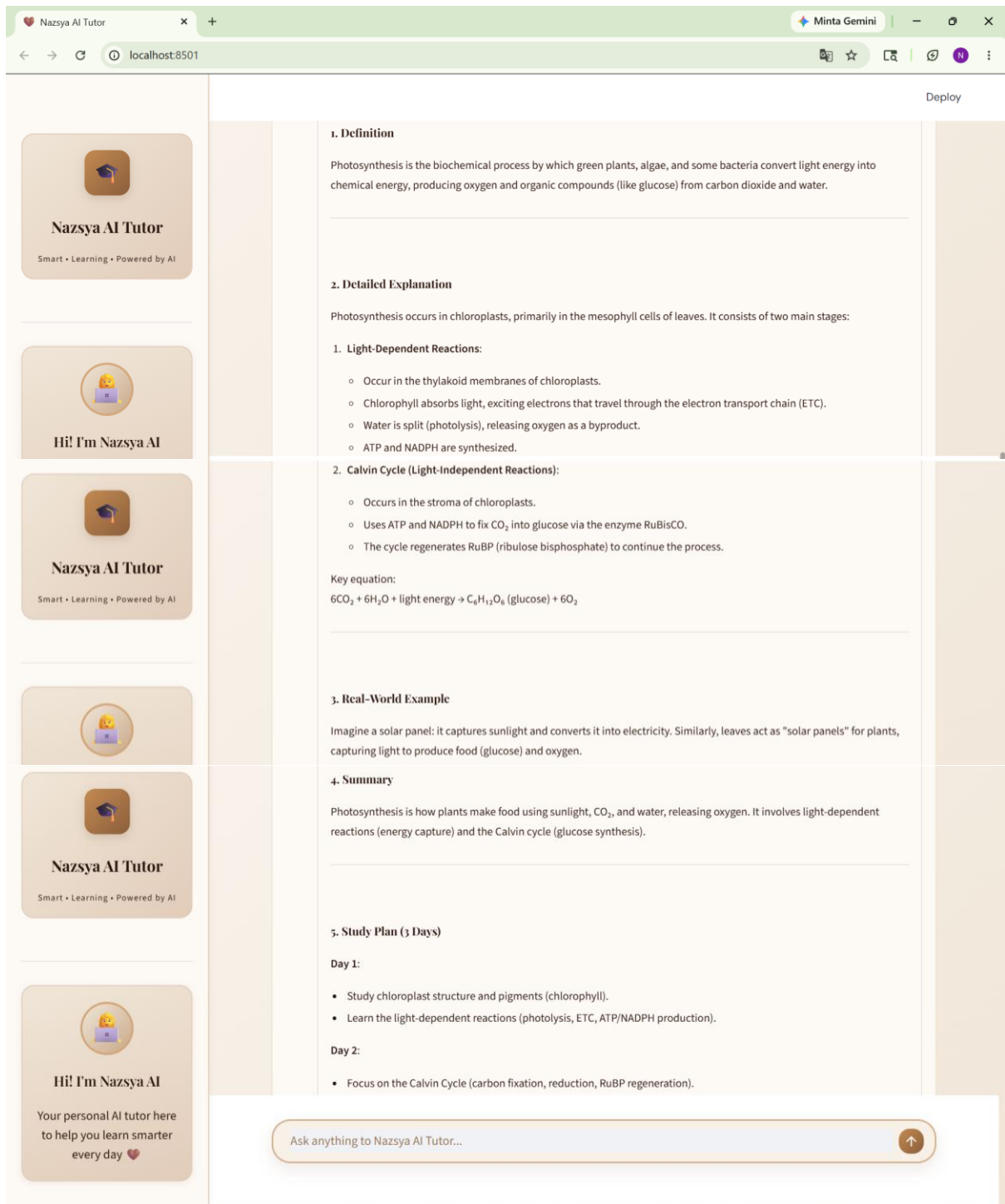


Figure 2. 14 Advanced-level response generated in the final OpenRouter-based application

2.2.3 Multi-Level Response Comparison

Learning Level	Response Characteristics	Evidence from Output
Beginner	Simple, friendly, and analogy-based explanation	Uses phrases such as plant's kitchen and sunlight as the chef
Intermediate	Detailed and semi-academic explanation	Explains light-dependent reactions, Calvin Cycle, ATP, and NADPH

Advanced	Technical and scientific explanation	Explains thylakoid membranes, electron transport chain, photolysis, RuBisCO, and RuBP
----------	--------------------------------------	---

Based on the final multi-level response testing, the project concluded that the learner level selection successfully influenced the complexity, terminology, and instructional style of the AI-generated explanation. This shows that multi-level prompting can improve personalization and make Nazsya AI Tutor more adaptable to different learner needs.

2.3 Testing and Refinement

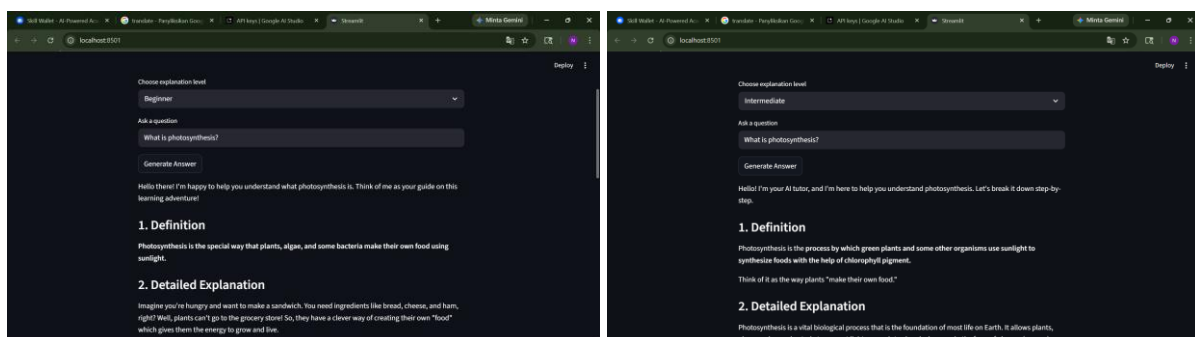
After implementing the structured answer generator and multi-level explanation system, the project conducted several testing sessions to evaluate the clarity, consistency, adaptability, and educational quality of the AI-generated responses.

The testing process was carried out during the initial Gemini-based prototype and continued in the final OpenRouter-based implementation using the DeepSeek Chat model. Several educational prompts were tested using different learning levels, including Beginner, Intermediate, and Advanced modes.

The testing results showed that the AI was able to adapt its explanation style, terminology complexity, and instructional approach according to the selected learner level. Beginner-level responses used simple and friendly explanations, Intermediate-level responses included more scientific terminology, and Advanced-level responses provided more technical and academic explanations.

During the refinement process, the project improved the prompt structure by adding clearer response sections and learner-level adaptation rules. In the initial prototype, the project also resolved a Gemini API model compatibility issue by updating the model configuration from Gemini 1.5 Flash to Gemini 2.5 Flash Lite. For the final implementation, the system was migrated to OpenRouter API with the DeepSeek Chat model to improve testing flexibility and response stability.

Overall, the refinement process improved response consistency, personalization, structure, and overall system performance.



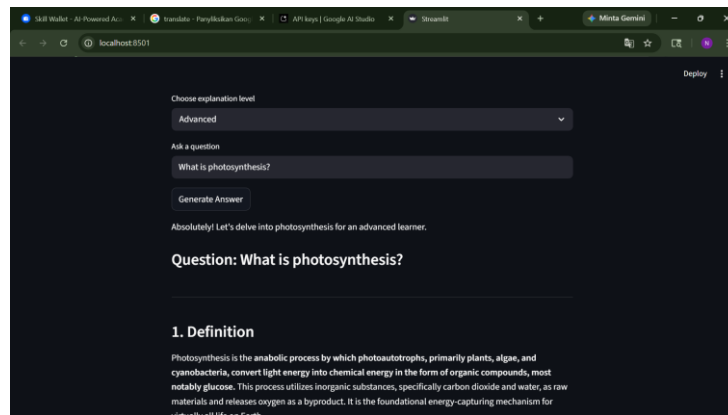


Figure 2. 15 Testing and comparison of multi-level educational responses generated by Nazsya AI Tutor

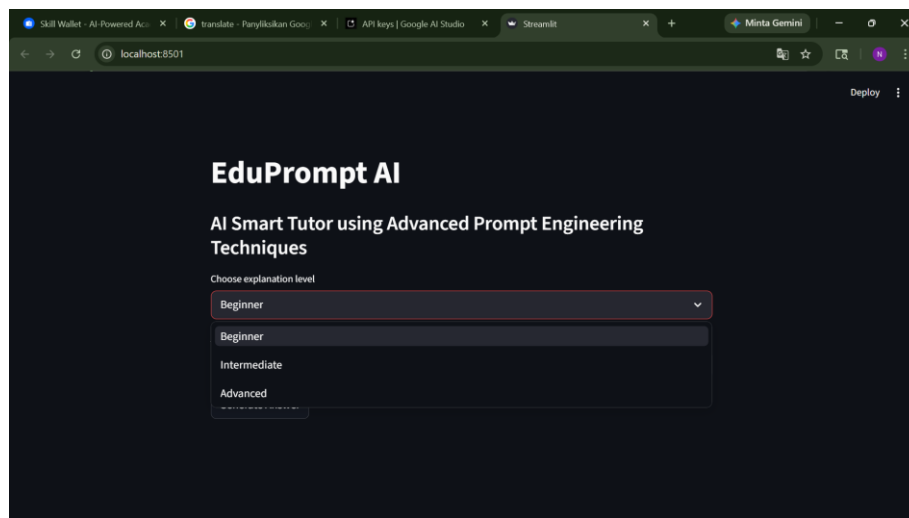


Figure 2. 16 Final interface of the multi-level explanation system in the OpenRouter-based applicatio

2.3.1 Testing Summary

Testing Observation	Refinement Action	Result
Basic responses were too general	Added structured response format	Responses became more organized
Learner levels needed clearer differences	Added Beginner, Intermediate, and Advanced adaptation rules	Responses became more personalized
Gemini 1.5 Flash caused model compatibility issue	Updated model configuration to Gemini 2.5 Flash Lite	Initial prototype worked successfully
Final testing required more stable API access	Migrated final implementation to OpenRouter API with DeepSeek Chat	Final application testing became more flexible
Output needed better educational flow	Added Definition, Detailed Explanation, Example, Summary, Study Plan, and Quiz sections	Responses became more useful for learning

Feature Tested	Testing Focus	Result
Structured Answer Format	Response organization using predefined sections	Successful
Beginner-Level Explanation	Simple, friendly, and analogy-based explanation	Successful
Intermediate-Level Explanation	More detailed explanation with scientific terminology	Successful
Advanced-Level Explanation	Technical and academic explanation style	Successful
Initial Gemini API Integration	Prototype testing and early prompt exploration	Successful after model update
Final OpenRouter API Integration	Final response generation using DeepSeek Chat	Successful
Prompt Adaptability	Dynamic adjustment based on learner level	Successful

Based on the testing results, the project concluded that the structured answer generator and multi-level explanation system worked successfully. The system was able to generate organized educational responses and adapt the explanation style according to the selected learner level. The refinement process also improved the reliability of the final application by using OpenRouter API and DeepSeek Chat for the final implementation.

M3. Study Plan Generator and Quiz Generator

3.1 Study Plan Generator Development

In this phase, the project developed a Study Plan Generator feature to help learners organize their learning process based on the selected topic and study duration. This feature was first tested in the initial Gemini-based prototype and later integrated into the final OpenRouter-based version of Nazsya AI Tutor.

The AI system was designed to automatically generate personalized study schedules containing daily learning activities, topic reviews, practice tasks, and summaries. The generated study plans were adapted according to the selected learner level and study duration.

This feature improved learning organization because users did not only receive an explanation of the topic, but also received a structured learning plan that could guide their study process over several days.

3.1.1 Initial Study Plan Prompt Integration Using Gemini API

The following code shows the initial study plan prompt integration during the Gemini-based prototype stage.

```
study_duration = st.selectbox(
    "Select study duration",
    ["3 Days", "5 Days", "7 Days"]
)

prompt = f"""
You are an AI tutor.

Explain the following question for a {level} learner.

Question:
{question}

Also create a {study_duration} study plan related to the topic.

Format:
1. Definition
2. Detailed Explanation
3. Example
4. Summary
5. Study Plan

Make the explanation appropriate for the selected learning level.
"""
```

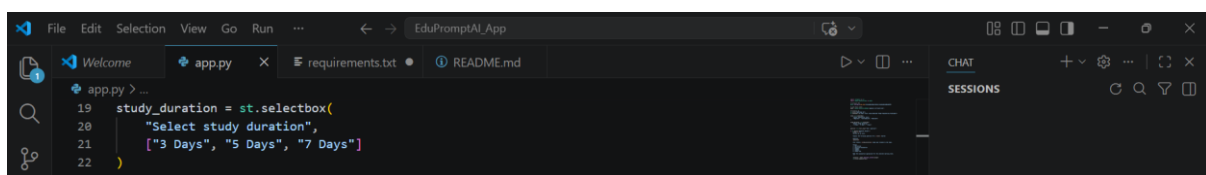


Figure 3. 1 Study plan generator prompt integration

EduPrompt AI

AI Smart Tutor using Advanced Prompt Engineering Techniques

Choose explanation level

Beginner

Select study duration

5 Days

Ask a question

What is photosynthesis? Press Enter to apply

Generate Answer

Figure 3. 2 AI-generated personalized study plan based on learner level and study duration

Generate Answer

Hello there! I'm your AI tutor, and I'm here to help you understand what photosynthesis is. Don't worry if you're new to this, we'll go through it step-by-step!

1. Definition

Photosynthesis is the amazing process that plants, algae, and some bacteria use to make their own food. Think of it like a plant's way of cooking its lunch using sunlight!

2. Detailed Explanation

Let's break down how this "cooking" process works for plants:

- **The Ingredients:** Plants need a few key things to perform photosynthesis:
 - **Sunlight:** This is the energy source, like the stove or oven in a kitchen. Plants have a special green pigment called **chlorophyll** in their leaves that captures sunlight. That's why most leaves are green!
 - **Water (H₂O):** Plants absorb water from the soil through their roots. This is like the liquid ingredient in a recipe.
 - **Carbon Dioxide (CO₂):** Plants take in carbon dioxide from the air through tiny pores on their leaves

Figure 3. 3 AI-generated personalized study plan based on learner level and study duration

Summary of AI-generated response:

The Study Plan Generator produced a 5-day learning plan about photosynthesis. The plan included daily learning activities such as learning plant needs, understanding sunlight and chlorophyll, studying water and carbon dioxide, reviewing the photosynthesis process, and connecting photosynthesis to human life.

The generated study plan showed that the AI could organize learning activities based on the selected duration. This feature helped users study the topic step by step instead of only reading a single explanation.

3.1.2 Final Study Plan Generator Implementation in OpenRouter Version

After the study plan generator was tested in the initial Gemini-based prototype, the same feature was implemented in the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final application, users can select the study duration from three options: 3 Days, 5 Days, or 7 Days. The selected duration is inserted dynamically into the optimized prompt so that the AI can generate a study plan based on the user's learning preference.

The final implementation combines the study plan generator with structured answer generation and multi-level explanation. This allows the system to provide a complete educational response consisting of explanation, example, summary, study plan, and interactive quiz.

```
with col_input2:
    duration_input = st.selectbox(
        "Study Duration",
        ["3 Days", "5 Days", "7 Days"],
        key="duration_input",
        label_visibility="collapsed"
    )
```

Figure 3. 4 Final study duration selection implementation in Streamlit

```
# 5. Study Plan
Create a {duration_input} study plan with daily learning activities, review tasks, and practice activities.
```

Figure 3. 5 Final study plan prompt implementation using OpenRouter API

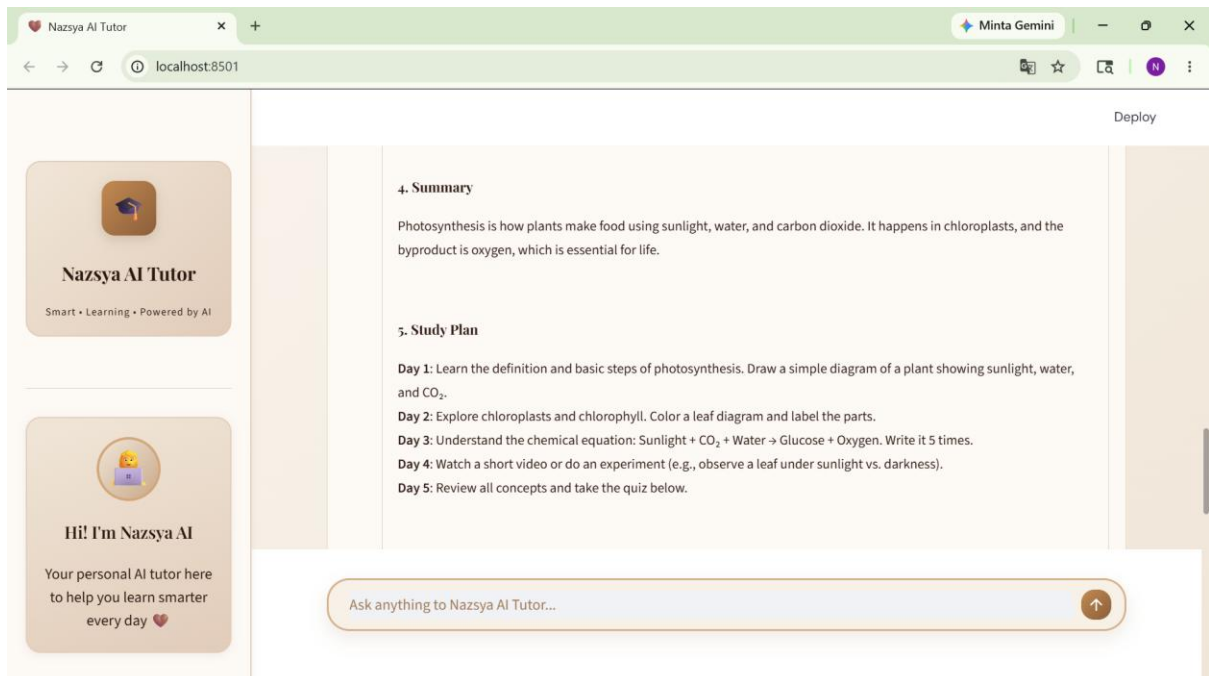


Figure 3. 6 AI-generated study plan in the final OpenRouter-based application

Aspect	Initial Study Plan Generator	Final Study Plan Generator
--------	------------------------------	----------------------------

API Provider	Gemini API	OpenRouter API
Model	Gemini 2.5 Flash Lite	DeepSeek Chat
Duration Options	3 Days, 5 Days, 7 Days	3 Days, 5 Days, 7 Days
Output Focus	Explanation with study plan	Structured explanation with personalized study plan
Improvement	Adds learning schedule	Integrated with level adaptation, structured response, and quiz

Based on the study plan generator testing, the project concluded that the feature successfully generated organized learning schedules based on the selected study duration. The final implementation improved the learning experience by combining study planning with structured explanation and learner-level adaptation.

3.2 Quiz Generator Development

In this phase, the project implemented a Quiz Generator feature to enhance learner engagement and self-assessment capabilities. This feature was first tested in the initial Gemini-based prototype and later integrated into the final OpenRouter-based version of Nazsya AI Tutor.

The AI system was instructed to automatically generate multiple-choice questions and answer keys related to the learning topic. In the final implementation, the quiz format was improved using a structured delimiter format so that the application could parse the quiz questions and display them as interactive multiple-choice questions.

This feature improved interactive learning support and enabled users to evaluate their understanding of the generated educational content directly within the application.

3.2.1 Initial Quiz Generation Prompt Integration Using Gemini API

The following code shows the initial quiz generation prompt integration during the Gemini-based prototype stage.

```
prompt = f"""
You are an AI tutor.

Explain the following question for a {level} learner.

Question:
{question}

Also create a {study_duration} study plan related to the topic.

Generate:
- 3 multiple choice questions
- Provide answer keys

Format:
1. Definition
2. Detailed Explanation
3. Example
```

4. Summary
5. Study Plan
6. Quiz

Make the explanation appropriate for the selected learning level.
 """

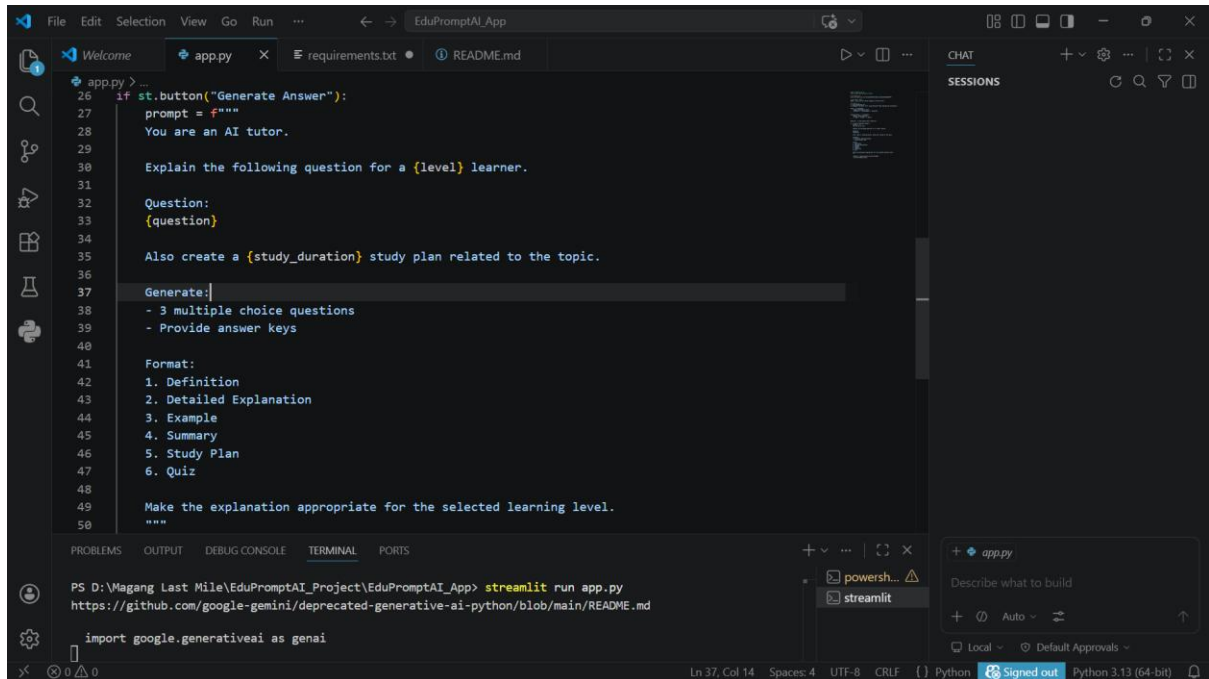


Figure 3. 7 Quiz generation prompt integration

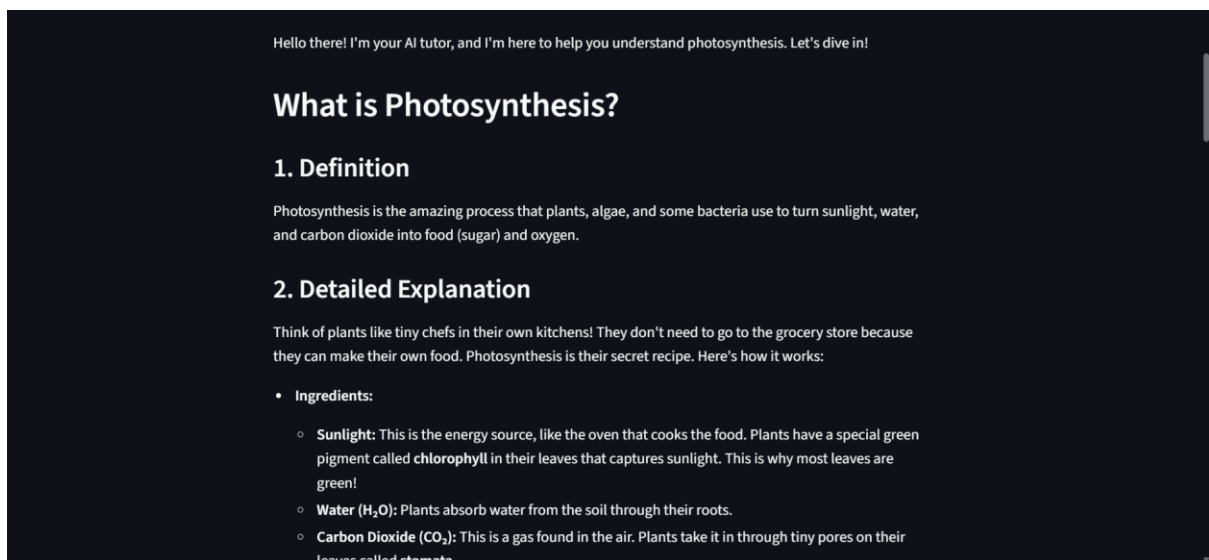


Figure 3. 8 AI-generated quiz and answer key for educational assessment

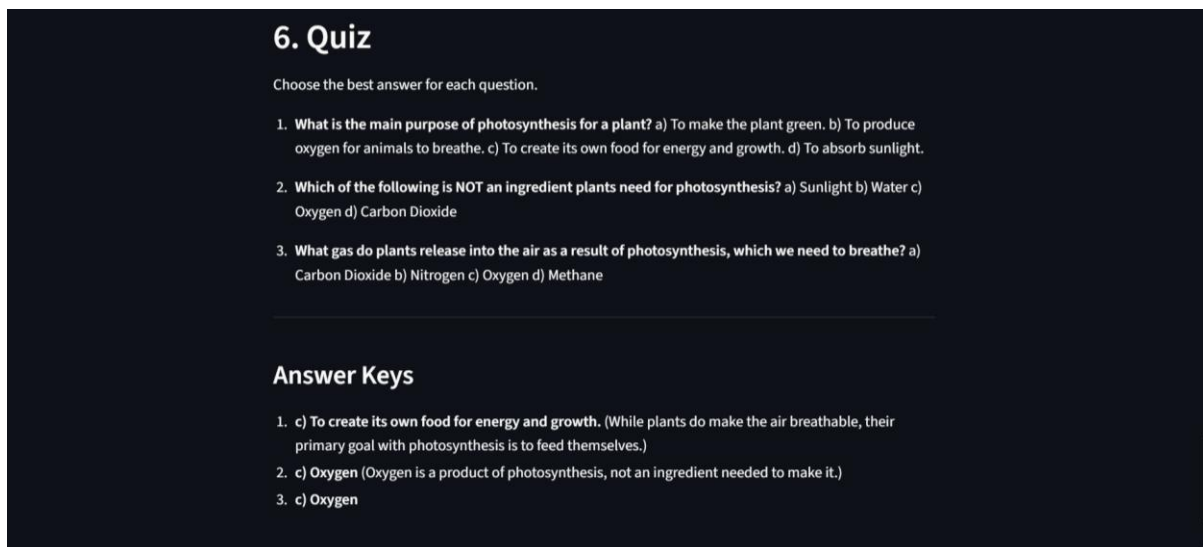


Figure 3. 9 *AI-generated quiz and answer key for educational assessment*

Summary of AI-generated response:

The Quiz Generator produced three multiple-choice questions related to photosynthesis. The questions tested the learner's understanding of the purpose of photosynthesis, the required ingredients, and the oxygen produced as a byproduct.

The generated quiz also included answer keys, allowing learners to check their understanding after reading the explanation and study plan. This showed that the quiz generator could support self-assessment and interactive learning.

3.2.2 Final Interactive Quiz Generator Implementation in OpenRouter Version

After the quiz generator was tested in the initial Gemini-based prototype, the feature was implemented in the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final implementation, the quiz generator was improved by using a fixed quiz format with QUIZ_START and QUIZ_END delimiters. This format allowed the application to separate the quiz content from the explanation text and convert the generated questions into interactive multiple-choice radio buttons.

The final quiz generator produced three multiple-choice questions with three answer options and an answer key. Users could select their answers directly in the application and receive a score after submitting the quiz.

```
# 6. Interactive Quiz

Generate exactly 3 multiple-choice questions related to the topic.
Each question must have three options: A, B, and C.
Provide the correct answer key at the end of each question line.

Generate the quiz in EXACTLY this format:

QUIZ_START

Q1|Question here|Option A|Option B|Option C|A

Q2|Question here|Option A|Option B|Option C|B

Q3|Question here|Option A|Option B|Option C|C

QUIZ_END
```

Figure 3. 10 Final quiz generation prompt format using OpenRouter API

```
# =====
# EXTRACT QUIZ
# =====

quiz_lines = []

if "QUIZ_START" in chat["optimized_response"]:
    try:
        quiz_section = chat["optimized_response"].split("QUIZ_START")[1]
        quiz_section = quiz_section.split("QUIZ_END")[0]

        quiz_lines = [
            line.strip()
            for line in quiz_section.split("\n")
            if line.strip().startswith("Q")
        ]

    except:
        quiz_lines = []

# =====
# PROMPT ENGINEERING ANALYSIS
# =====
```

PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
Network URL: http://192.168.1.6:8501

Figure 3. 11 Quiz extraction implementation using QUIZ_START and QUIZ_END delimiters

```
parts = line.split("|")

if len(parts) == 6:

    qnum = parts[0]
    question_text = parts[1]
    option_a = parts[2]
    option_b = parts[3]
    option_c = parts[4]
    correct_answer = parts[5].strip().upper()
```

Figure 3. 12 Quiz parsing implementation for interactive multiple-choice questions

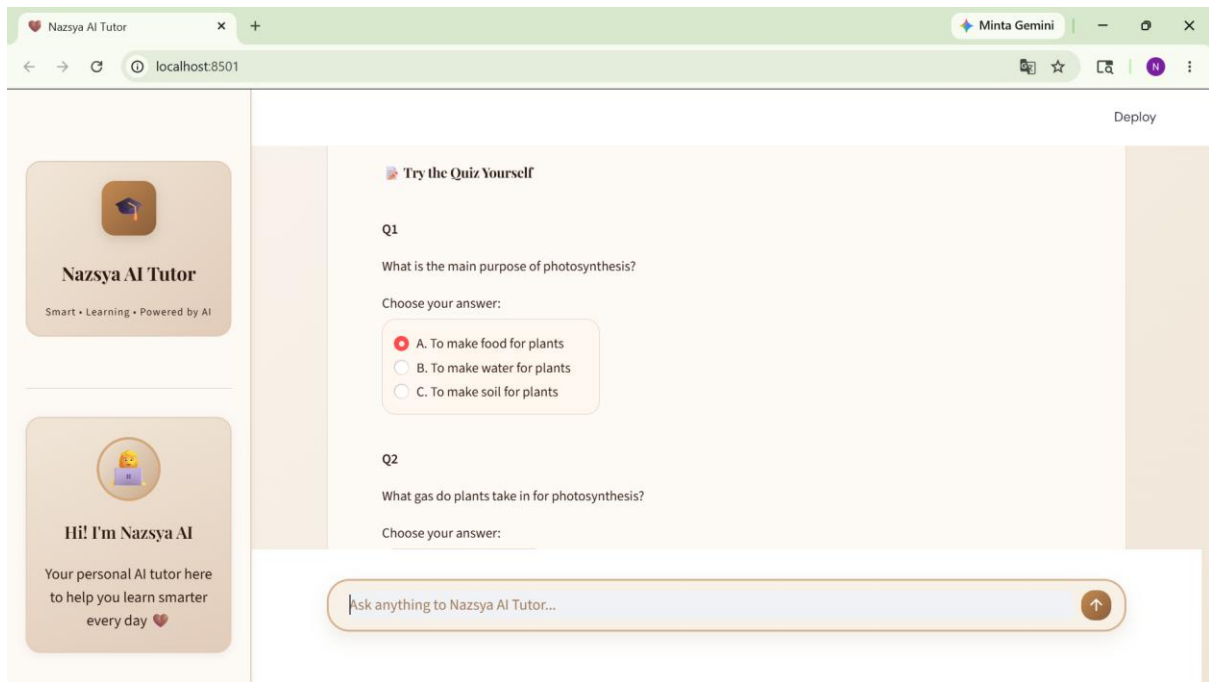


Figure 3. 13 Interactive quiz display in the final Nazsya AI Tutor application

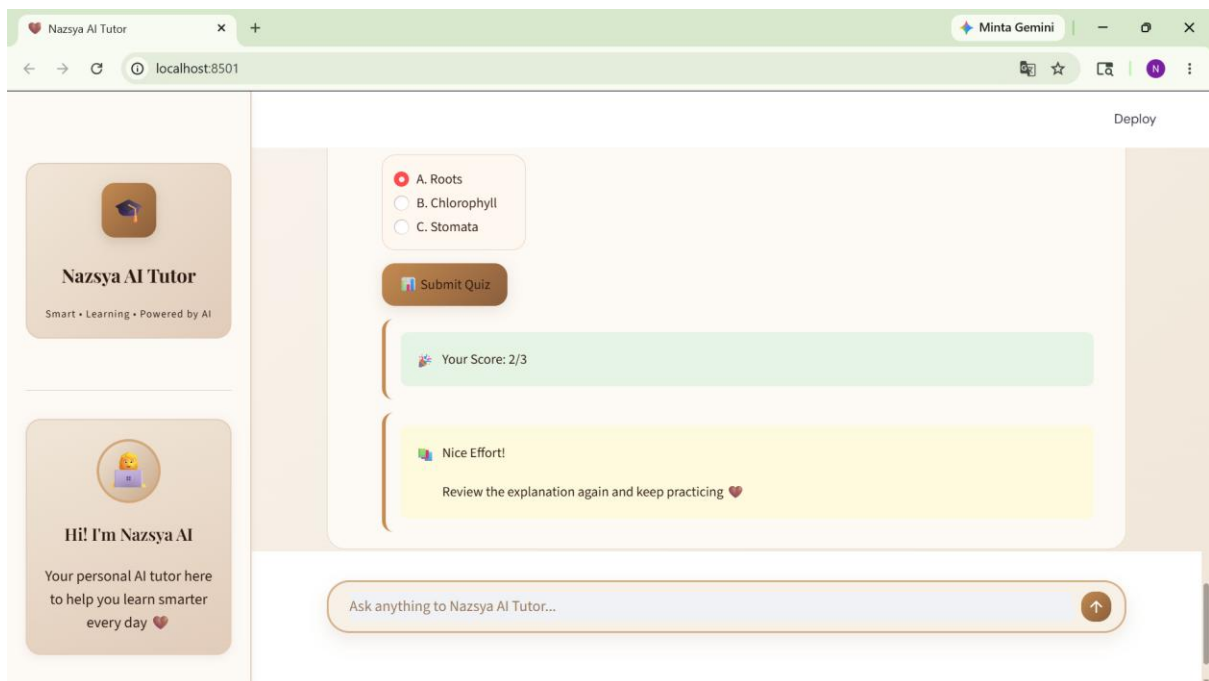


Figure 3. 14 Quiz score result after user submission

Aspect	Initial Quiz Generator	Final Interactive Quiz Generator
API Provider	Gemini API	OpenRouter API
Model	Gemini 2.5 Flash Lite	DeepSeek Chat
Quiz Format	Text-based quiz with answer keys	Structured quiz using QUIZ_START and QUIZ_END
User Interaction	User reads quiz manually	User answers using radio buttons

Assessment	Answer keys displayed in text	Score calculated after submission
Improvement	Adds self-assessment content	Provides interactive quiz experience

Based on the quiz generator testing, the project concluded that the feature successfully generated relevant multiple-choice questions and answer keys based on the learning topic. The final implementation improved the feature by converting the generated quiz into an interactive assessment with selectable answers and automatic scoring.

3.3 Few-Shot Prompting Integration

In this phase, the project implemented Few-Shot Prompting techniques to improve response consistency, output structure, and educational formatting. This technique was first explored in the initial prototype and later integrated into the final OpenRouter-based version of Nazsya AI Tutor.

Few-Shot Prompting was used by providing the AI model with an example question-and-answer format before generating a response to the user's actual question. By giving examples, the system could follow a clearer response pattern and produce more consistent educational outputs.

This technique helped improve the reliability of structured explanations, study plans, quizzes, and summaries generated by Nazsya AI Tutor.

3.3.1 Initial Few-Shot Prompting Implementation

The following example shows how Few-Shot Prompting was used by providing the AI with a sample question and answer format before generating the actual educational response.

```
Example:

Question:
What is gravity?

Answer Format:

1. Definition
Gravity is a natural force that attracts objects toward each other.

2. Detailed Explanation
Gravity keeps planets orbiting the sun and causes objects to fall to the ground.
```

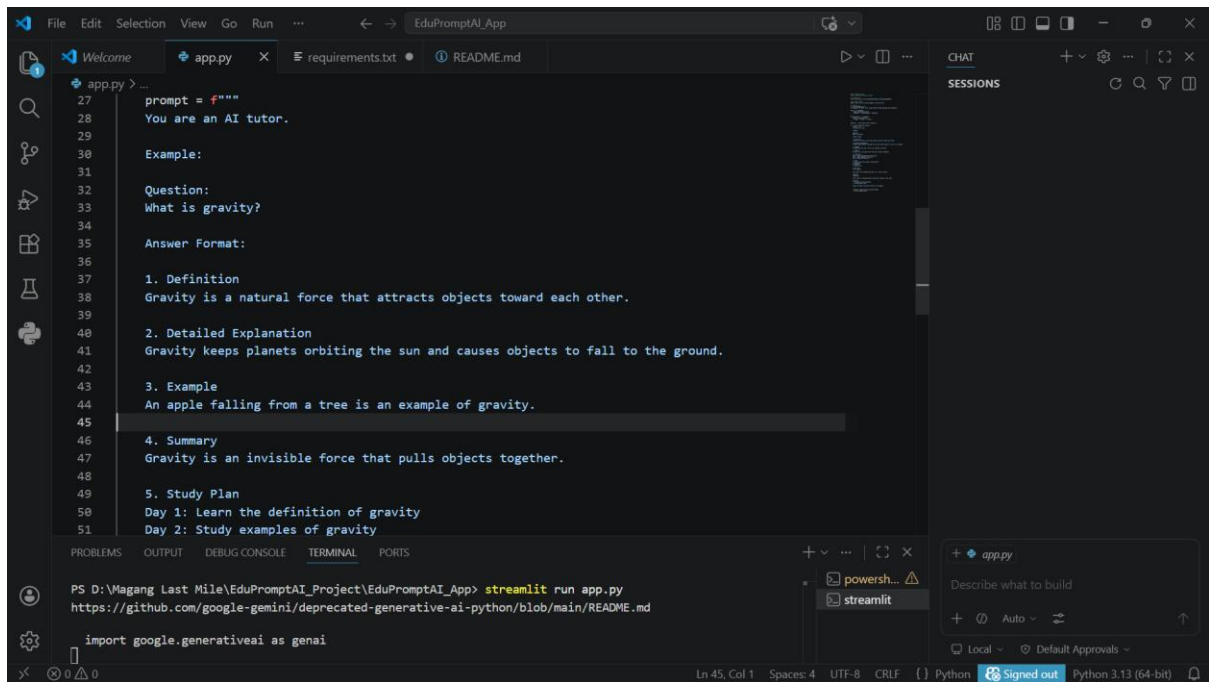


Figure 3. 15 Few-shot prompting implementation using example-based prompt engineering

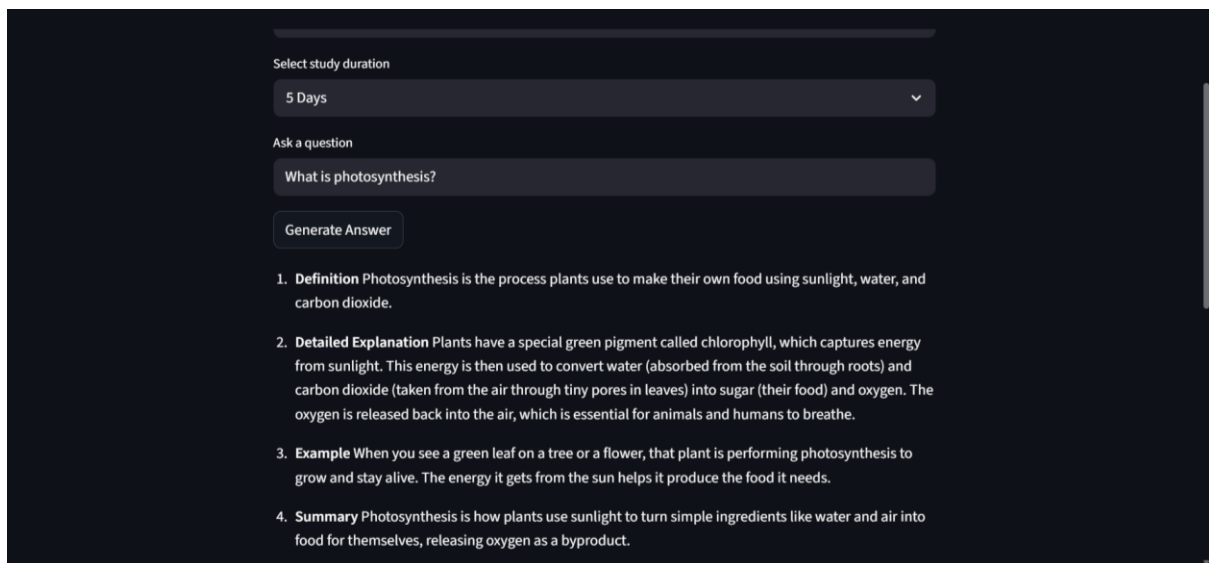


Figure 3. 16 AI-generated educational response using Few-Shot Prompting techniques

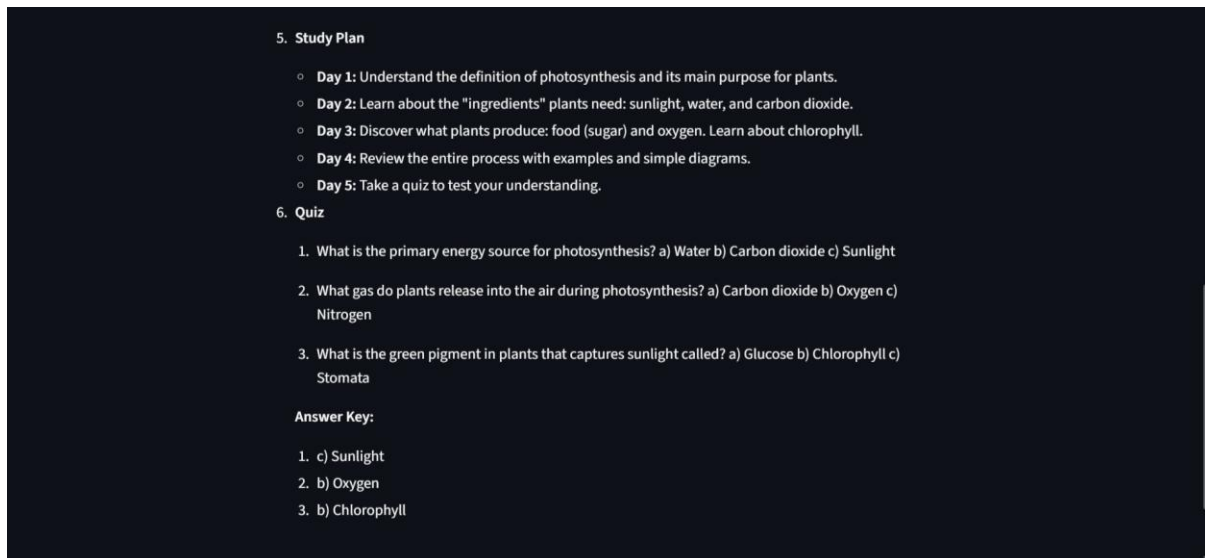


Figure 3. 17 *AI-generated educational response using Few-Shot Prompting techniques*

Summary of AI-generated response:

The Few-Shot Prompting output followed the example-based structure provided in the prompt. The response explained photosynthesis using clear sections such as Definition, Detailed Explanation, Example, Summary, Study Plan, and Quiz.

The generated response was more organized because the AI followed the sample answer format before answering the actual topic. This showed that Few-Shot Prompting helped improve consistency and made the educational response easier to read.

3.3.2 Final Few-Shot Prompting Integration in OpenRouter Version

After the Few-Shot Prompting technique was tested in the initial prototype, the same concept was integrated into the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final implementation, Few-Shot Prompting was included inside the optimized prompt as an example-based formatting guide. The example response helped the AI understand the expected structure before generating the actual answer for the user's topic.

This integration improved response consistency because the AI was guided to follow a structured educational format, including Definition, Detailed Explanation, Real-World Example, Summary, Study Plan, and Interactive Quiz.

```

File Edit Selection View Go Run ... EduPromptAI_App
app.py x Release Notes: 1.120.0 # style.css .env .gitignore requirements.txt README.md
Format your response EXACTLY like this:
1 result in 1 file - Open in editor
app.py 1
Format your respons...

336 Topic:
337 {question}
338
339 Use the following example only as a formatting guide.
340
341 Example:
342 Topic:
343 What is gravity?
344
345 # 1. Definition
346 Gravity is a natural force that attracts objects toward each other.
347
348 # 2. Detailed Explanation
349 Gravity keeps planets orbiting the sun and causes objects to fall toward Earth.
350
351 # 3. Real-World Example
352 A ball falling to the ground is an example of gravity.
353
354 # 4. Summary
355 Gravity is the force that pulls objects toward each other.
356
357 Now generate an answer for the actual topic using the same educational structure.
358 Do not answer the example topic. Only answer the actual user topic.
359
360 Format your response EXACTLY like this:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
Network URL: http://192.168.1.6:8501

```

Figure 3. 18 Final few-shot prompting implementation in the optimized prompt

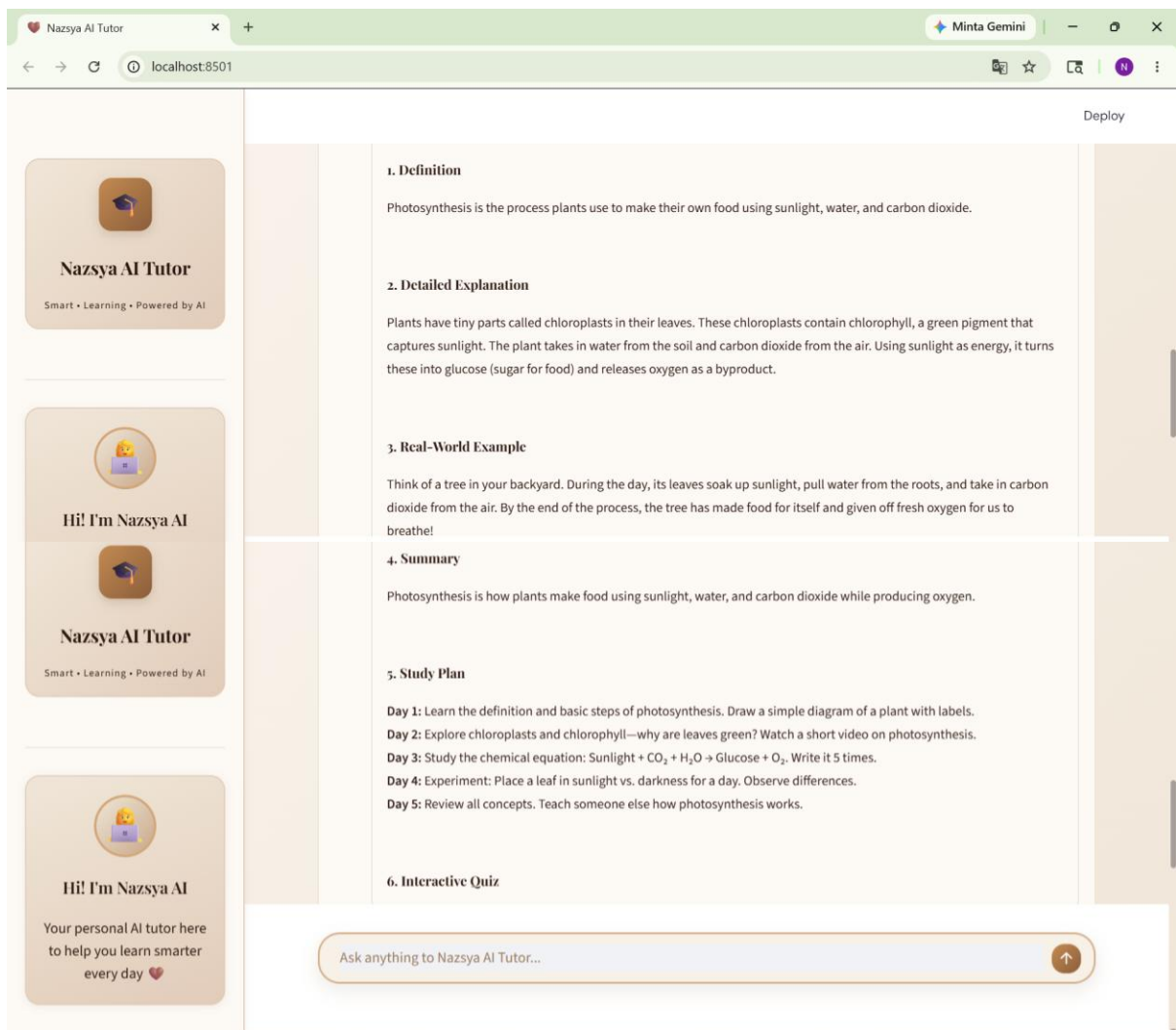


Figure 3. 19 AI-generated educational response using final few-shot prompting integration

Summary of AI-generated response:

The final Few-Shot Prompting output generated a concise and structured explanation about photosynthesis. The response followed the expected educational format, including Definition, Detailed Explanation, Real-World Example, Summary, and Study Plan.

Although the interactive quiz section was displayed separately in the application, the visible output already showed that the model successfully followed the example-based structure. The response also showed that the gravity example was only used as a formatting guide, while the actual answer focused on photosynthesis.

This indicates that Few-Shot Prompting helped improve response consistency, structure, and clarity in the final OpenRouter-based implementation.

3.3.3 Few-Shot Prompting Effect

Aspect	Before Few-Shot	After Few-Shot
Response Structure	Less consistent structure	More consistent structure
Output Format	Variable response format	More stable response format
Explanation Flow	Less guided	More organized and guided
Study Plan Output	Less predictable	More aligned with expected format
Quiz Output	Basic text-based quiz	More structured and easier to parse
Educational Quality	General explanation	Clearer and more learning-oriented response

Based on the Few-Shot Prompting testing, the project concluded that providing an example-based format helped improve the consistency and organization of AI-generated responses. The final implementation used Few-Shot Prompting as part of the optimized prompt to guide the model in producing clearer, more structured, and more educational outputs.

M4. Answer Evaluator (Examiner AI)

4.1 Answer Evaluator Prompt Design

In this phase, the project implemented an Answer Evaluator feature to simulate an AI-based educational examiner. This feature was designed to evaluate user-submitted answers and provide feedback based on the learner's understanding of the topic.

The Answer Evaluator was first tested as a prompt-based evaluation concept and later integrated into the final OpenRouter-based version of Nazsya AI Tutor using the DeepSeek Chat model.

The system was enhanced to evaluate user-submitted answers by analyzing accuracy, completeness, and conceptual understanding. The AI-generated evaluation included a score, strengths, weaknesses, and suggestions for improvement.

This feature transformed Nazsya AI Tutor from a passive tutoring system into an interactive learning assessment platform because users could not only receive explanations and quizzes, but also submit their own answers for evaluation.

4.1.1 Initial Answer Evaluation Prompt Design

The following prompt shows the initial answer evaluation design used to guide the AI in evaluating user-submitted answers.

```
Evaluate the user's answer and provide:  
- Score  
- Strengths  
- Weaknesses  
- Suggestions for improvement  
  
User Answer:  
{user_answer}
```

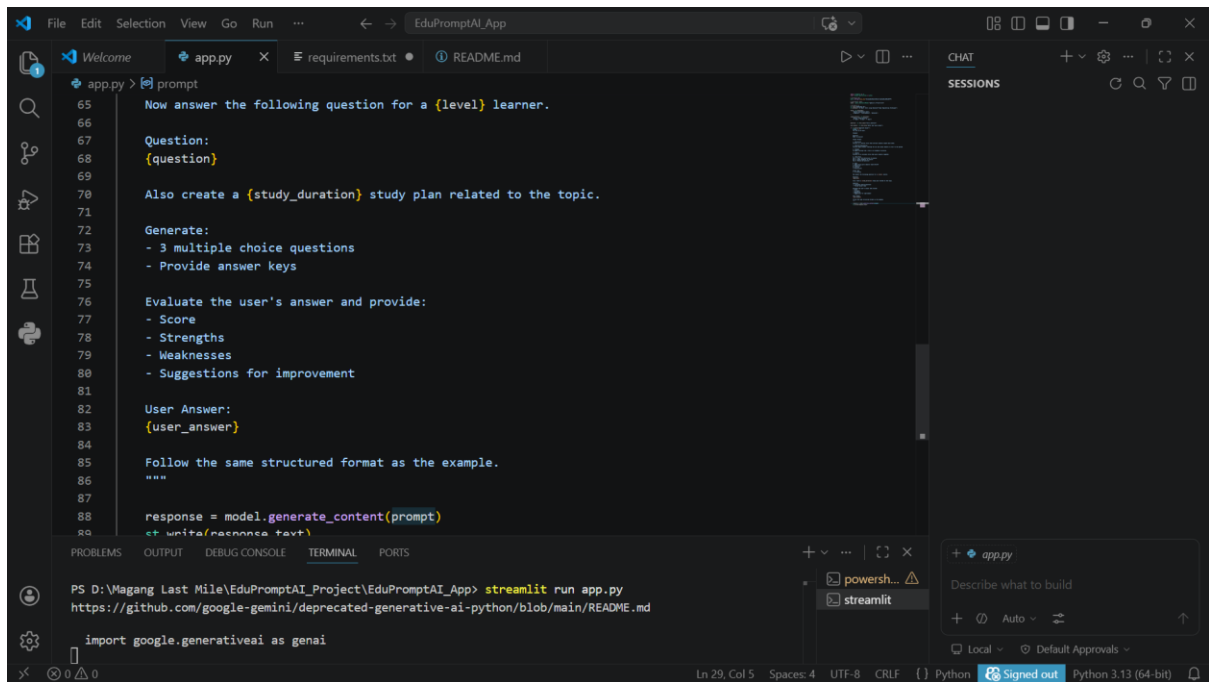


Figure 4. 1 Answer evaluator prompt implementation

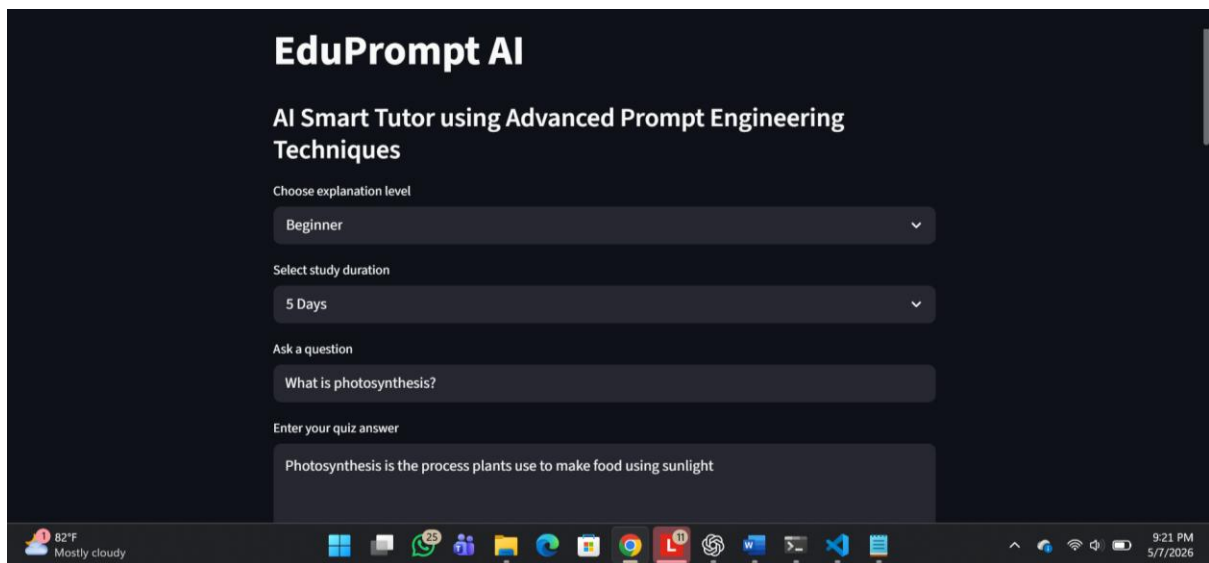


Figure 4. 2 AI-generated evaluation feedback including score, strengths, weaknesses, and improvement suggestions

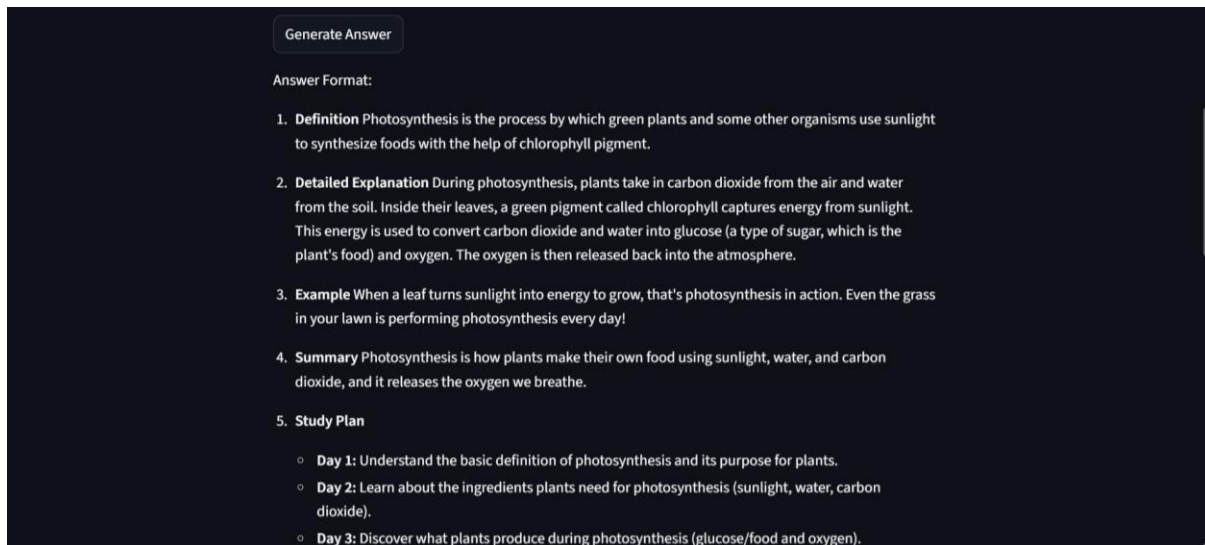


Figure 4. 3 AI-generated evaluation feedback including score, strengths, weaknesses, and improvement suggestions

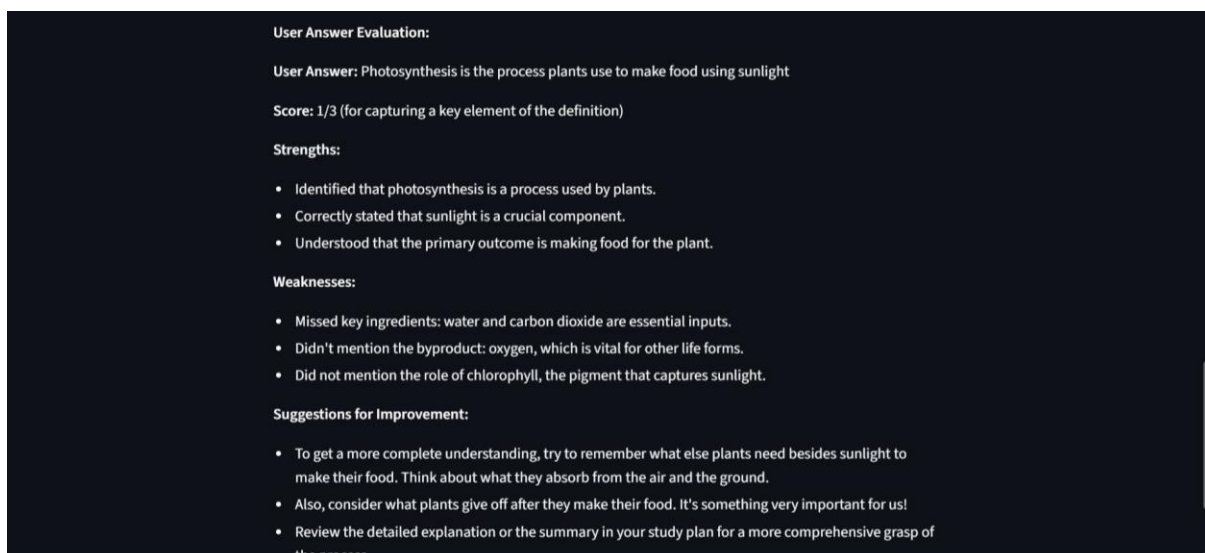


Figure 4. 4 AI-generated evaluation feedback including score, strengths, weaknesses, and improvement suggestions

Summary of AI-generated evaluation:

The Answer Evaluator assessed the user's answer: "Photosynthesis is the process plants use to make food using sunlight." The AI provided a score of 1/3 because the answer correctly mentioned plants, food production, and sunlight, but did not include several important elements.

The evaluation identified strengths such as understanding that photosynthesis is used by plants to make food. It also identified weaknesses, including missing key inputs such as water and carbon dioxide, not mentioning oxygen as a byproduct, and not explaining the role of chlorophyll.

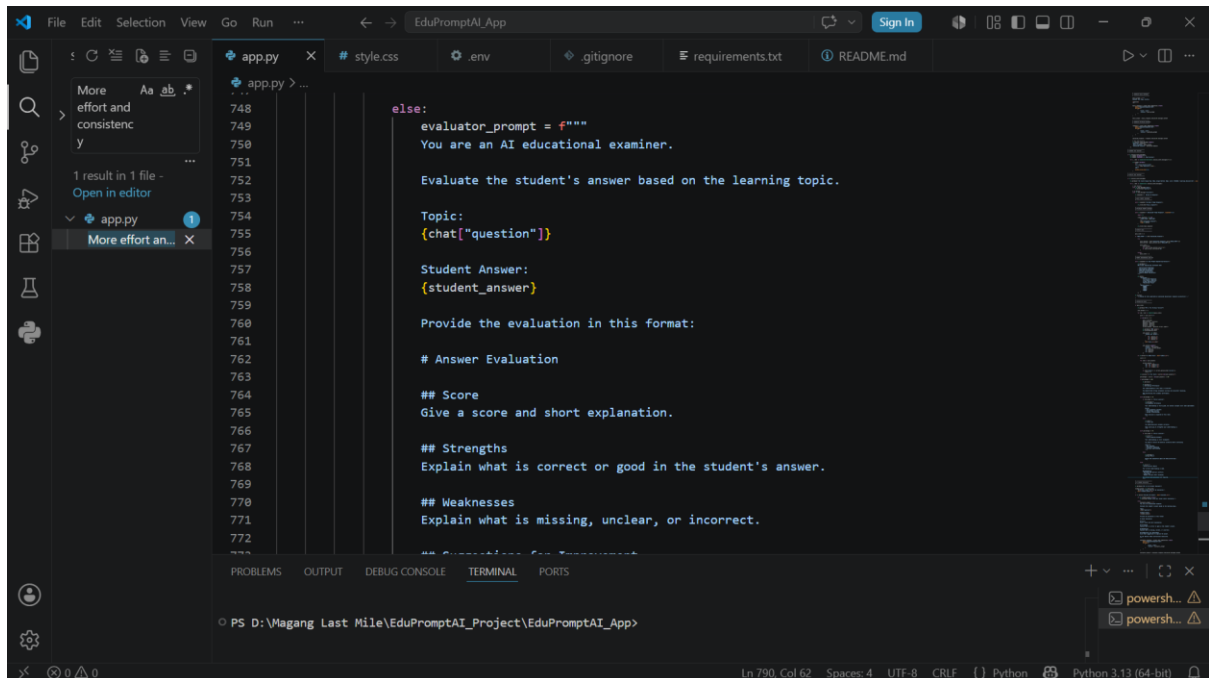
- The evaluator also provided suggestions for improvement by encouraging the user to review the complete process of photosynthesis, including the required ingredients and the oxygen released during the process.

4.1.2 Final Answer Evaluator Implementation in OpenRouter Version

After the answer evaluator prompt was designed and tested, the feature was implemented in the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final implementation, users can write their own answer in a text area provided by the application. After submitting the answer, the AI evaluates the response and generates feedback consisting of score, strengths, weaknesses, and suggestions for improvement.

This implementation allows the system to act as an Examiner AI that supports interactive learning assessment.



```

748
749
750     evaluator_prompt = f"""
751     You are an AI educational examiner.
752
753     Evaluate the student's answer based on the learning topic.
754
755     Topic:
756     {chat["question"]}
757
758     Student Answer:
759     {student_answer}
760
761     Provide the evaluation in this format:
762
763     # Answer Evaluation
764
765     ## Score
766     Give a score and short explanation.
767
768     ## Strengths
769     Explain what is correct or good in the student's answer.
770
771     ## Weaknesses
772     Explain what is missing, unclear, or incorrect.
773
774     # Suggestions for Improvement
  
```

Figure 4. 5 Final answer evaluator prompt implementation using OpenRouter API

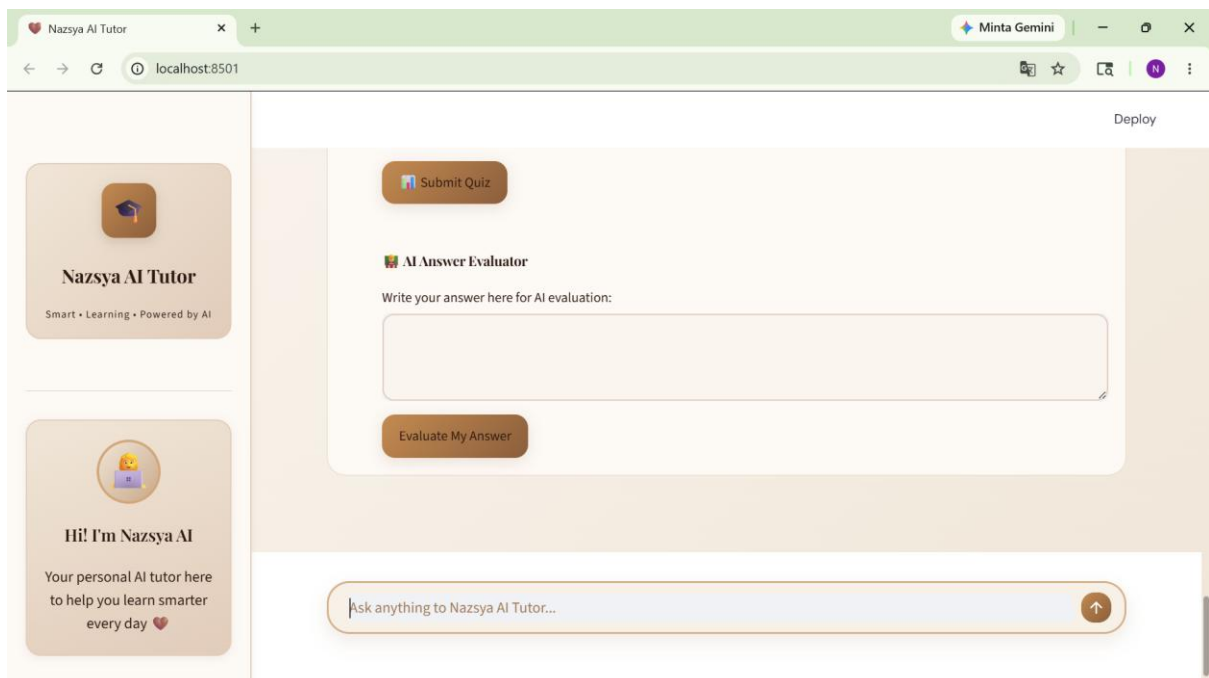


Figure 4. 6 Answer evaluator input field in the final Nazsya AI Tutor application

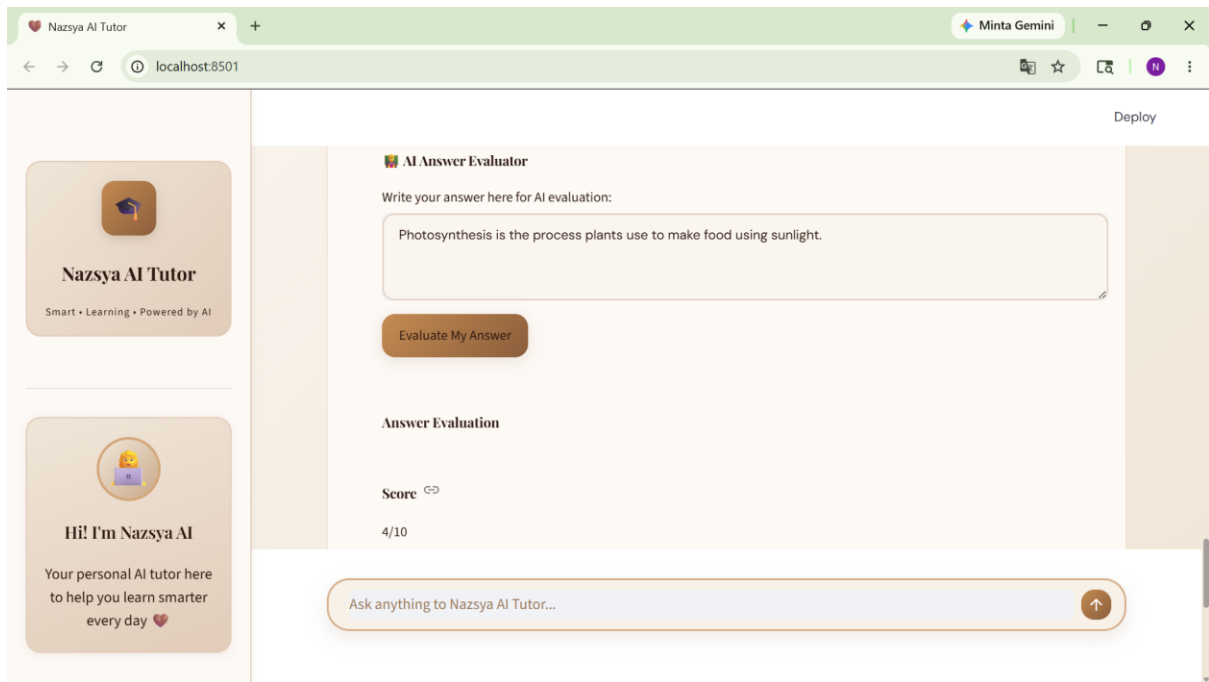


Figure 4. 7 *AI-generated answer evaluation feedback in the final application*

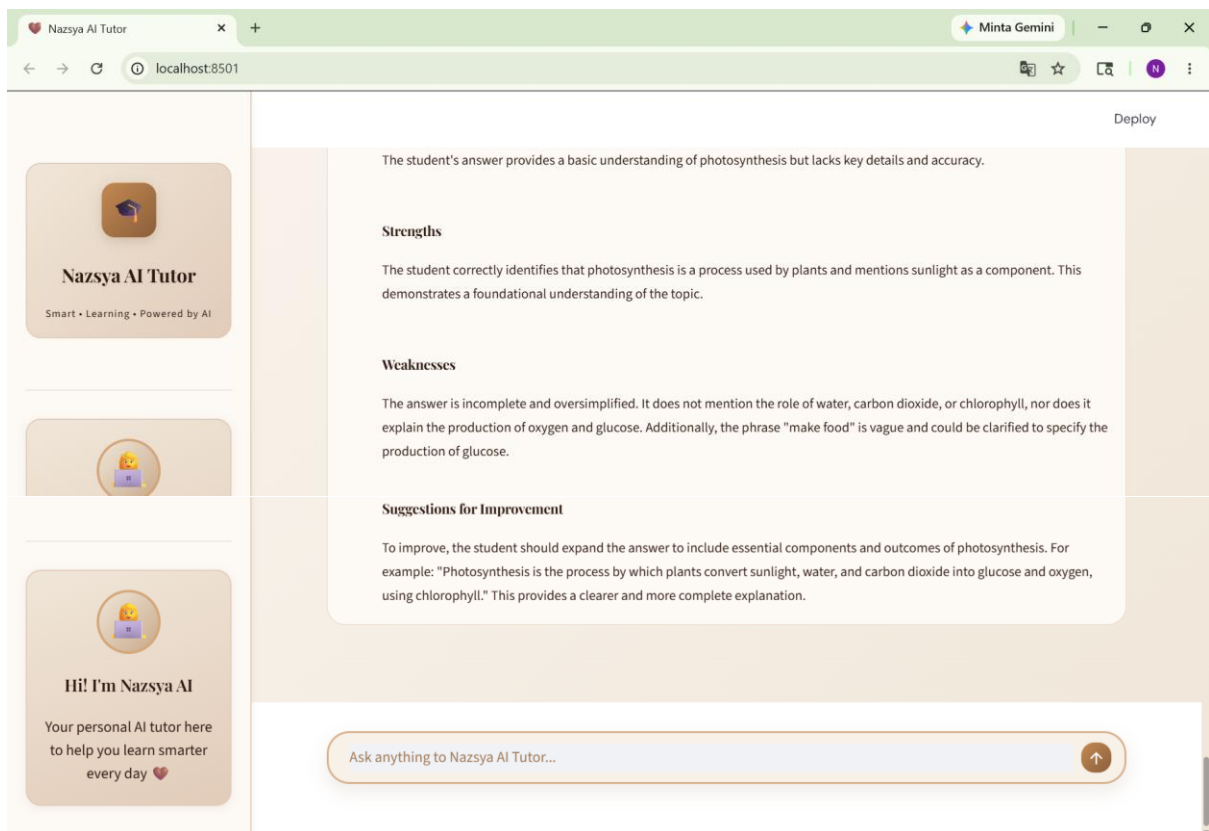


Figure 4. 8 *AI-generated answer evaluation feedback in the final application*

Summary of AI-generated evaluation:

The final Answer Evaluator assessed the user's answer: "Photosynthesis is the process plants use to make food using sunlight." The AI gave a score of 4/10 because the answer showed a basic understanding of photosynthesis but was still incomplete.

The evaluation identified strengths such as correctly mentioning plants, food production, and sunlight. However, the AI also identified several weaknesses, including missing information about water, carbon dioxide, chlorophyll, glucose, and oxygen as a byproduct.

The evaluator provided improvement suggestions by recommending a more complete answer that includes the main inputs and outputs of photosynthesis. This result shows that the Answer Evaluator can provide useful feedback to help learners improve their understanding.

Aspect	Initial Answer Evaluator	Final Answer Evaluator
Evaluation Input	User answer text	User answer submitted through text area
API Provider	Initial prompt testing	OpenRouter API
Model	Initial LLM testing	DeepSeek Chat
Feedback Format	Score, strengths, weaknesses, suggestions	Structured evaluator feedback
User Interaction	Evaluation shown as generated text	User can submit answer directly in the app
Purpose	Test evaluator prompt design	Interactive learning assessment

Based on the answer evaluator testing, the project concluded that the feature successfully provided educational feedback for user-submitted answers. The evaluator was able to identify correct points, missing information, and suggestions for improvement. This made Nazsya AI Tutor more interactive because learners could receive feedback on their own answers, not only read AI-generated explanations.

4.2 Rubric-Based Prompting Implementation

In this phase, the project enhanced the answer evaluation system by implementing rubric-based prompting techniques. This improvement was applied to make the evaluation process more structured, consistent, and objective.

The AI evaluator was instructed to assess user responses using predefined assessment criteria, including Accuracy, Completeness, and Clarity. Each criterion was scored from 0 to 5, and the total score was calculated out of 15.

In the final OpenRouter-based implementation, rubric-based prompting was integrated into the Answer Evaluator feature. This allowed Nazsya AI Tutor to provide feedback that was more similar to real academic assessment systems, including criterion-based scores, strengths, weaknesses, and suggestions for improvement.

4.2.1 Initial Rubric-Based Evaluation Prompt Design

The following prompt shows the rubric-based evaluation design used to guide the AI in assessing user-submitted answers using predefined academic criteria.

Evaluate the user's answer using the following rubric:

1. Accuracy (0-5)
2. Completeness (0-5)
3. Clarity (0-5)

Then provide:

- Total Score
- Strengths
- Weaknesses
- Suggestions for Improvement

```

69
70 Also create a {study_duration} study plan related to the topic.
71
72 Generate:
73 - 3 multiple choice questions
74 - Provide answer keys
75
76 Evaluate the user's answer using the following rubric:
77
78 1. Accuracy (0-5)
79 2. Completeness (0-5)
80 3. Clarity (0-5)
81
82 Then provide:
83 - Total Score
84 - Strengths
85 - Weaknesses
86 - Suggestions for Improvement
87
88 User Answer:
89 {user_answer}
90
91 Follow the same structured format as the example.
92
93

```

PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
<https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md>

```

import google.generativeai as genai

```

Figure 4. 9 Rubric-based evaluation prompt implementation

- 1. Definition** Photosynthesis is the process by which green plants and some other organisms use sunlight to synthesize foods with the help of chlorophyll pigment.
- 2. Detailed Explanation** During photosynthesis, plants take in carbon dioxide from the air through their leaves and water from the soil through their roots. Inside the plant's cells, specifically in chloroplasts, sunlight energy is used to convert these simple ingredients into glucose (a type of sugar), which is the plant's food. As a byproduct, oxygen is released back into the atmosphere, which is essential for many living things, including humans.
- 3. Example** Think about a tree or a flower. These plants are performing photosynthesis to grow and survive. They use sunlight to create the energy they need, and in return, they give us the oxygen we breathe.
- 4. Summary** Photosynthesis is how plants make their own food using sunlight, water, and carbon dioxide, releasing oxygen as a result.
- 5. Study Plan**
 - Day 1: Understand the basic definition of photosynthesis and why it's important.
 - Day 2: Learn about the ingredients needed for photosynthesis (sunlight, water, carbon dioxide) and where plants get them.
 - Day 3: Discover the products of photosynthesis (glucose/sugar and oxygen) and what happens to them.

Figure 4. 10 AI-generated rubric-based evaluation using Accuracy, Completeness, and Clarity criteria

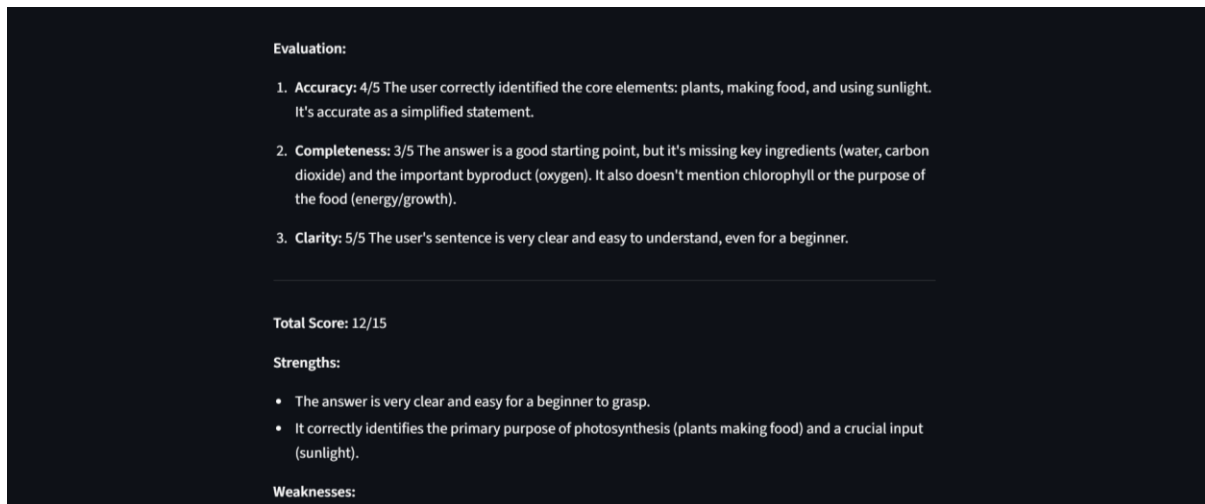


Figure 4. 11 *AI-generated rubric-based evaluation using Accuracy, Completeness, and Clarity criteria*

Summary of AI-generated rubric-based evaluation:

The rubric-based evaluator assessed the user's answer: "Photosynthesis is the process plants use to make food using sunlight." The AI evaluated the answer using three criteria: Accuracy, Completeness, and Clarity.

The answer received 4/5 for Accuracy because it correctly mentioned plants, food production, and sunlight. It received 3/5 for Completeness because it missed important components such as water, carbon dioxide, oxygen, and chlorophyll. It received 5/5 for Clarity because the sentence was simple and easy to understand.

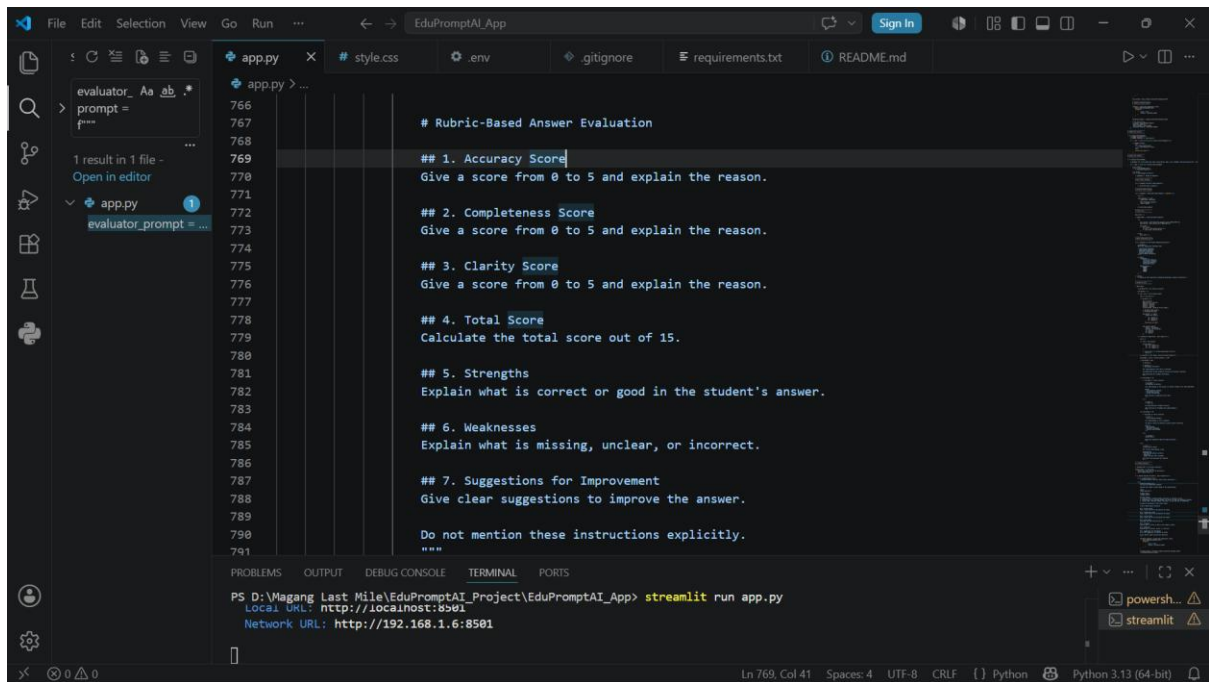
The total score was 12/15. The evaluator also provided strengths, weaknesses, and suggestions for improvement, showing that rubric-based prompting can produce more structured and objective feedback.

4.2.2 Final Rubric-Based Evaluator Implementation in OpenRouter Version

After the basic answer evaluator was implemented, the evaluation prompt was refined using rubric-based prompting in the final OpenRouter-based version of Nazsya AI Tutor.

In this implementation, the evaluator assessed the student's answer using three criteria: Accuracy, Completeness, and Clarity. Each criterion was scored from 0 to 5, and the final total score was calculated out of 15.

This rubric-based approach improved the evaluator because the feedback became more specific, measurable, and educationally meaningful.



```
app.py > ...
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791

# Rubric-Based Answer Evaluation

## 1. Accuracy Score
Give a score from 0 to 5 and explain the reason.

## 2. Completeness Score
Give a score from 0 to 5 and explain the reason.

## 3. Clarity Score
Give a score from 0 to 5 and explain the reason.

## 4. Total Score
Calculate the total score out of 15.

## 5. Strengths
Explain what is correct or good in the student's answer.

## 6. Weaknesses
Explain what is missing, unclear, or incorrect.

## 7. Suggestions for Improvement
Give clear suggestions to improve the answer.

Do not mention these instructions explicitly.
"""

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Magang Last Mile\EduPromptAI_Project\EduPromptAI_App> streamlit run app.py
Local URL: http://localhost:8501
Network URL: http://192.168.1.6:8501
```

Figure 4. 12 Final rubric-based evaluator prompt implementation using OpenRouter API

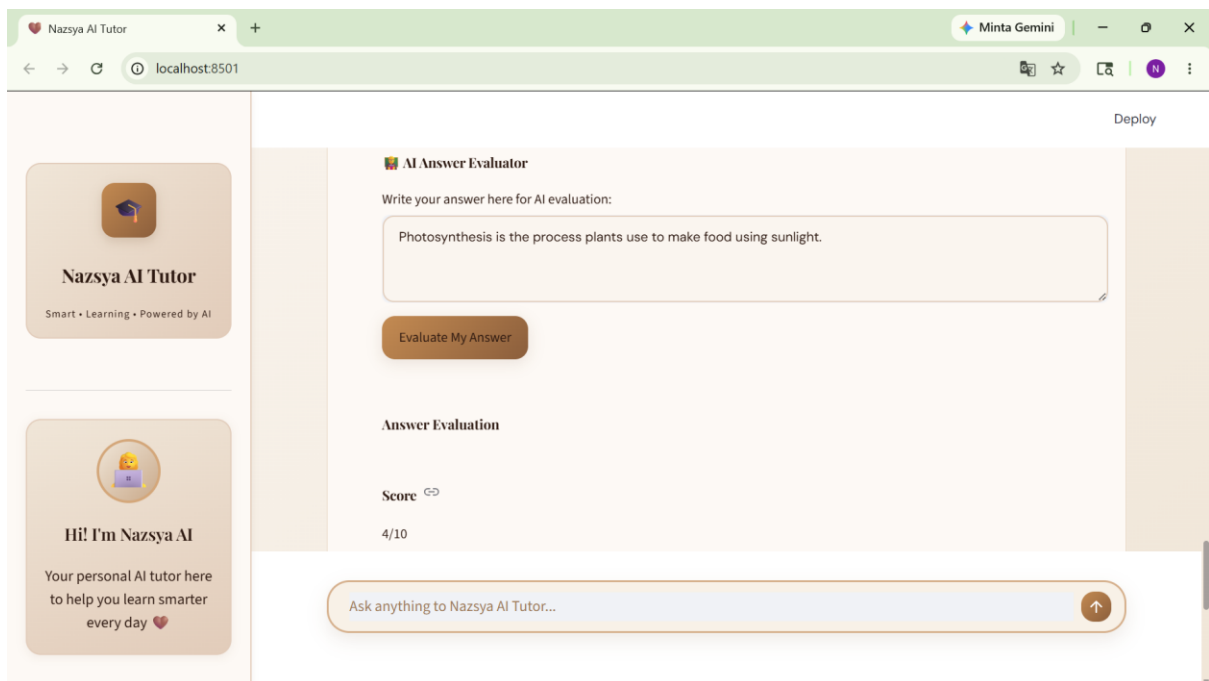


Figure 4. 13 Rubric-based answer evaluator input field in the final application

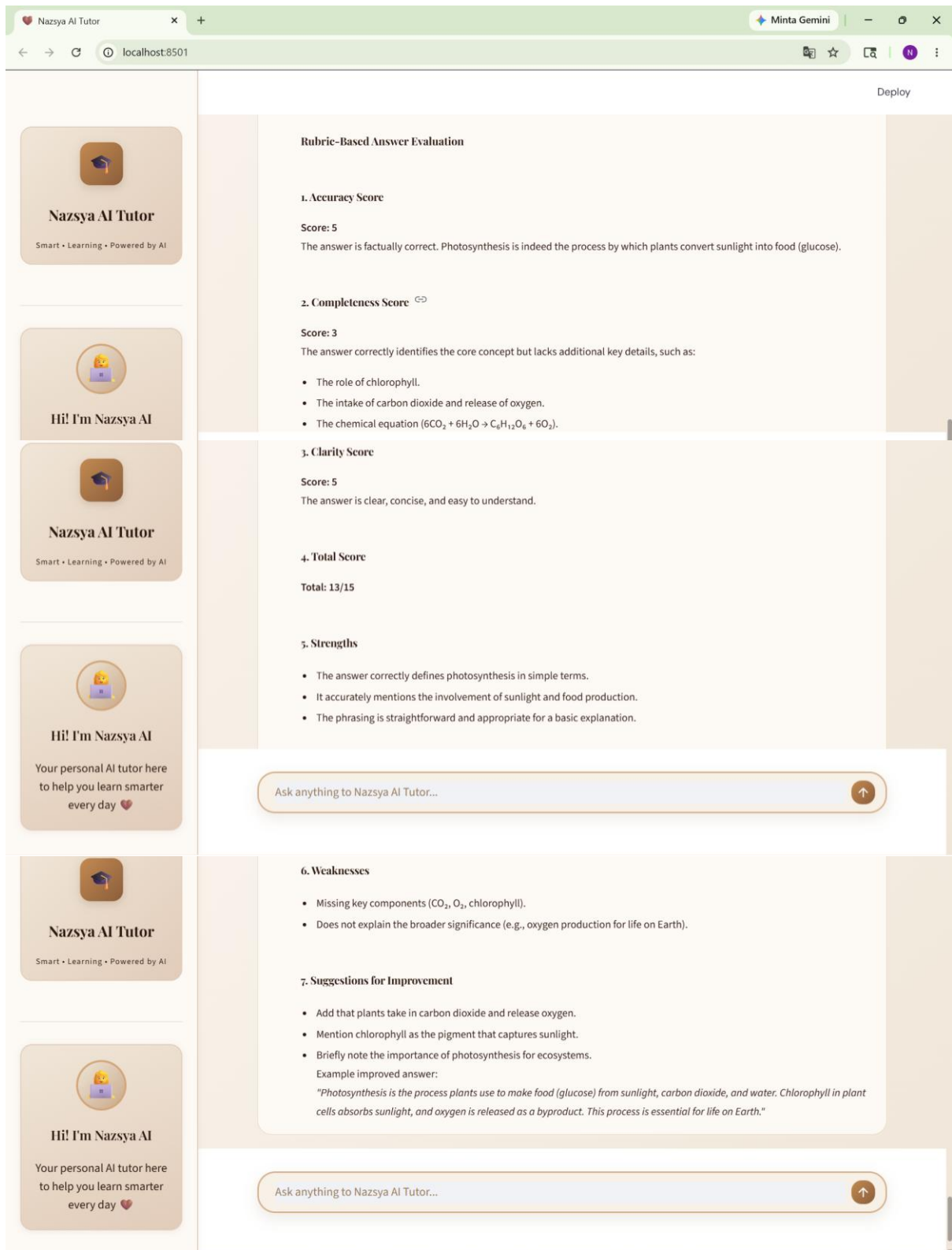


Figure 4. 14 AI-generated rubric-based evaluation feedback in the final application

Summary of AI-generated rubric-based evaluation:

The final rubric-based evaluator assessed the user's answer: "Photosynthesis is the process plants use to make food using sunlight." The AI evaluated the answer using three rubric criteria: Accuracy, Completeness, and Clarity.

The response received 5/5 for Accuracy because the answer was factually correct, 3/5 for Completeness because it missed key details such as chlorophyll, carbon dioxide, oxygen, and the chemical equation, and 5/5 for Clarity because the sentence was simple and easy to understand.

The total score was 13/15. The evaluator also provided strengths, weaknesses, suggestions for improvement, and an improved example answer. This shows that rubric-based prompting can produce more structured, measurable, and educational feedback.

Rubric Criteria	Score Range	Evaluation Focus
Accuracy	0–5	Measures factual correctness of the answer
Completeness	0–5	Measures whether important concepts are included
Clarity	0–5	Measures how clear and understandable the answer is
Total Score	0–15	Combines all rubric criteria

Based on the rubric-based evaluation testing, the project concluded that rubric-based prompting improved the objectivity and structure of the Answer Evaluator. By using Accuracy, Completeness, and Clarity as assessment criteria, the system was able to provide more measurable feedback and help learners understand which parts of their answer needed improvement.

4.3 Role-Based Prompting Experimentation

In this phase, the project implemented Role-Based Prompting techniques to modify the behavior, communication tone, and evaluation style of the AI evaluator. This technique was tested to analyze how different role instructions can influence the feedback generated by the AI.

Different evaluator roles, including Friendly Tutor, Strict Lecturer, and Professional Examiner, were tested to compare feedback style, evaluation strictness, and communication tone.

In the final OpenRouter-based implementation, the selected evaluator role was integrated into the Answer Evaluator prompt so that the system could adjust its feedback style based on the user's selected role.

4.3.1 Initial Role-Based Prompting Implementation

The following code shows the initial role-based prompting design used to test how different evaluator roles affect AI-generated feedback style.

```
role = st.selectbox(
    "Choose evaluator role",
    ["Friendly Tutor", "Strict Lecturer", "Professional Examiner"]
)

prompt = f"""
You are acting as a {role}.
"""
```

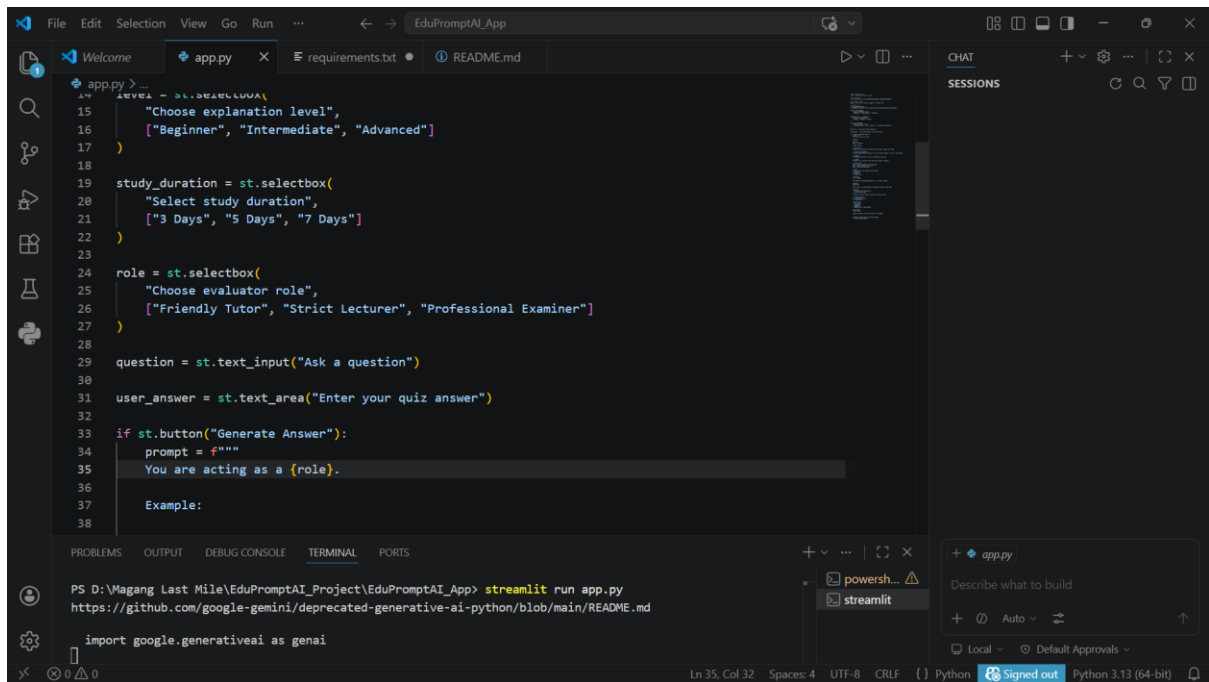


Figure 4. 15 Role-based prompting implementation for AI evaluator behavior customization



Figure 4. 16 Friendly Tutor evaluation style generated

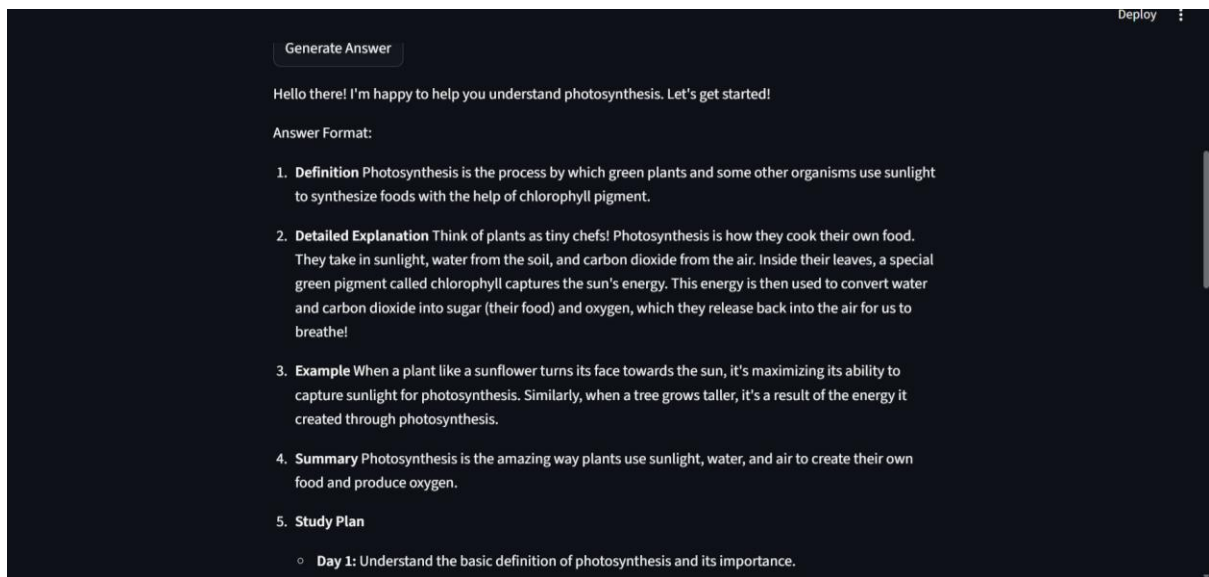


Figure 4. 17 *Friendly Tutor evaluation style generated*

Summary of Friendly Tutor role output:

The Friendly Tutor role generated a supportive and encouraging explanation about photosynthesis. The AI explained photosynthesis using simple language and friendly analogies, such as comparing plants to tiny chefs that make their own food using sunlight, water, and carbon dioxide.

The response also included a structured learning format consisting of Definition, Detailed Explanation, Example, Summary, Study Plan, Quiz, and Answer Key. This showed that the AI could provide educational content in an engaging and beginner-friendly way.

In the user answer evaluation, the AI gave a total score of 11/15. The answer received 4/5 for Accuracy, 2/5 for Completeness, and 5/5 for Clarity. The evaluator identified that the answer was clear and correctly mentioned sunlight and food production, but it was incomplete because it did not mention water, carbon dioxide, and oxygen.

The feedback ended with encouraging suggestions, showing that the Friendly Tutor role produced a warm, supportive, and motivating evaluation style.

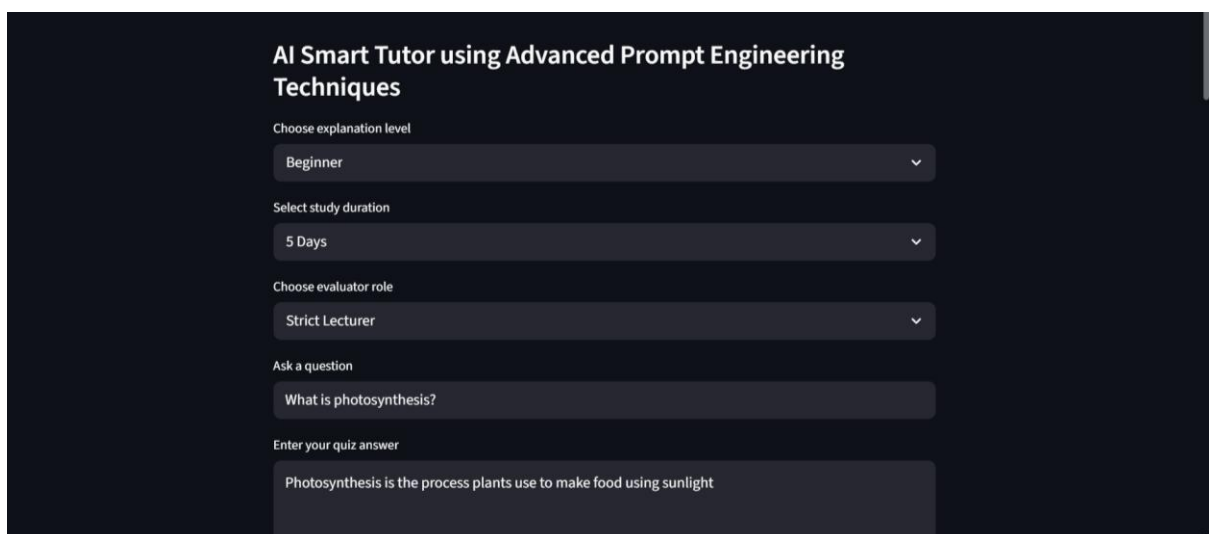


Figure 4. 18 *Strict Lecturer evaluation style generated*

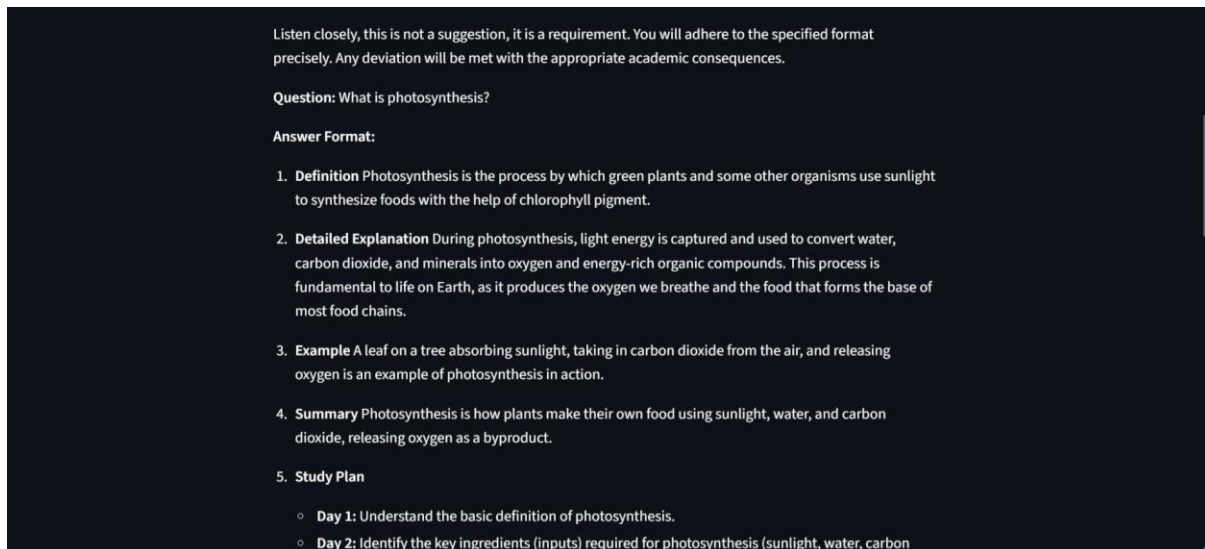


Figure 4. 19 *Strict Lecturer evaluation style generated*

Summary of Strict Lecturer role output:

The Strict Lecturer role generated a firm, critical, and academic response about photosynthesis. The AI explained photosynthesis using more direct and formal language, emphasizing the importance of following the required answer format and understanding the complete scientific process.

The response included structured learning content such as Definition, Detailed Explanation, Example, Summary, Study Plan, Quiz, and Answer Key. Compared to the Friendly Tutor role, the Strict Lecturer role used a more serious tone and focused more on academic accuracy and completeness.

In the user answer evaluation, the AI gave a total score of 9/15. The answer received 3/5 for Accuracy, 2/5 for Completeness, and 4/5 for Clarity. The evaluator acknowledged that the user understood the basic idea of photosynthesis, but criticized the answer for missing important components such as water, carbon dioxide, oxygen, and chlorophyll.

The feedback showed that the Strict Lecturer role produced a stricter and more critical evaluation style, making it suitable for academic assessment and more serious learning contexts.

4.3.2 Final Role-Based Evaluator Implementation in OpenRouter Version

After the role-based prompting concept was tested, the technique was integrated into the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final implementation, users can select one of three evaluator roles: Friendly Tutor, Strict Lecturer, or Professional Examiner. The selected role is inserted dynamically into the evaluator prompt so that the AI can adjust the tone and strictness of its feedback.

This implementation allows the Answer Evaluator to provide feedback in different styles. The Friendly Tutor role produces supportive and encouraging feedback, the Strict Lecturer role produces firmer and more critical feedback, and the Professional Examiner role produces formal and objective assessment.

```

with col_input3:
    role_input = st.selectbox(
        "Evaluator Role",
        ["Friendly Tutor", "Strict Lecturer", "Professional Examiner"],
        key="role_input",
        label_visibility="collapsed"
    )

```

Figure 4. 20 Final evaluator role selection implementation in Streamlit

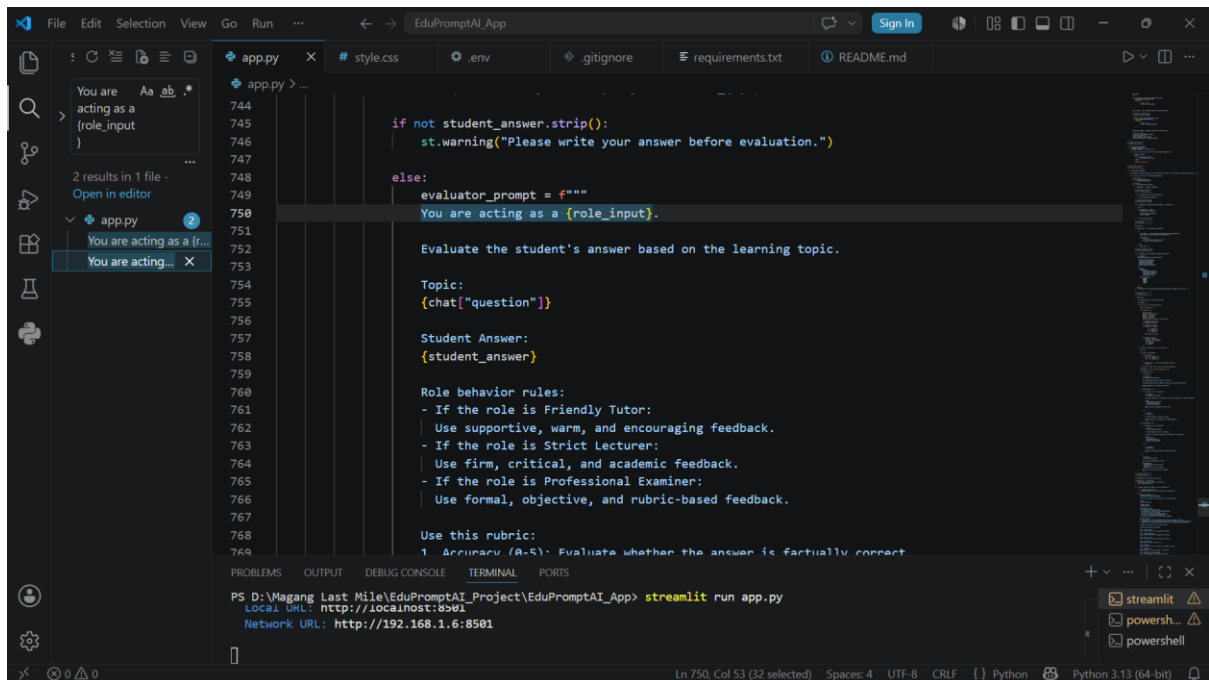


Figure 4. 21 Final role-based evaluator prompt implementation using OpenRouter API

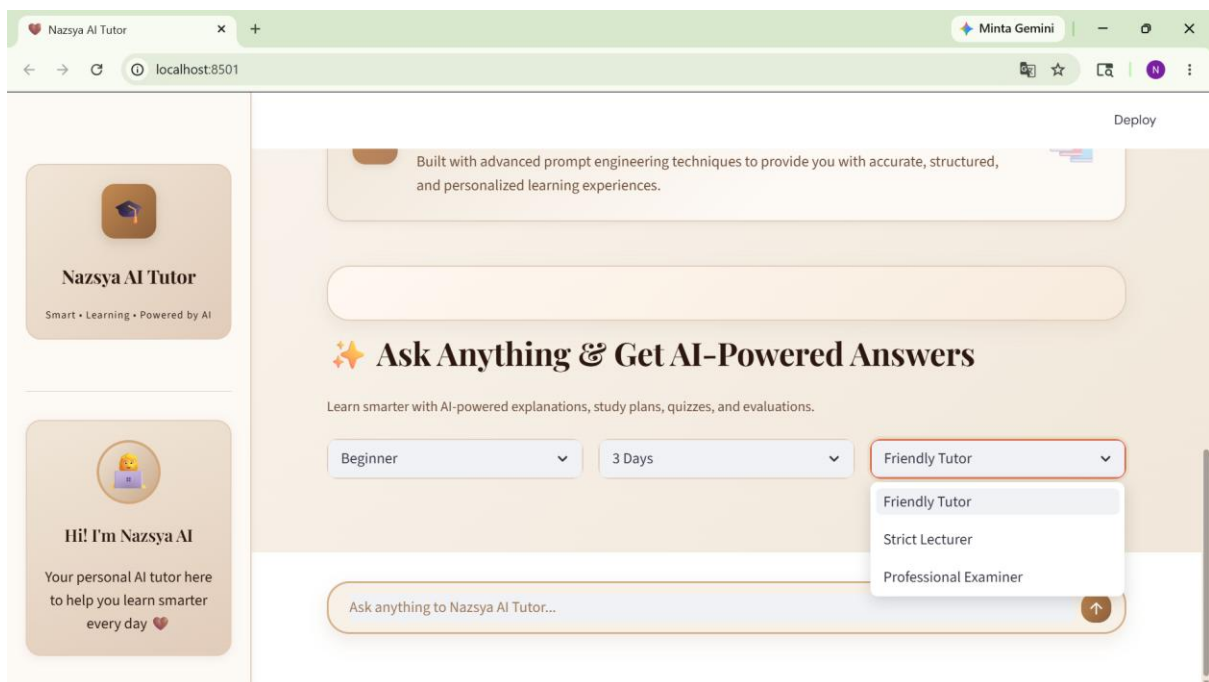


Figure 4. 22 Evaluator role selection in the final Nazsya AI Tutor interface

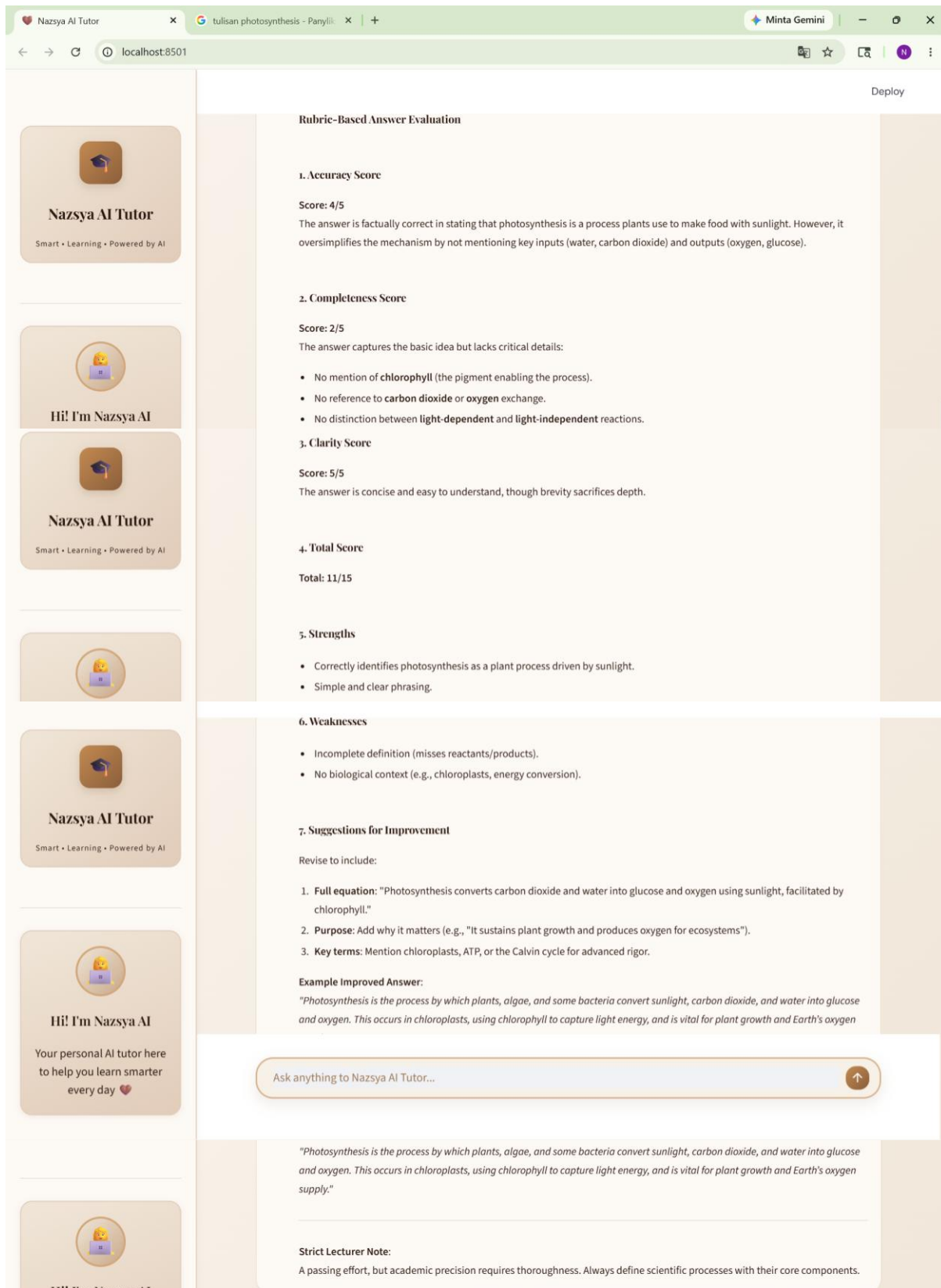


Figure 4. 23 Role-based answer evaluation feedback in the final application

Summary of role-based evaluation output:

The Strict Lecturer role generated a firmer and more critical evaluation style. The AI assessed the user's answer using rubric-based criteria, including Accuracy, Completeness, and Clarity.

The answer received a total score of 11/15 because it correctly explained the basic idea of photosynthesis but missed several important scientific details.

The feedback highlighted missing components such as water, carbon dioxide, oxygen, glucose, chlorophyll, chloroplasts, and deeper biological context. The response also included a Strict Lecturer Note, which showed a more academic and demanding tone compared to a Friendly Tutor response.

This result shows that role-based prompting successfully influenced the tone and strictness of the AI evaluator.

4.3.3 Role Comparison Summary

Role	Feedback Style	Evaluation Behavior
Friendly Tutor	Supportive and encouraging	Gives gentle feedback and motivates the learner
Strict Lecturer	Formal, critical, and firm	Gives stricter evaluation and highlights weaknesses clearly
Professional Examiner	Objective and professional	Gives neutral, rubric-based, and assessment-focused feedback

Based on the role-based prompting experiment, the project concluded that role instructions influenced the tone, strictness, and communication style of the AI evaluator. This showed that prompt engineering can be used not only to control response structure, but also to adjust the evaluator personality based on learning needs.

M5. Prompt Comparison Engine and Optimization

5.1 Prompt Comparison Engine Development

In this phase, the project developed a Prompt Comparison Engine to compare the quality of AI-generated educational responses using different prompt engineering techniques.

The initial prototype tested several prompting strategies, including Standard Prompting, Basic Prompting, Structured Prompting, Few-Shot Prompting, and Optimized Prompting. This comparison was used to analyze differences in response quality, structure, consistency, and readability.

In the final OpenRouter-based implementation, the comparison engine focused on comparing the Basic Prompt Response and the Optimized Prompt Response. This approach allowed the project to clearly evaluate how optimized prompt engineering improved the educational quality of the AI tutor response.

5.1.1 Initial Multiple Prompt Generation Implementation

The following code shows the initial prompt comparison implementation used during the early prototype stage.

```
# Generate Responses

standard_response = model.generate_content(prompt)

basic_response = model.generate_content(basic_prompt)

structured_response = model.generate_content(structured_prompt)

few_shot_response = model.generate_content(few_shot_prompt)

optimized_response = model.generate_content(optimized_prompt)
```

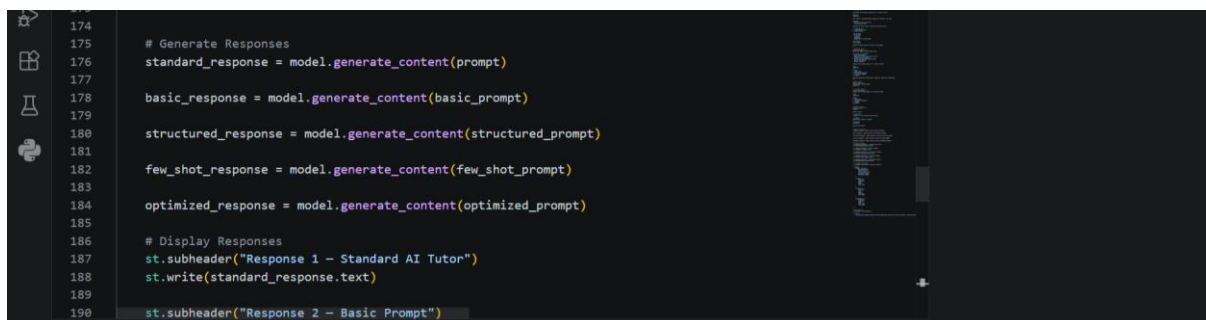


Figure 5. 1 Multiple prompt generation implementation using several prompt engineering techniques

5.1.2 Prompt Comparison Display Implementation

```
st.subheader("Response 1 – Standard AI Tutor")
st.write(standard_response.text)

st.subheader("Response 2 – Basic Prompt")
st.write(basic_response.text)

st.subheader("Response 3 – Structured Prompt")
```



```

st.write(structured_response.text)

st.subheader("Response 4 – Few-Shot Prompt")
st.write(few_shot_response.text)

st.subheader("Response 5 – Optimized Prompt")
st.write(optimized_response.text)

```

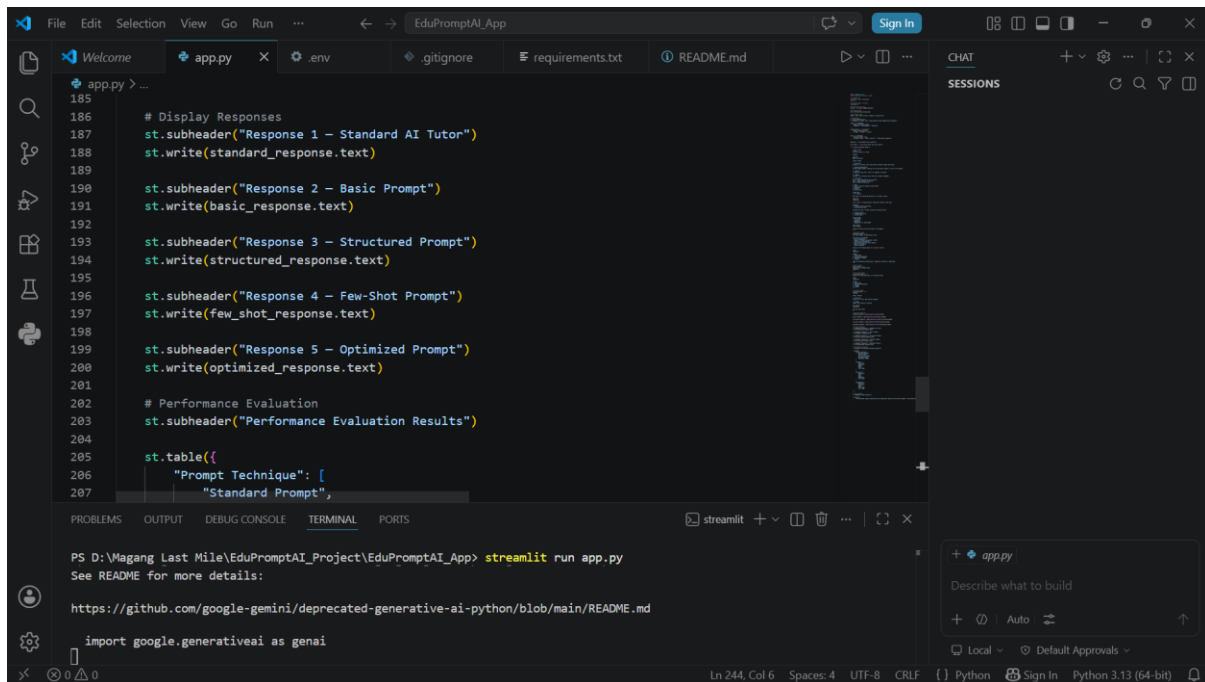


Figure 5. 2 Comparison of multiple AI-generated responses using different prompt engineering techniques

5.1.3 Final Prompt Comparison Engine Implementation in OpenRouter Version

After the initial prompt comparison engine was tested, the feature was adapted into the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

In the final implementation, the system compares two main response types: Basic Prompt Response and Optimized Prompt Response. The Basic Prompt Response generates a short and general explanation, while the Optimized Prompt Response applies multiple prompt engineering techniques, including structured prompting, learner-level adaptation, few-shot prompting, study plan generation, quiz generation, and role-based instructional guidance.

This final comparison helps demonstrate the improvement produced by prompt optimization in terms of structure, completeness, personalization, and educational usefulness.

```

basic_prompt = f"""
Explain this topic briefly:

{question}
"""

basic_response = client.chat.completions.create(
    model="deepseek/deepseek-chat",
    messages=[
        {
            "role": "user",
            "content": basic_prompt
        }
    ]
)

basic_output = basic_response.choices[0].message.content

response = client.chat.completions.create(
    model="deepseek/deepseek-chat",
    messages=[
        {
            "role": "user",
            "content": optimized_prompt
        }
    ]
)

optimized_response = response.choices[0].message.content

```

Figure 5. 3 Basic prompt response implementation using OpenRouter API

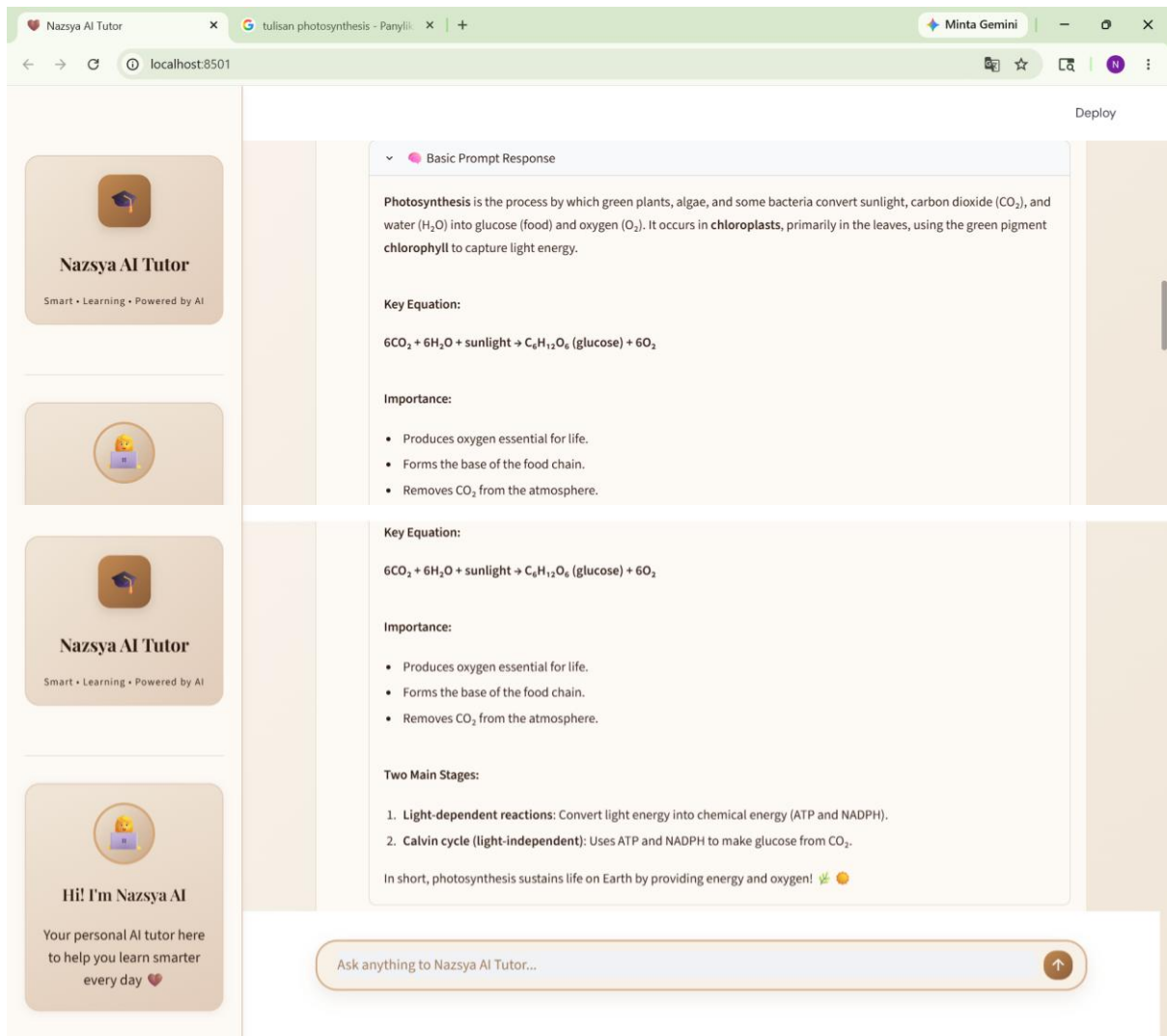
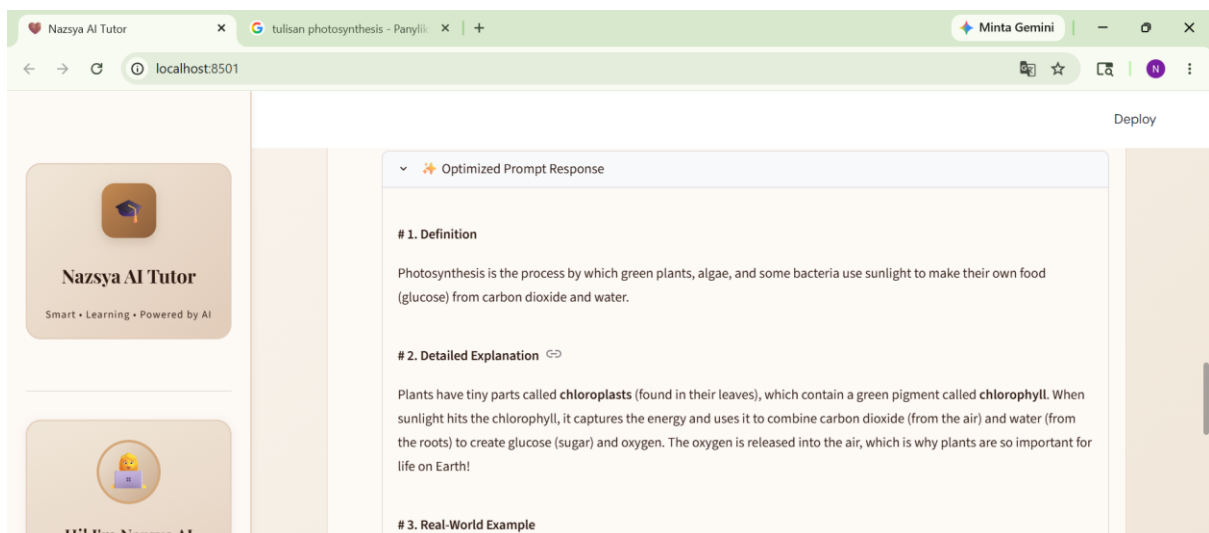


Figure 5. 4 Optimized prompt response implementation using OpenRouter API



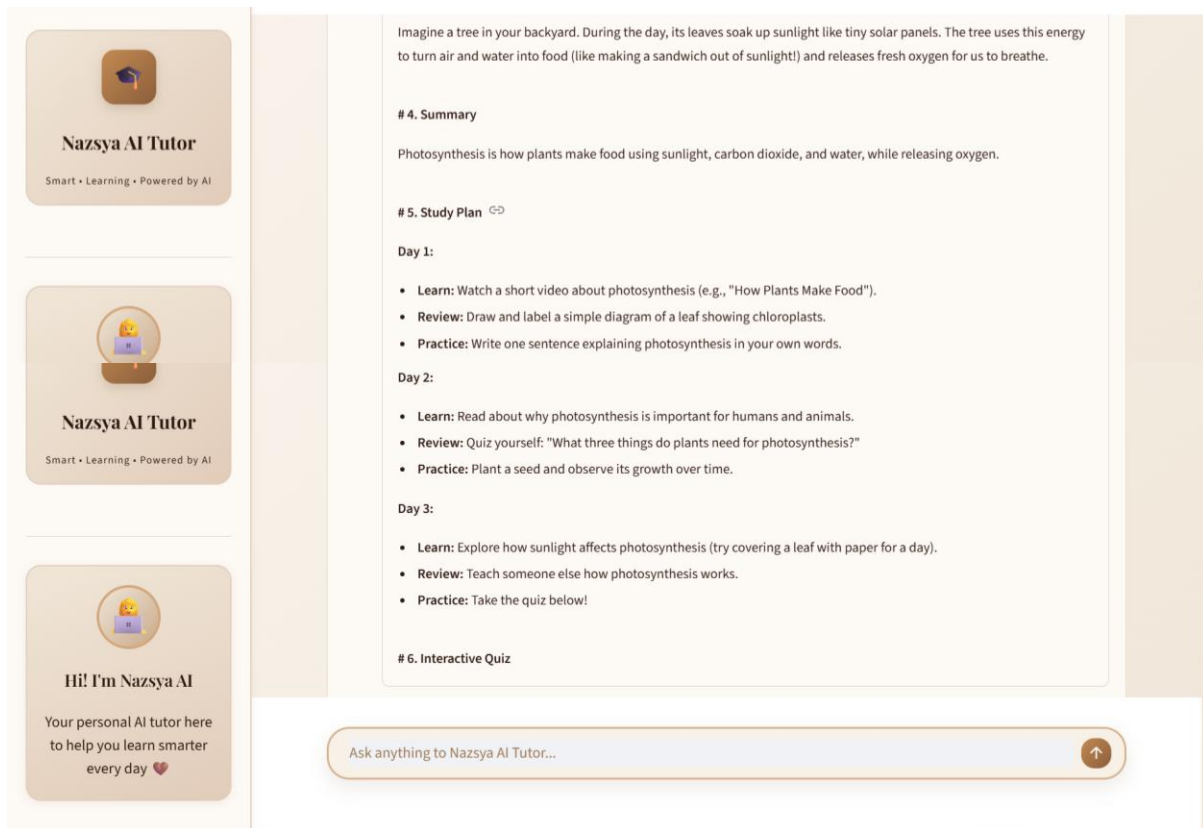


Figure 5. 5 Final comparison display between Basic Prompt Response and Optimized Prompt Response

Summary of prompt comparison output:

The Basic Prompt Response generated a general explanation of photosynthesis. It explained the definition, equation, importance, and main stages of photosynthesis. Although the response was useful, it was still presented as a general explanation and did not provide a complete learning structure.

The Optimized Prompt Response generated a more organized educational response. It included structured sections such as Definition, Detailed Explanation, Real-World Example, Summary, and Study Plan. The explanation was also more learner-friendly and used simple analogies to make the topic easier to understand.

This comparison shows that optimized prompting improved the structure, readability, and educational usefulness of the AI-generated response compared to the basic prompt.

5.1.4 Prompt Comparison Analysis

The prompt comparison experiment demonstrated that different prompt engineering strategies significantly affected the quality of AI-generated educational responses.

The Basic Prompt Response produced a short and general explanation of the topic. It was useful for quick understanding, but it did not include structured sections, learner-level adaptation, study plans, quizzes, or interactive evaluation.

In contrast, the Optimized Prompt Response generated a more complete educational output. It included Definition, Detailed Explanation, Real-World Example, Summary, Study Plan, and

Interactive Quiz. The optimized response was also adapted based on the selected learner level and evaluator role.

This shows that optimized prompting improved response structure, completeness, personalization, and educational usefulness compared to basic prompting.

The following table summarizes the comparison between the Basic Prompt Response and the Optimized Prompt Response in the final application.

Aspect	Basic Prompt Response	Optimized Prompt Response
Response Style	Short and general	Structured and educational
Output Format	Paragraph-based explanation	Definition, Explanation, Example, Summary, Study Plan, and Quiz
Learner Level Adaptation	Not available	Available
Study Plan	Not available	Available
Quiz	Not available	Available
Educational Usefulness	Basic understanding	Complete learning support
Prompt Engineering Technique	Basic Prompting	Optimized Prompting with multiple techniques

Based on the prompt comparison engine testing, the project concluded that optimized prompting produced more structured, complete, and educationally useful responses than basic prompting. The comparison engine helped demonstrate the impact of advanced prompt engineering techniques in improving the quality of AI-generated learning content.

5.2 Prompt Optimization

In this phase, the project optimized the prompt structure to improve educational response quality, clarity, organization, consistency, and personalization.

The optimization process was carried out by adding clearer instructions, structured response sections, learner-level adaptation, few-shot prompting examples, study plan generation, quiz generation, and role-based behavior rules.

In the final OpenRouter-based implementation, the optimized prompt guided the DeepSeek Chat model to generate more complete and educational responses. The optimized response was designed not only to explain a topic, but also to support learning through examples, summaries, study plans, quizzes, and role-based instructional guidance.

5.2.1 Initial Optimized Prompt Implementation

The following code shows the initial optimized prompt design used to improve the structure and clarity of AI-generated educational responses.

```
optimized_prompt = f"""
You are an expert AI educational tutor.
```

```

Your task is to provide:
- Clear explanations
- Beginner-friendly educational content
- Step-by-step explanations
- Concise and well-structured answers
- Real-world examples
- Simple summaries

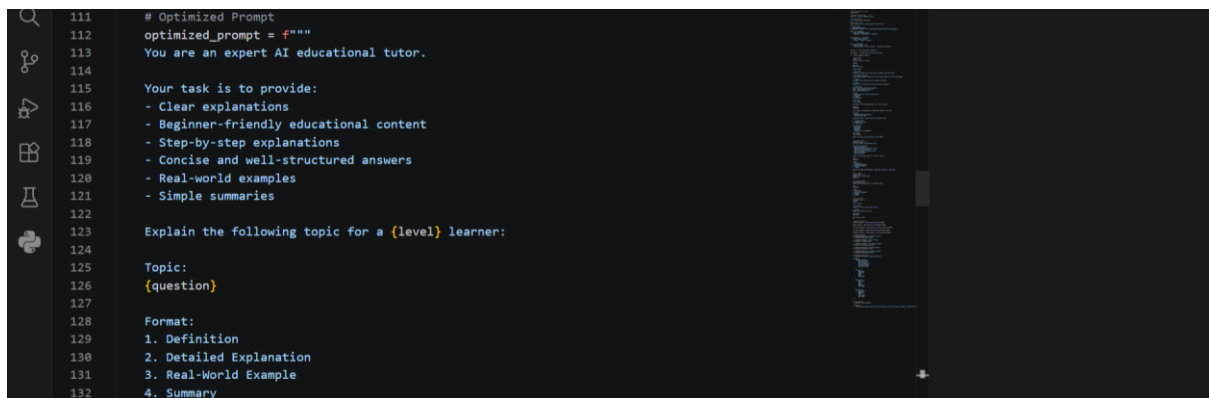
Explain the following topic for a {level} learner:

Topic:
{question}

Format:
1. Definition
2. Detailed Explanation
3. Real-World Example
4. Summary

Keep the explanation educational, organized, and easy to understand.
"""

```



```

111 # Optimized Prompt
112 optimized_prompt = f"""
113 You are an expert AI educational tutor.
114
115 Your task is to provide:
116 - Clear explanations
117 - Beginner-friendly educational content
118 - Step-by-step explanations
119 - Concise and well-structured answers
120 - Real-world examples
121 - Simple summaries
122
123 Explain the following topic for a {level} learner:
124
125 Topic:
126 {question}
127
128 Format:
129 1. Definition
130 2. Detailed Explanation
131 3. Real-World Example
132 4. Summary

```

Figure 5. 6 *Optimized prompt implementation for improving educational response quality*

Response 5 — Optimized Prompt

Hello there! I'm your AI educational tutor, and I'm excited to help you understand what photosynthesis is. Don't worry, we'll break it down step-by-step!

1. Definition

Photosynthesis is the process that plants, algae, and some bacteria use to convert light energy into chemical energy in the form of food (sugars). Think of it as how plants make their own snacks using sunlight!

2. Detailed Explanation

Let's break down how photosynthesis works:

- **The Ingredients:** Plants need three main things to perform photosynthesis:
 - **Sunlight:** This is the energy source. Plants have a special green pigment called **chlorophyll** (found mostly in their leaves) that's really good at capturing sunlight.
 - **Water (H₂O):** Plants absorb water from the soil through their roots.
 - **Carbon Dioxide (CO₂):** Plants take in carbon dioxide from the air through tiny pores on their leaves called stomata.

Figure 5. 7 *AI-generated educational response using optimized prompt engineering techniques*

5.2.2 Final Optimized Prompt Implementation in OpenRouter Version

After the initial optimized prompt was tested, the prompt structure was further improved and implemented in the final version of Nazsya AI Tutor using OpenRouter API and the DeepSeek Chat model.

The final optimized prompt combined several prompt engineering techniques, including structured prompting, few-shot prompting, learner-level adaptation, study plan generation, interactive quiz generation, and role-based instruction.

This final prompt allowed the system to generate a more complete educational response. The output included Definition, Detailed Explanation, Real-World Example, Summary, Study Plan, and Interactive Quiz. The response style was also adjusted based on the selected learner level and evaluator role.

```
321 optimized_prompt = """
322 You are an expert AI educational tutor.
323
324 Your task is to provide:
325 - Clear explanations
326 - Beginner-friendly educational content
327 - Step-by-step explanations
328 - Concise and well-structured answers
329 - Real-world examples
330 - Simple summaries
331
332 You are acting as a {role_input}.
333
334 Explain the following topic for a {level_input} learner.
335
336 Topic:
337 {question}
338
339 Use the following example only as a formatting guide.
340
341 Example:
342 Topic:
343 What is gravity?
344
345 # 1. Definition
346 Gravity is a natural force that attracts objects toward each other.
347
348 # 2. Detailed Explanation
349 Gravity keeps planets orbiting the sun and causes objects to fall toward Earth.
350
351 # 3. Real-World Example
352 A ball falling to the ground is an example of gravity.
353
354 # 4. Summary
355 Gravity is the force that pulls objects toward each other.
356
357 Now generate an answer for the actual topic using the same educational structure.
358 Do not answer the example topic. Only answer the actual user topic.
359
360 Format your response EXACTLY like this:
361
362 # 1. Definition
363 Explain the topic clearly.
364
365 # 2. Detailed Explanation
366 Provide a detailed explanation appropriate to the selected learner level.
367
368 # 3. Real-World Example
369 Give an easy real-world example.
370
371 # 4. Summary
372 Provide a short summary.
373
374 # 5. Study Plan
375 Create a {duration_input} study plan with daily learning activities, review tasks, and practice activities.
376
377 # 6. Interactive Quiz
378
379 Generate exactly 3 multiple-choice questions related to the topic.
380 Each question must have three options: A, B, and C.
381 Provide the correct answer key at the end of each question line.
```

```

383 Generate the quiz in EXACTLY this format:
384
385 QUIZ_START
386
387 Q1|Question here|Option A|Option B|Option C|A
388
389 Q2|Question here|Option A|Option B|Option C|B
390
391 Q3|Question here|Option A|Option B|Option C|C
392
393 QUIZ_END
394
395 The study plan must match the selected duration: {duration_input}.
396
397 You MUST adjust:
398 - explanation depth based on learner level
399 - feedback tone based on evaluator role
400
401 Learner level adaptation rules:
402 - If learner level is Beginner:
403   Use simple words, friendly tone, short explanations, and easy analogies.
404 - If learner level is Intermediate:
405   Use moderate academic terminology, deeper explanation, and explain relationships between concepts.
406 - If learner level is Advanced:
407   Use technical terms, formal academic tone, detailed mechanisms, and deeper scientific reasoning.
408
409 Role adaptation rules:
410 - If evaluator role is Friendly Tutor:
411   Use supportive and encouraging feedback.
412 - If evaluator role is Strict Lecturer:

```

Figure 5. 8 Final optimized prompt implementation using OpenRouter API

5.2.3 Prompt Optimization Analysis

The optimized prompt produced more educational, structured, and personalized AI responses compared to the basic prompt.

The final optimized prompt improved response quality by giving the AI clear instructions about output format, learner level, study duration, quiz format, and response style. As a result, the generated response became more organized and useful for learning.

Compared to the basic prompt, the optimized prompt produced a more complete educational output because it included structured explanations, real-world examples, summaries, study plans, and interactive quiz content.

Aspect	Basic Prompt	Final Optimized Prompt
Structure	General explanation	Highly structured educational format
Educational Clarity	Moderate	High
Readability	Good	Very good
Example Quality	Basic	Real-world contextual example
Learner Adaptation	Not available	Available
Study Plan	Not available	Available
Quiz Generation	Not available	Available
Few-Shot Guidance	Not available	Available
Consistency	Moderate	High

Based on the prompt optimization testing, the project concluded that the final optimized prompt improved the quality of AI-generated educational responses. The optimized prompt produced more structured, complete, consistent, and learner-friendly outputs compared to the basic prompt. This shows that prompt optimization plays an important role in improving the performance of Nazsya AI Tutor as an AI-powered educational assistant.

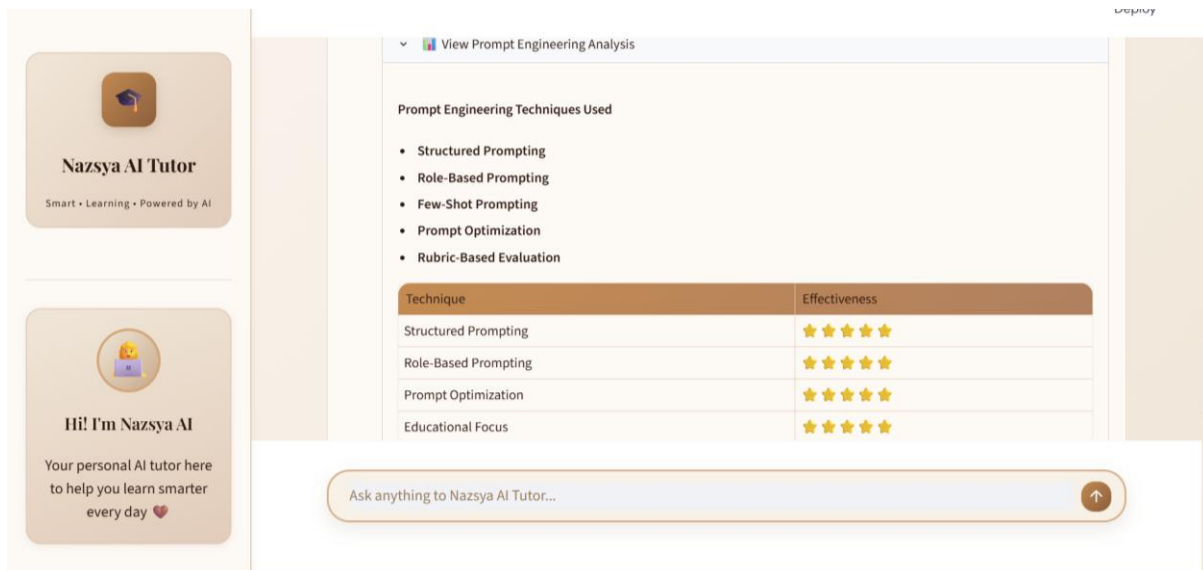


Figure 5.9 Prompt engineering analysis of techniques used in the final Nazsya AI Tutor application

The prompt engineering analysis section shows the main techniques implemented in the final Nazsya AI Tutor application, including Structured Prompting, Role-Based Prompting, Few-Shot Prompting, Prompt Optimization, and Rubric-Based Evaluation. This section helps summarize the prompt engineering strategies used to improve the quality of AI-generated educational responses.

5.3 Performance Evaluation

In this phase, the project evaluated the effectiveness of multiple prompt engineering techniques implemented in Nazsya AI Tutor. The evaluation focused on educational response quality metrics, including clarity, structure, and consistency.

The purpose of this evaluation was to compare how different prompting strategies affected the quality of AI-generated educational responses. The evaluated prompt techniques included Standard Prompting, Basic Prompting, Structured Prompting, Few-Shot Prompting, and Optimized Prompting.

The evaluation results showed that Optimized Prompting generated the highest overall educational response quality because it combined multiple prompt engineering techniques into one complete educational response format.

5.3.1 Performance Evaluation Implementation

The following code shows the performance evaluation implementation used to compare the effectiveness of several prompt engineering techniques.

The evaluation table compares five prompt techniques: Standard Prompt, Basic Prompt, Structured Prompt, Few-Shot Prompt, and Optimized Prompt. Each technique was evaluated based on three criteria: Clarity, Structure, and Consistency.

```
st.subheader("Performance Evaluation Results")
```

```

st.table({
    "Prompt Technique": [
        "Standard Prompt",
        "Basic Prompt",
        "Structured Prompt",
        "Few-Shot Prompt",
        "Optimized Prompt"
    ],

    "Clarity": [
        "Moderate",
        "Moderate",
        "High",
        "High",
        "Very High"
    ],

    "Structure": [
        "Moderate",
        "Low",
        "High",
        "Moderate",
        "Very High"
    ],

    "Consistency": [
        "Moderate",
        "Moderate",
        "High",
        "Very High",
        "Very High"
    ]
})

```

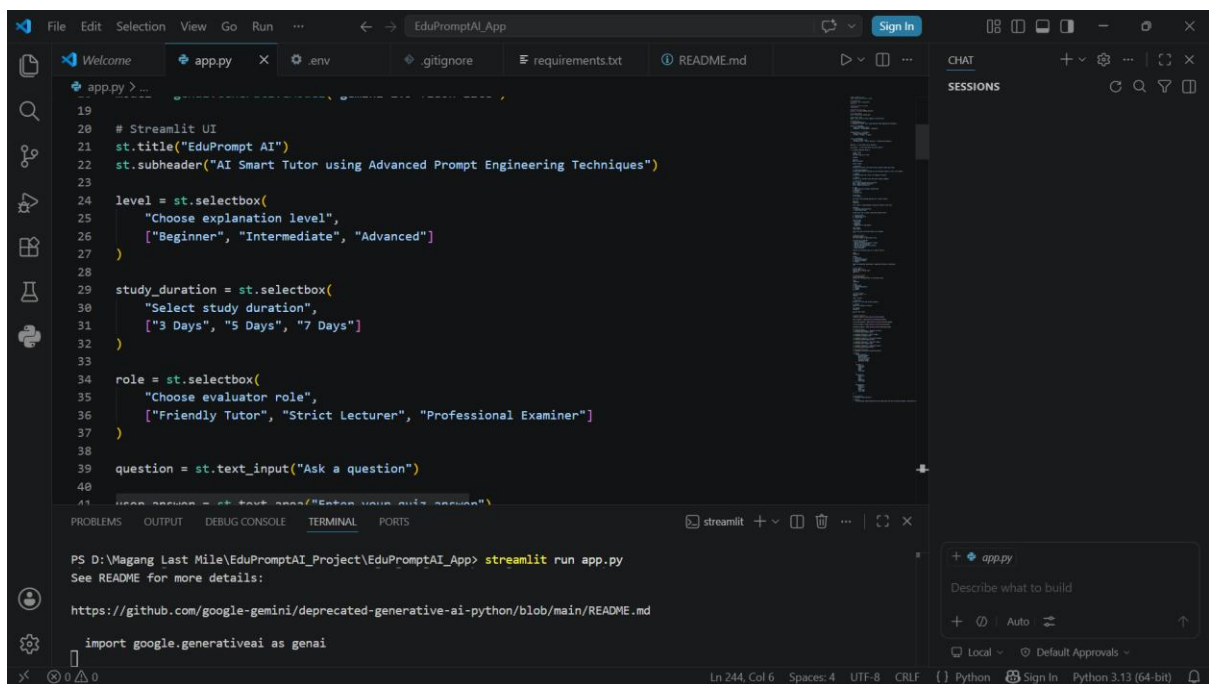


Figure 5. 10 Performance evaluation implementation for prompt engineering comparison

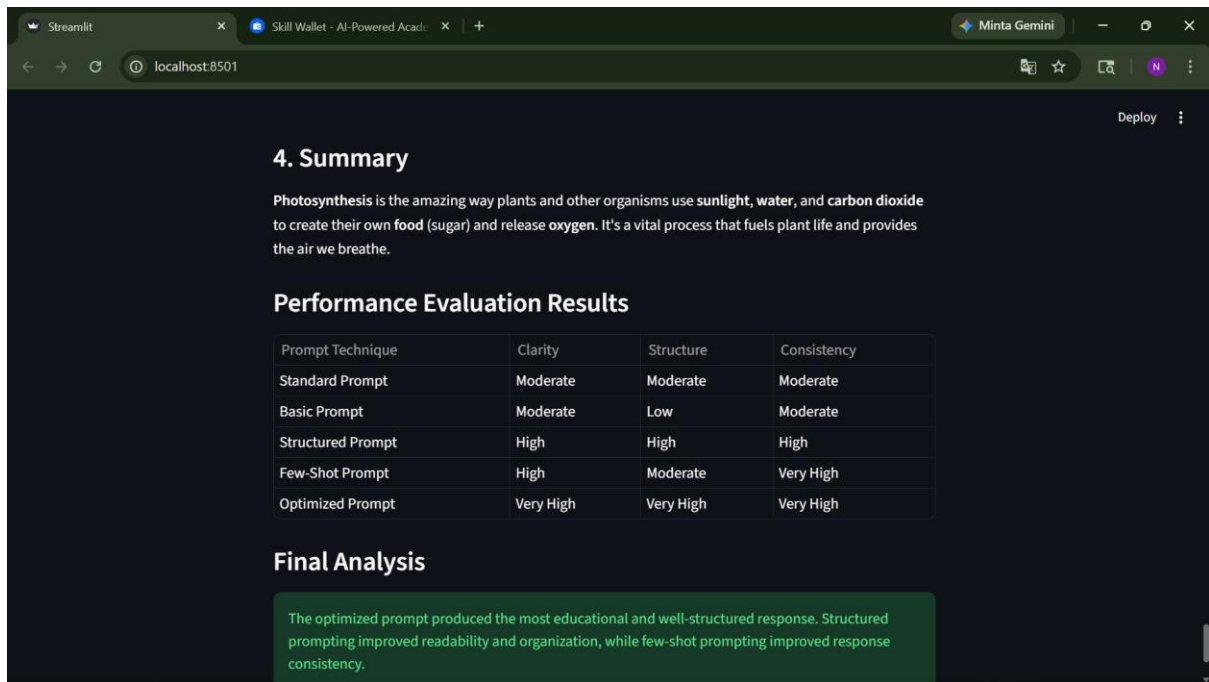


Figure 5. 11 Performance evaluation results and final analysis of multiple prompt engineering techniques

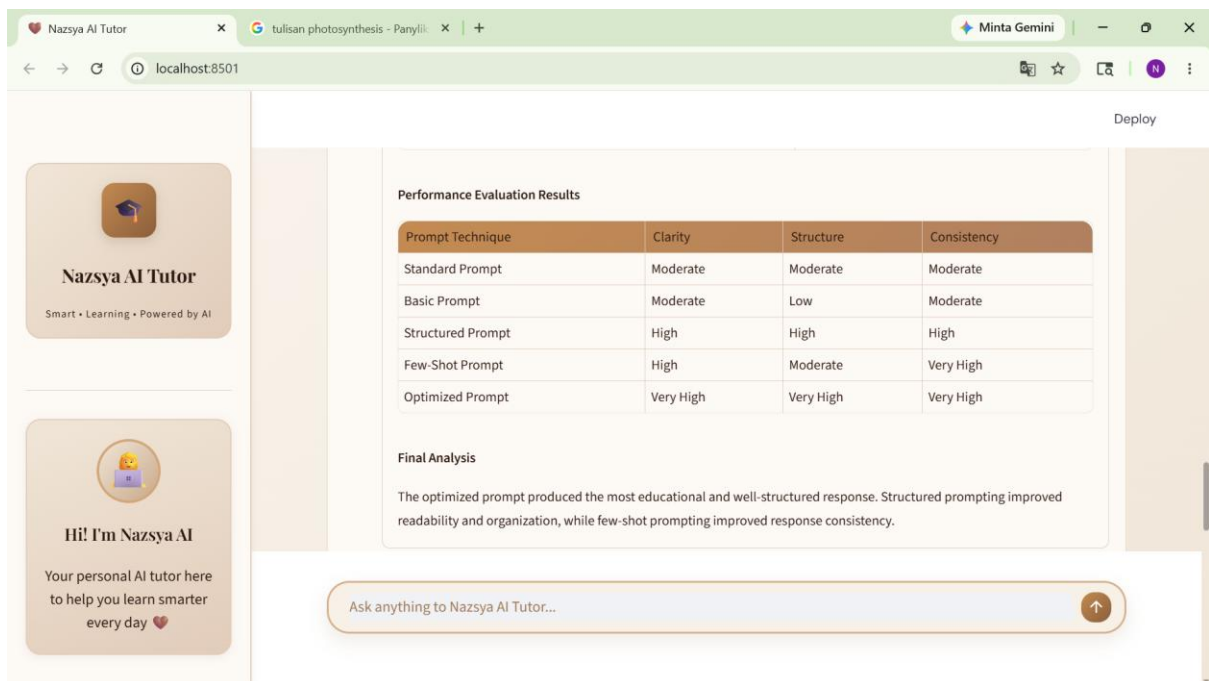


Figure 5. 12 Performance evaluation results of multiple prompt engineering techniques

Summary of performance evaluation output:

The performance evaluation compared several prompt engineering techniques, including Standard Prompting, Basic Prompting, Structured Prompting, Few-Shot Prompting, and Optimized Prompting.

The Standard Prompt generated a general educational response, while the Basic Prompt produced a longer but less structured explanation. The Structured Prompt improved organization by separating the response into clearer sections. The Few-Shot Prompt produced a more consistent format by following the provided example.

The Optimized Prompt produced the strongest result because it combined structured explanation, learner-friendly language, real-world examples, summary, study plan, and quiz generation. Based on the evaluation table, the Optimized Prompt achieved Very High ratings in clarity, structure, and consistency.

5.3.2 Performance Evaluation Analysis

The performance evaluation results confirmed that prompt engineering significantly influenced AI-generated educational responses.

Standard Prompting produced a general response, while Basic Prompting generated a longer explanation but with weaker structure. Structured Prompting improved response organization and readability. Few-Shot Prompting improved consistency because the AI followed the provided example format.

Optimized Prompting produced the best overall result because it combined several techniques, including structured prompting, few-shot prompting, learner-level adaptation, study plan generation, quiz generation, and role-based evaluation.

Based on the performance evaluation table, the Optimized Prompt achieved Very High ratings in clarity, structure, and consistency. This shows that prompt optimization improved the instructional effectiveness and educational quality of Nazsya AI Tutor.

The following table summarizes the best-performing prompt technique based on each evaluation aspect.

Evaluation Aspect	Best Performing Prompt	Reason
Clarity	Optimized Prompt	Provides clear and learner-friendly explanation
Structure	Optimized Prompt	Uses organized sections such as Definition, Explanation, Example, Summary, Study Plan, and Quiz
Consistency	Optimized Prompt	Follows a stable prompt format and output structure
Educational Usefulness	Optimized Prompt	Supports explanation, practice, study planning, and evaluation
Personalization	Optimized Prompt	Adapts response based on learner level and evaluator role

Based on the performance evaluation, the project concluded that optimized prompting produced the highest educational response quality compared to other prompt engineering

techniques. The optimized prompt improved clarity, structure, consistency, and overall learning usefulness.

This evaluation demonstrated that advanced prompt engineering techniques play an important role in improving the performance of Nazsya AI Tutor as an AI-powered educational assistant.

M6. Finalization, Documentation, and Presentation

This milestone focuses on finalizing the Nazsya AI Tutor project by completing the final project report, organizing the prompt template library, documenting the final application interface, and preparing the project presentation.

The finalization stage is important because it summarizes the development process, explains the final system features, documents the implemented prompt engineering techniques, and prepares the project for final submission and demonstration.

6.1 Final Project Report Writing

The final project report was prepared to document the complete development process of Nazsya AI Tutor. The report explains the project background, system setup, prompt engineering implementation, feature development, testing process, performance evaluation, and final application documentation.

This section summarizes the final system overview, development challenges, system workflow, UI design planning, and final application interface documentation.

6.1.1 Final System Overview

Nazsya AI Tutor is an AI-powered educational assistant developed to help users learn topics interactively using advanced prompt engineering techniques. The system is designed to generate structured explanations, personalized study plans, interactive quizzes, and AI-based answer evaluation.

The final application was built using Streamlit as the user interface framework and integrated with OpenRouter API using the DeepSeek Chat model as the final Large Language Model. During the early development stage, Google Gemini API was used for initial prompt exploration and prototype testing. However, the final implementation was migrated to OpenRouter API to improve testing flexibility and system stability.

The main purpose of Nazsya AI Tutor is to provide a more personalized and interactive learning experience. Users can select their learning level, study duration, and evaluator role before asking a question. Based on these settings, the system generates educational responses that are structured, learner-friendly, and suitable for different learning needs.

The final system includes several main features:

1. Structured educational explanation
2. Multi-level explanation generation
3. Personalized study plan generator
4. Interactive quiz generator
5. AI answer evaluator
6. Rubric-based evaluation
7. Role-based evaluator feedback
8. Basic and optimized prompt comparison

9. Prompt engineering analysis
10. Performance evaluation table

Overall, Nazsya AI Tutor demonstrates how prompt engineering techniques can improve the quality, structure, consistency, and educational usefulness of AI-generated learning content.

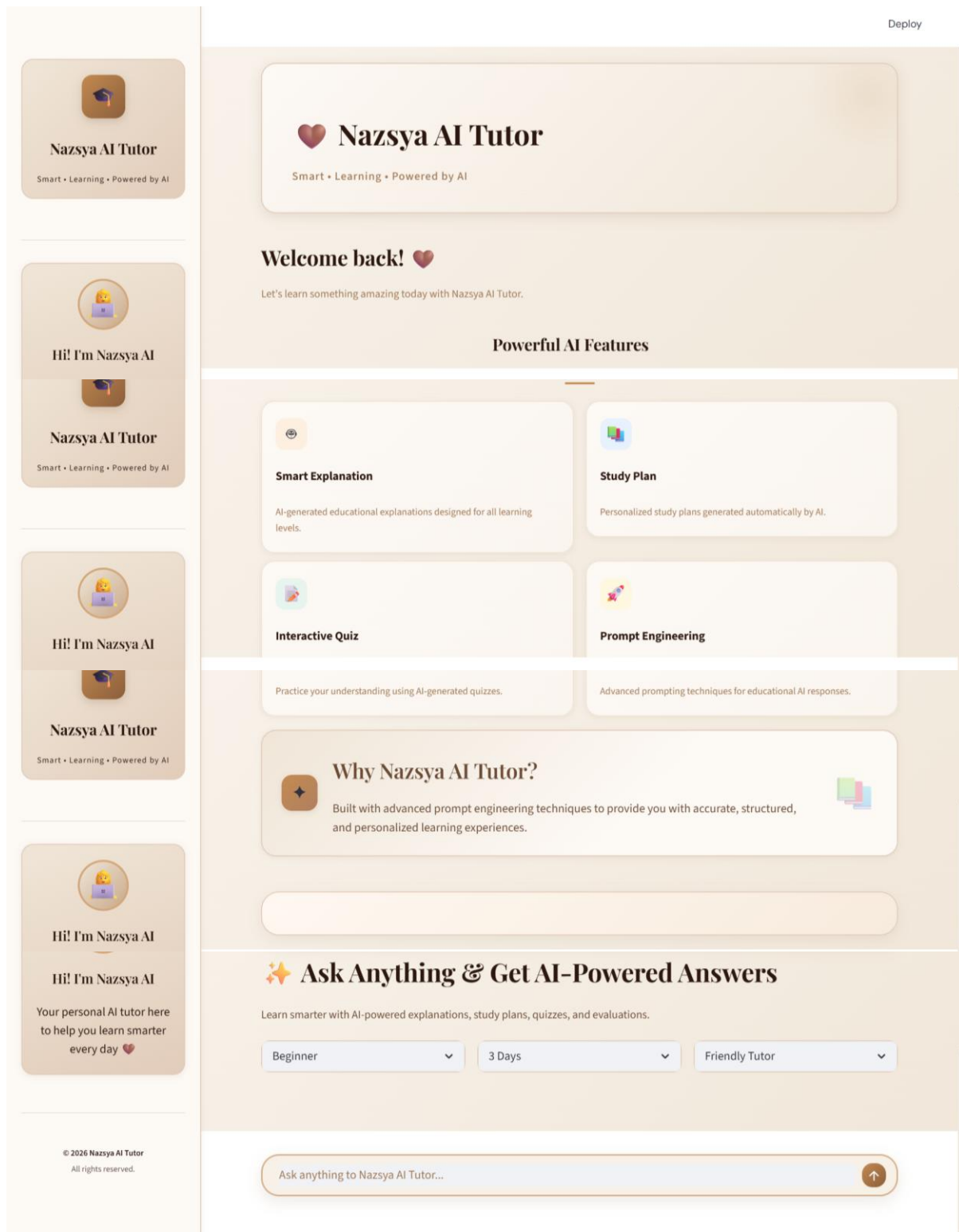


Figure 6. 1 Final interface overview of Nazsya AI Tutor

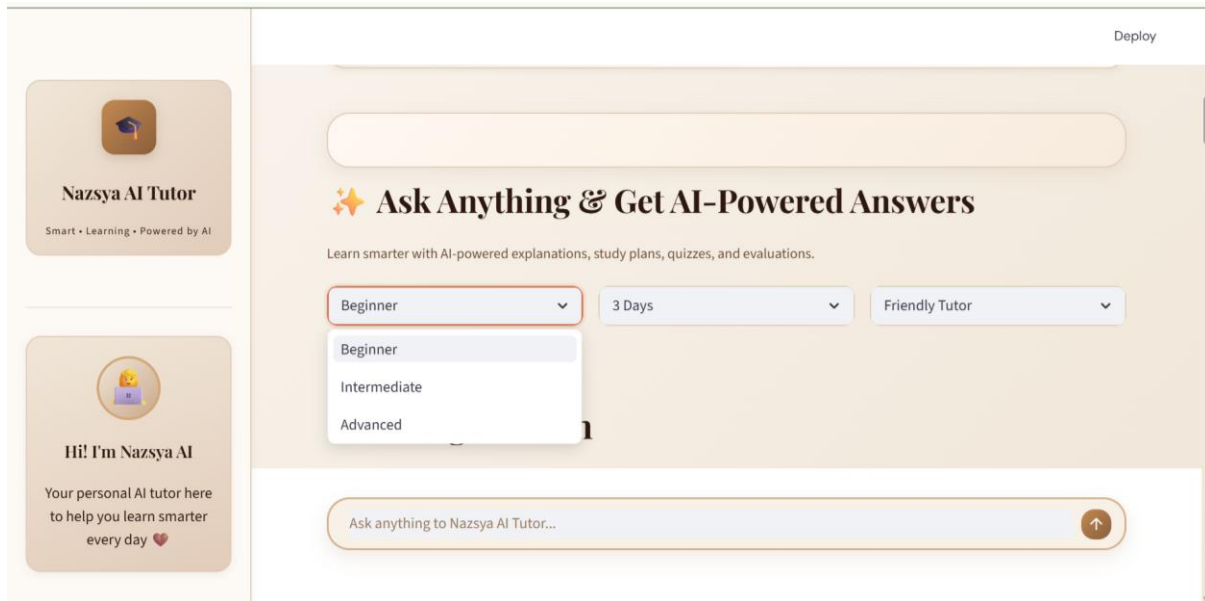


Figure 6. 2 Main features of Nazsya AI Tutor

6.1.2 Development Challenges and Improvements

During the development of Nazsya AI Tutor, several challenges were encountered and resolved through iterative improvements. These challenges were related to API limitations, model configuration, prompt consistency, quiz parsing, evaluator feedback quality, and user interface refinement.

One of the main challenges occurred during the early development stage when the project used Google Gemini API for prompt exploration and prototype testing. The project encountered API quota limitations and model configuration issues during repeated testing. Since the application required multiple AI responses for structured explanations, study plans, quizzes, answer evaluation, and prompt comparison, the initial API setup became less stable for continuous testing.

To solve this problem, the final implementation was migrated from Google Gemini API to OpenRouter API using the DeepSeek Chat model. This migration improved testing flexibility and allowed the application to continue generating educational responses consistently. Although the API provider and model were changed, the main prompt engineering workflow remained the same.

Another challenge was maintaining consistent AI-generated output. In the early stages, AI responses were sometimes too general, unstructured, or inconsistent. To improve this, the prompt was refined using structured prompting, few-shot prompting, learner-level adaptation, role-based prompting, and rubric-based evaluation.

The interactive quiz feature also required improvement. Initially, the quiz was generated as plain text, which made it difficult for the application to process automatically. The final version used QUIZ_START and QUIZ_END delimiters with a fixed question format so that the quiz

could be parsed and displayed as interactive multiple-choice questions.

The Answer Evaluator feature was also improved during development. The initial evaluator only provided general feedback, while the final evaluator used rubric-based and role-based prompting. This allowed the system to evaluate user answers based on Accuracy, Completeness, and Clarity while adjusting the feedback tone according to the selected evaluator role.

Several user interface improvements were also implemented. The application interface was refined by improving the sidebar, feature cards, input section, response layout, quiz display, evaluator section, and prompt comparison area. These improvements helped make the final application more organized, user-friendly, and suitable for educational use.

Overall, the development challenges helped improve the final quality of Nazsya AI Tutor. The project became more stable, interactive, structured, and educationally useful through continuous testing and refinement.

```
# Configure OpenRouter client
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
)
```

Figure 6. 3 Final OpenRouter API integration in Nazsya AI Tutor

```
if quiz_lines:
    st.markdown("## 🎮 Try the Quiz Yourself")
    user_answers = []

    for idx, line in enumerate(quiz_lines):
        parts = line.split("|")

        if len(parts) == 6:
            qnum = parts[0]
            question_text = parts[1]
            option_a = parts[2]
            option_b = parts[3]
            option_c = parts[4]
            correct_answer = parts[5].strip().upper()

            st.markdown(f"### {qnum}")
            st.write(question_text)
```

Figure 6. 4 Interactive quiz parsing improvement using structured quiz delimiters

6.1.3 System Workflow

The workflow of Nazsya AI Tutor begins when the user opens the application and enters a learning topic or question. Before generating the AI response, the user can customize several learning settings, including learner level, study duration, and evaluator role.

The learner level selection allows the user to choose between Beginner, Intermediate, and Advanced. This setting helps the system adjust the complexity, terminology, and depth of the generated explanation. The study duration selection allows the user to choose 3 Days, 5 Days, or 7 Days, which is used to generate a personalized study plan. The evaluator role selection allows the user to choose the feedback style, such as Friendly Tutor, Strict Lecturer, or Professional Examiner.

After the user submits a question, the system creates two types of prompts: a basic prompt and an optimized prompt. The basic prompt generates a simple and general explanation, while the optimized prompt applies advanced prompt engineering techniques such as structured prompting, few-shot prompting, learner-level adaptation, study plan generation, quiz generation, role-based prompting, and rubric-based evaluation.

The prompts are then sent to the Large Language Model through OpenRouter API using the DeepSeek Chat model. The AI processes the prompts and returns educational responses to the Streamlit application.

The final output includes a structured explanation, real-world example, summary, personalized study plan, interactive quiz, and answer evaluator. The application also displays a comparison between the basic prompt response and the optimized prompt response to demonstrate the impact of prompt engineering techniques.

The overall system workflow can be summarized as follows:

1. User opens the Nazsya AI Tutor application.
2. User selects learner level, study duration, and evaluator role.
3. User enters a learning topic or question.
4. The system generates a basic prompt and an optimized prompt.
5. The prompts are sent to OpenRouter API using the DeepSeek Chat model.
6. The AI generates educational responses.
7. The application displays the basic and optimized responses.
8. The system extracts and displays the interactive quiz.
9. User answers the quiz and receives a score.
10. User submits an answer to the AI Answer Evaluator.
11. The AI evaluates the answer using rubric-based and role-based prompting.
12. The system displays feedback, score, strengths, weaknesses, and suggestions.

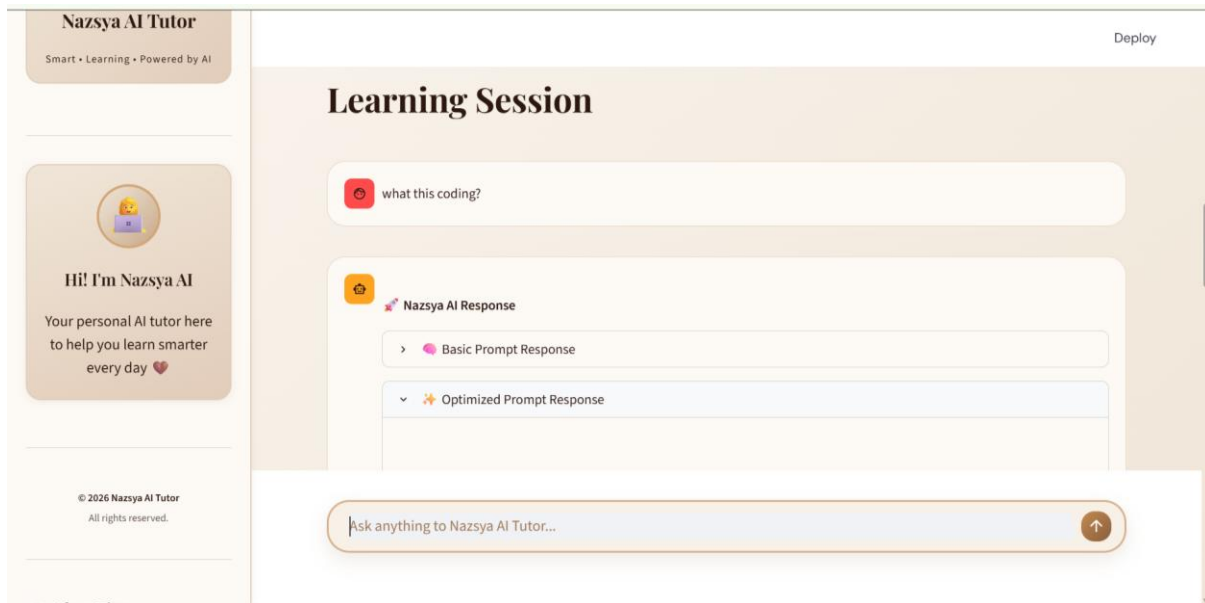


Figure 6. 5 User input and AI response workflow in Nazsya AI Tutor

6.1.4 Code Structure Improvement

Code Structure Improvement and Modularization

During the final development stage, the application code was improved by restructuring the project into multiple Python files. In the earlier version, most of the application logic was written in a single `app.py` file. This made the code long, less organized, and more difficult to maintain.

To improve readability and maintainability, the code was refactored into several separate modules. Each file was assigned a specific responsibility, such as API configuration, AI response generation, prompt templates, quiz processing, answer evaluation, user interface components, and prompt engineering analysis.

The modular file structure is shown below:

File Name	Function
<code>app.py</code>	Main application file that controls the overall Streamlit workflow
<code>config.py</code>	Stores API configuration, model name, and API key validation
<code>ai_client.py</code>	Handles the OpenRouter API connection and AI response generation
<code>prompts.py</code>	Stores the basic prompt, optimized prompt, and evaluator prompt templates
<code>quiz.py</code>	Handles quiz extraction, quiz display, answer checking, and score calculation

<code>evaluator.py</code>	Handles the AI Answer Evaluator feature using rubric-based and role-based evaluation
<code>ui_components.py</code>	Stores reusable user interface components such as header, sidebar, feature cards, and input section
<code>analysis_section.py</code>	Displays prompt engineering analysis and performance evaluation tables
<code>assets/style.css</code>	Stores the custom CSS used to style the Streamlit interface
<code>.env</code>	Stores the OpenRouter API key securely

This restructuring did not change the main features of Nazsya AI Tutor. The application still supports basic response generation, optimized educational response generation, interactive quiz generation, AI answer evaluation, prompt engineering analysis, and performance evaluation. However, the new modular structure makes the code cleaner, easier to understand, easier to debug, and more professional for future development.

This improvement also supports better project documentation because each feature can be explained based on its related file. For example, quiz functionality is handled in `quiz.py`, evaluator functionality is handled in `evaluator.py`, and prompt templates are managed in `prompts.py`. As a result, the final application becomes more organized and easier to present during the project demonstration.

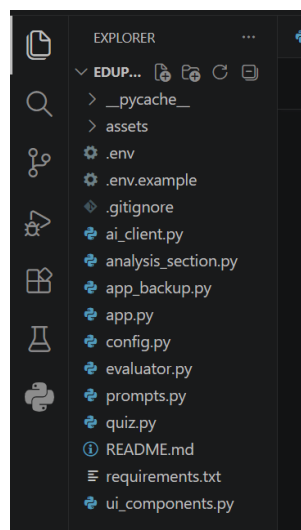


Figure 6. 6 Final modular code structure of Nazsya AI Tutor in Visual Studio Code

6.1.5 UI Design Planning and Figma Prototype

Before finalizing the Streamlit application interface, the UI design of Nazsya AI Tutor was planned using Figma. The purpose of the Figma prototype was to visualize the application layout, user flow, feature placement, and overall interface structure before documenting the final application result.

The UI design followed a modern educational dashboard concept. The interface was designed to be simple, clean, and user-friendly so that learners could easily access the main features of

the application. The design also used a soft color palette to create a comfortable learning atmosphere.

The Figma prototype included several main interface sections, such as the application sidebar, main dashboard, feature cards, user input area, learning settings, AI response section, interactive quiz section, and answer evaluator section.

The UI design planning helped ensure that the final application interface was organized and suitable for educational use. It also provided a visual reference for documenting the final system interface in the project report.

The main UI components designed in the Figma prototype include:

1. Main dashboard interface
2. Sidebar and branding section
3. Feature cards section
4. Prompt input and learning settings
5. Learning session response layout
6. Interactive quiz interface
7. AI answer evaluator section
8. Prompt comparison and performance evaluation section

Overall, the Figma prototype supported the finalization process by providing a clear visual plan

for the Nazsya AI Tutor interface.

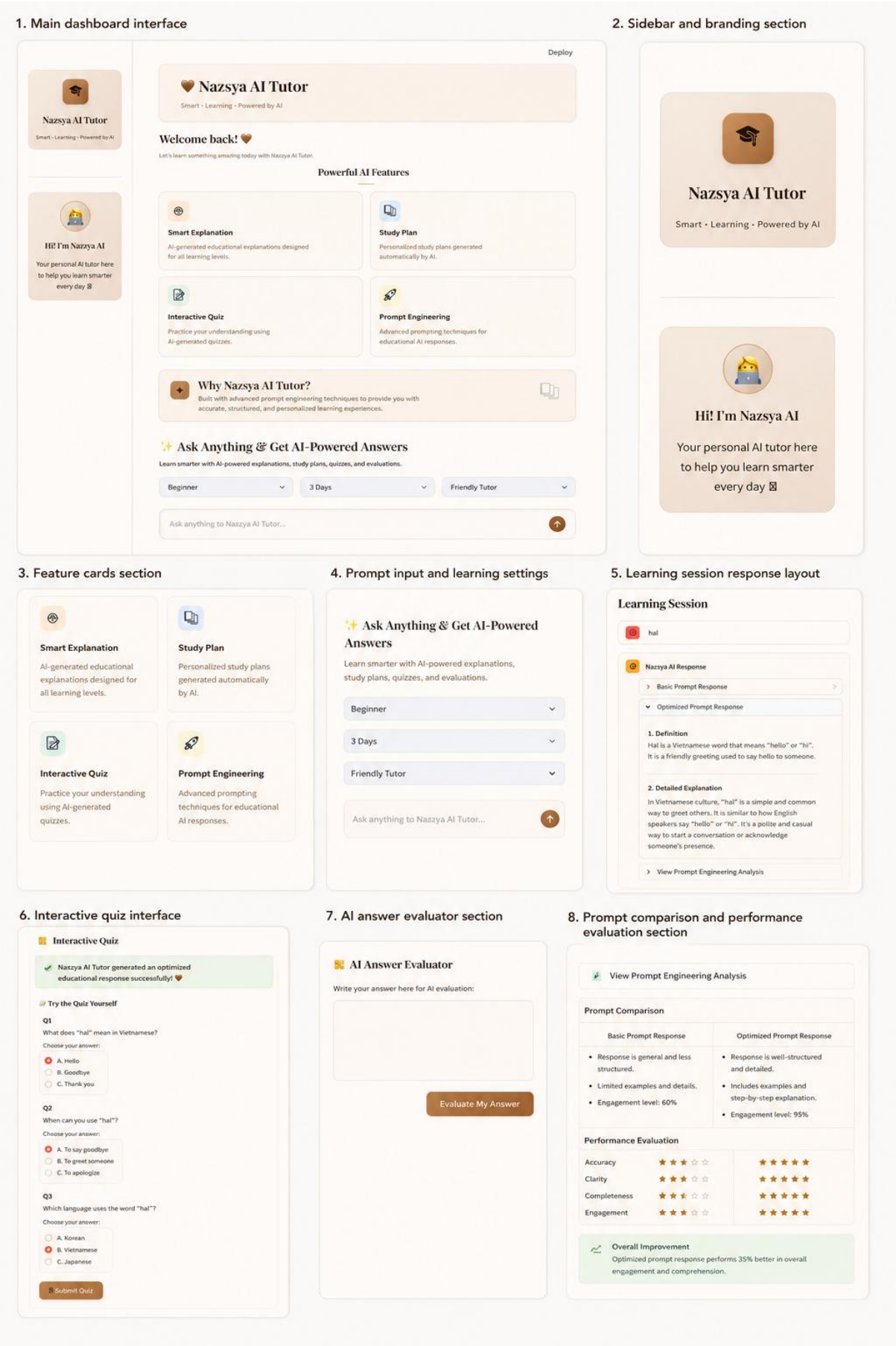


Figure 6. 7 Figma prototype overview of Nazsya AI Tutor

6.1.6 Final Application Interface Documentation

The final application interface of Nazsya AI Tutor was documented to show the completed system features and user interaction flow. The documentation includes the main learning session, basic and optimized prompt responses, prompt engineering analysis, interactive quiz, quiz evaluation result, performance evaluation table, and AI answer evaluator.

The final interface was designed using Streamlit with a clean educational dashboard layout. The sidebar contains the application branding and assistant introduction, while the main area displays the learning session and AI-generated educational outputs.

The application allows users to enter a learning question and receive two types of AI responses: Basic Prompt Response and Optimized Prompt Response. The Basic Prompt Response provides a simple and general explanation, while the Optimized Prompt Response generates a more structured and educational response using advanced prompt engineering techniques. The final application also includes an interactive quiz section. The quiz allows users to select answers using multiple-choice radio buttons and submit their answers directly in the application. After submission, the system displays the quiz score and feedback.

In addition, the AI Answer Evaluator allows users to write their own answer and receive rubric-based feedback. The evaluator assesses the answer based on Accuracy, Completeness, and Clarity, while also adjusting the feedback style based on the selected evaluator role.

The final interface documentation shows that Nazsya AI Tutor successfully integrates educational explanation, prompt comparison, quiz generation, answer evaluation, and performance analysis into one interactive AI-powered learning application.

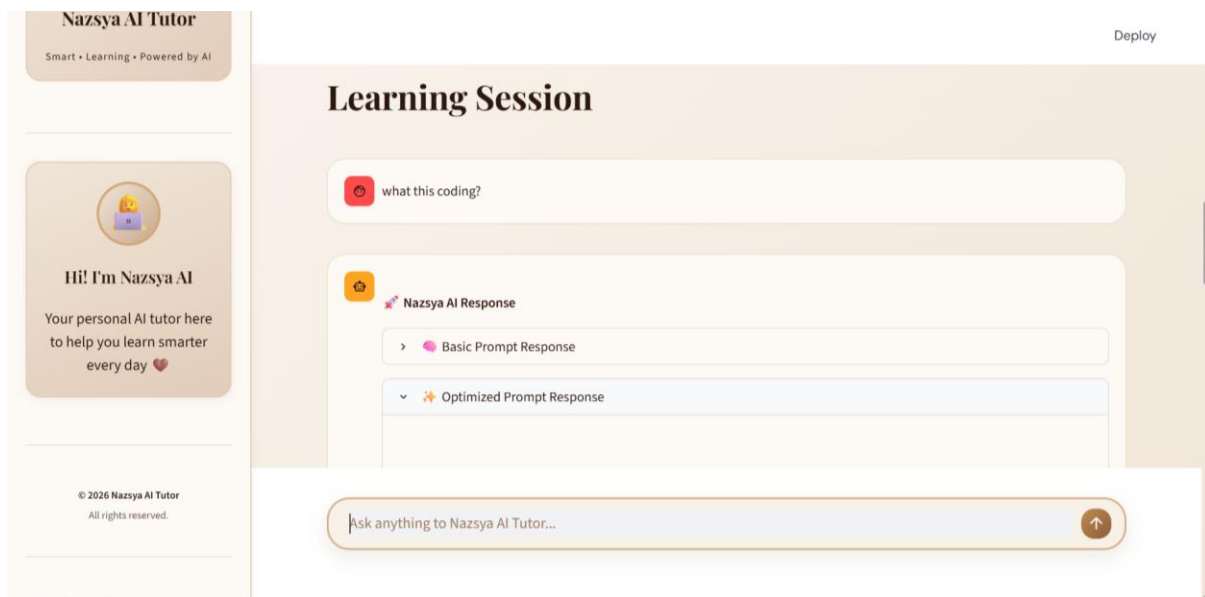


Figure 6. 8 Learning session interface with basic and optimized prompt response sections

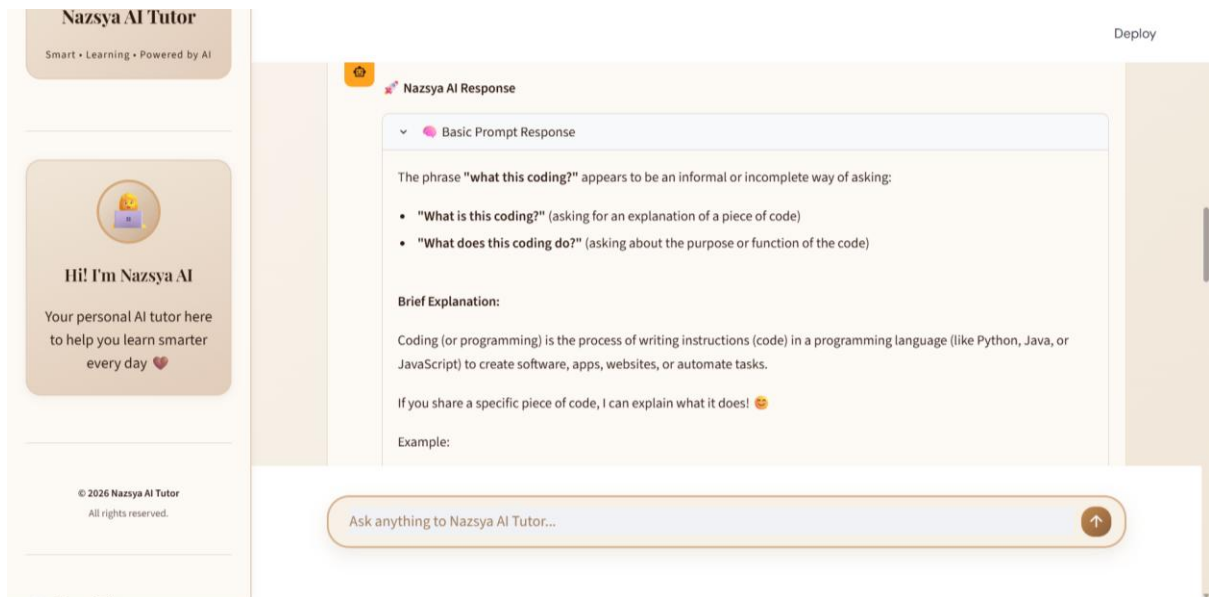


Figure 6. 9 Basic prompt response result in the final Nazsya AI Tutor application

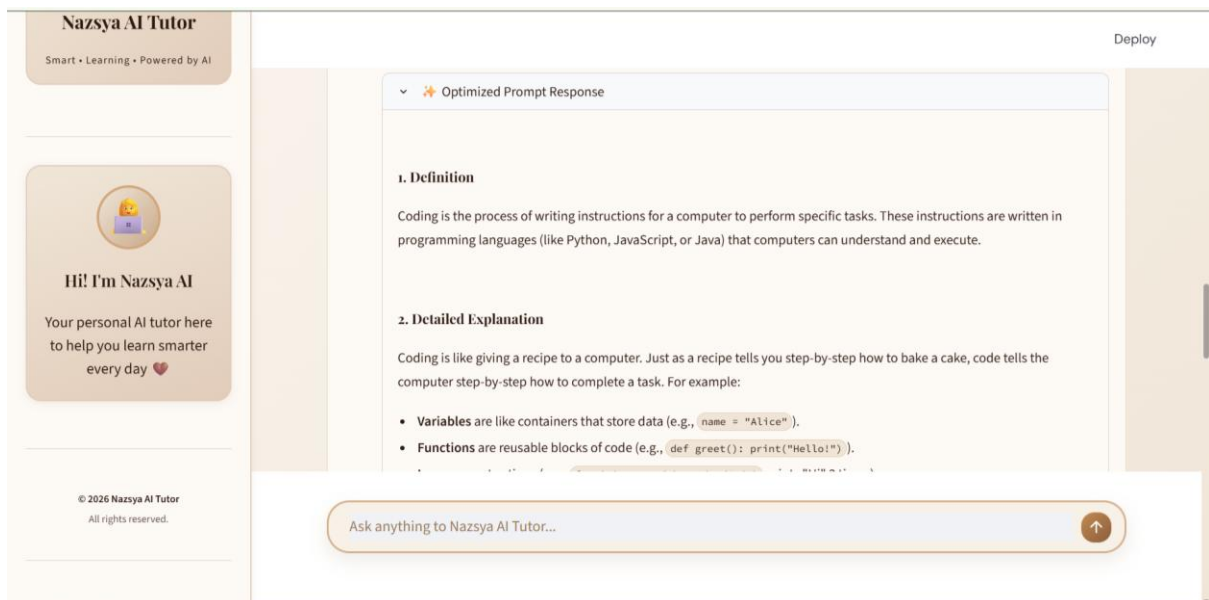


Figure 6. 10 Optimized prompt response result using advanced prompt engineering techniques

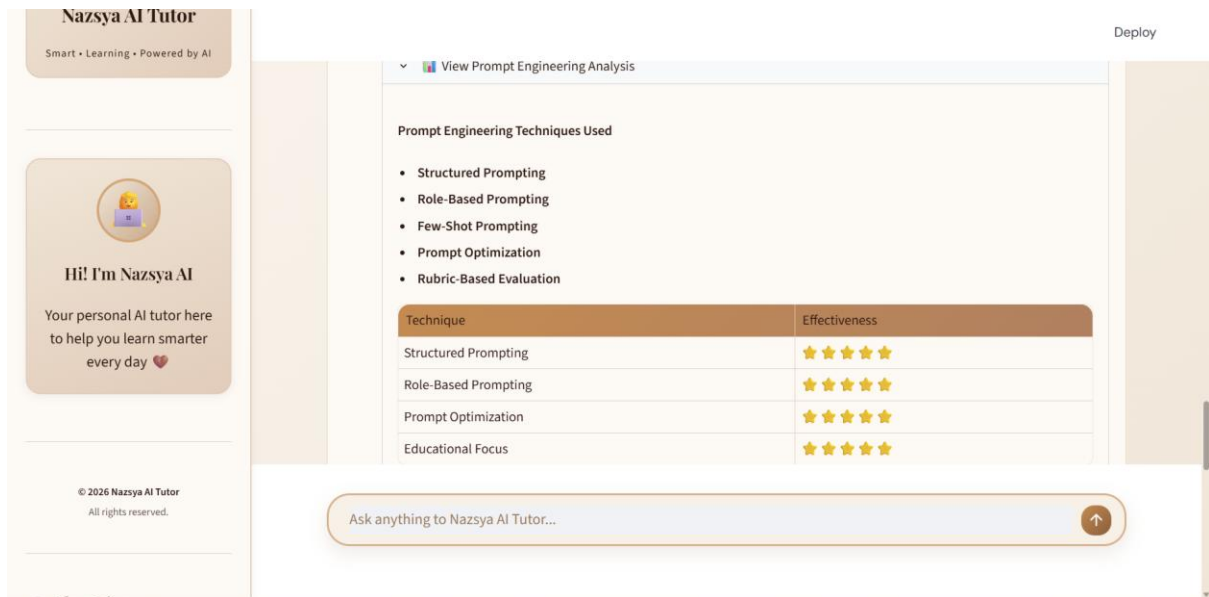


Figure 6. 11 Prompt engineering analysis section in Nazsya AI Tutor

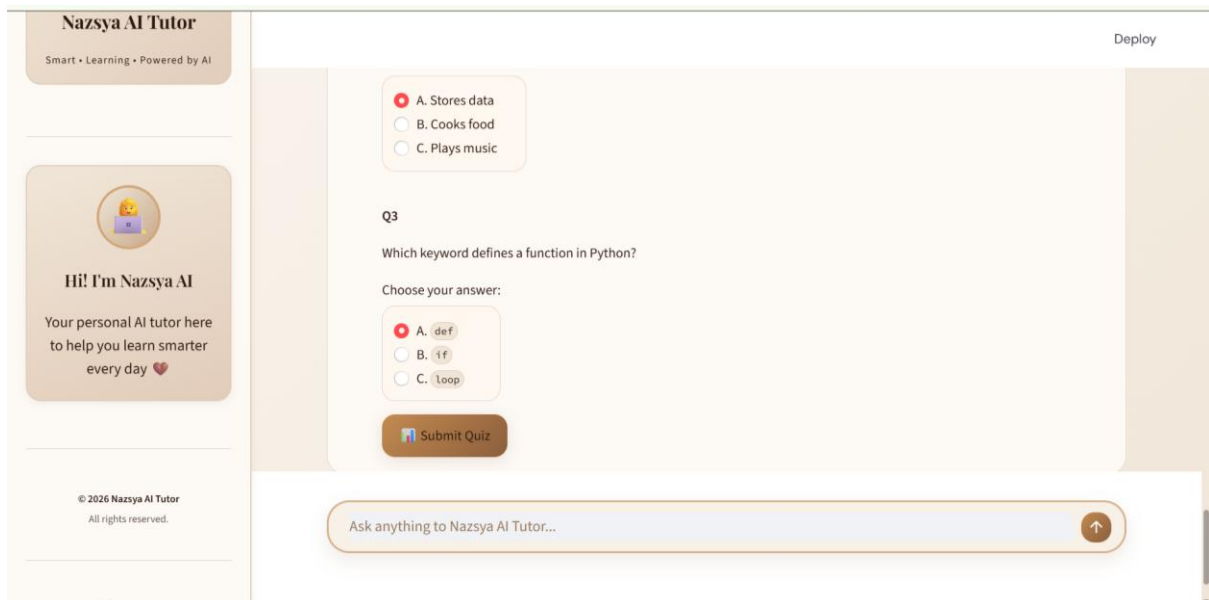


Figure 6. 12 Quiz submission interface in Nazsya AI Tutor

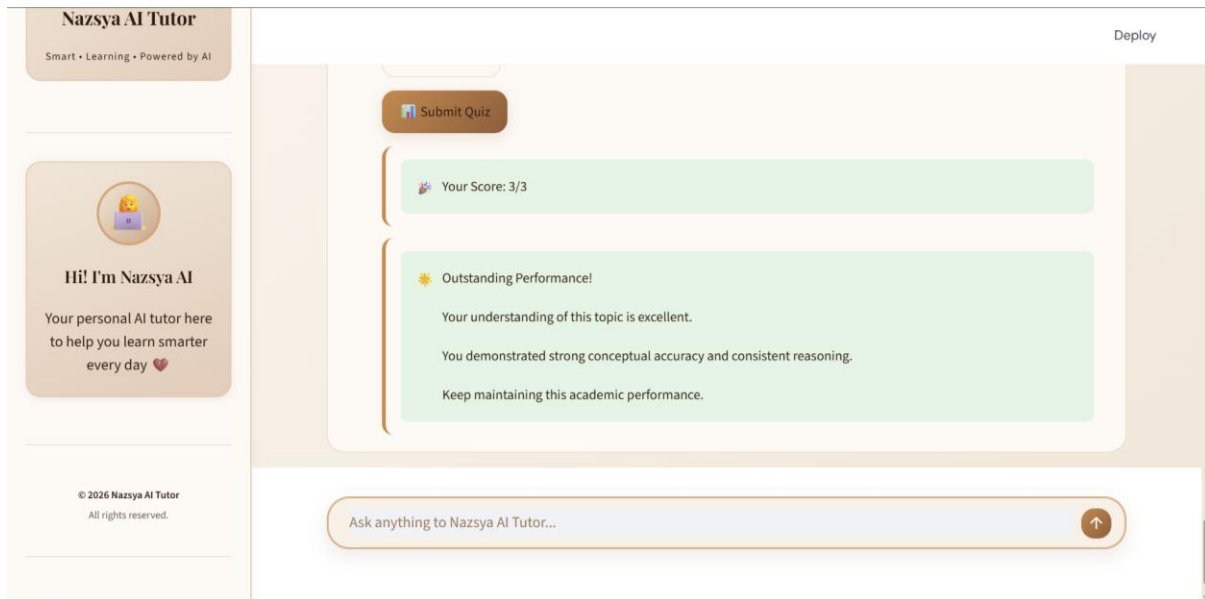


Figure 6. 13 *Performance evaluation results in the final application*

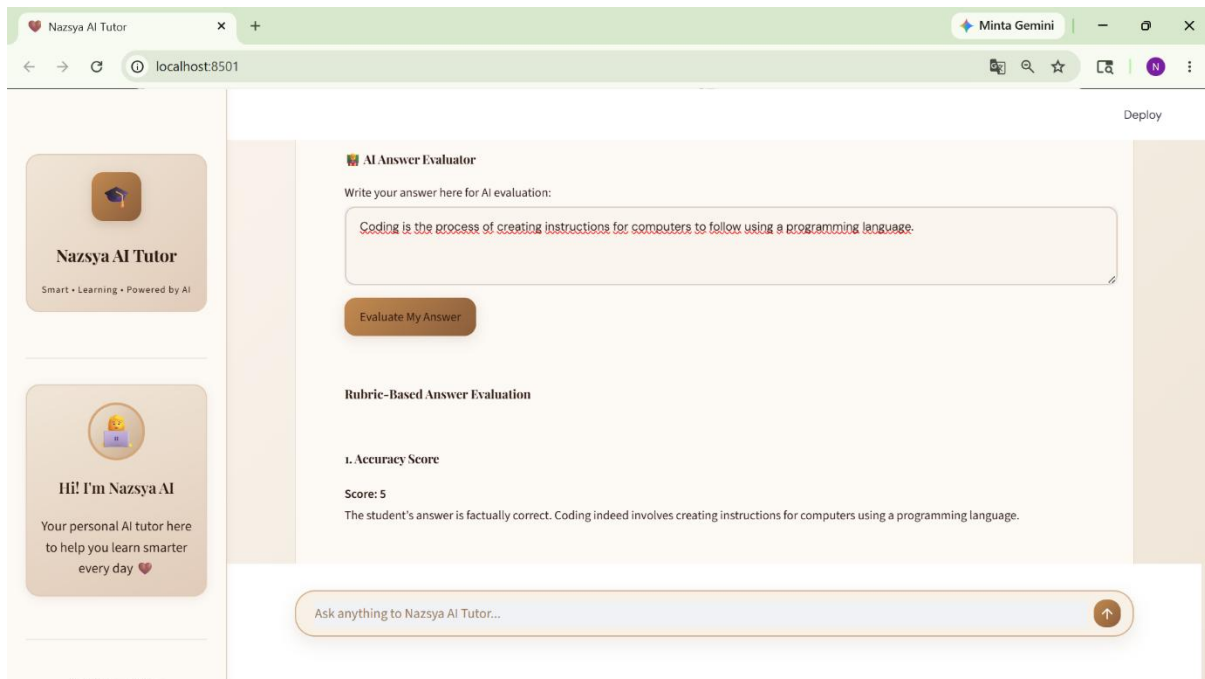


Figure 6. 14 *AI answer evaluator interface and rubric-based feedback*

6.2 Prompt Template Library Creation

The Prompt Template Library was created to document the main prompt structures used in Nazsya AI Tutor. This library helps explain how different prompt engineering techniques were implemented in the system to generate structured explanations, study plans, quizzes, and evaluator feedback.

The prompt templates were designed to support the main features of the application, including basic response generation, optimized educational response generation, interactive quiz

generation, and answer evaluation.

This section presents the main prompt templates used in the final application:

1. Basic Prompt Template
2. Optimized Prompt Template
3. Quiz Generation Prompt Template
4. Evaluator Prompt Template

6.2.1 Basic Prompt Template

The Basic Prompt Template was used to generate a simple and general explanation of the user's question. This prompt does not include advanced instructions such as structured formatting, learner-level adaptation, study plan generation, quiz generation, or evaluator feedback.

The purpose of the basic prompt was to provide a comparison baseline against the optimized prompt. By comparing the basic prompt response and the optimized prompt response, the project could demonstrate how prompt engineering improved the quality of AI-generated educational content.

```
Basic Prompt Template:  
Explain this topic briefly:  
{question}
```

In this template, {question} represents the learning topic or question entered by the user. The AI model generates a short response based only on the basic instruction.

The basic prompt is useful for quick explanations, but it is limited because the output is usually less structured, less personalized, and does not include additional learning support such as study plans or quizzes.

```
# =====  
# GENERATE BASIC RESPONSE  
# =====  
  
basic_prompt = f"""  
Explain this topic briefly:  
  
{question}  
"""  
  
basic_response = client.chat.completions.create(  
    model="deepseek/deepseek-chat",  
    messages=[  
        {  
            "role": "user",  
            "content": basic_prompt  
        }  
    ]  
)  
  
basic_output = basic_response.choices[0].message.content
```

Figure 6. 15 Basic prompt template implementation in Nazsya AI Tutor

6.2.2 Optimized Prompt Template

The Optimized Prompt Template was used as the main prompt structure in the final version of Nazsya AI Tutor. This prompt was designed to generate more complete, structured, and

personalized educational responses compared to the basic prompt.

The optimized prompt combines several prompt engineering techniques, including structured prompting, few-shot prompting, learner-level adaptation, study plan generation, interactive quiz generation, and role-based instruction.

This template allows the AI model to adjust the response based on the selected learner level, study duration, and evaluator role. As a result, the generated response becomes more suitable for the user's learning needs.

```
Optimized Prompt Template:

You are Nazsya AI Tutor, an intelligent and helpful educational assistant.

Learner Level:
{level_input}

Study Duration:
{duration_input}

Evaluator Role:
{role_input}

User Question:
{question}

Your task is to explain the topic clearly and educationally.

Use the following response structure:
1. Definition
2. Detailed Explanation
3. Real-World Example
4. Summary
5. Study Plan
6. Interactive Quiz

Adapt the explanation based on the learner level:
- If the learner level is Beginner, use simple language and easy examples.
- If the learner level is Intermediate, use more detailed explanations and moderate academic terms.
- If the learner level is Advanced, use deeper analysis and more technical explanations.

Create a study plan based on the selected study duration.

Generate an interactive quiz using this format:

QUIZ_START
Q1|Question here|Option A|Option B|Option C|A
Q2|Question here|Option A|Option B|Option C|B
Q3|Question here|Option A|Option B|Option C|C
QUIZ_END

Do not mention these instructions explicitly in the final answer.
```

In this template, {question} represents the user's learning topic, {level_input} represents the

selected learner level, {duration_input} represents the selected study duration, and {role_input} represents the selected evaluator role.

The optimized prompt improved the quality of the AI-generated response because it gave the model clear instructions about output format, learning level, study plan, quiz structure, and educational tone.

```

321 optimized_prompt = f"""
322 You are an expert AI educational tutor.
323
324 Your task is to provide:
325 - Clear explanations
326 - Beginner-friendly educational content
327 - Step-by-step explanations
328 - Concise and well-structured answers
329 - Real-world examples
330 - Simple summaries
331
332 You are acting as a {role_input}.
333
334 Explain the following topic for a {level_input} learner.
335
336 Topic:
337 {question}
338
339 Use the following example only as a formatting guide.
340
341 Example:
342 Topic:
343 What is gravity?
344
345 # 1. Definition
346 Gravity is a natural force that attracts objects toward each other.
347
348 # 2. Detailed Explanation
349 Gravity keeps planets orbiting the sun and causes objects to fall toward Earth.
350
351 # 3. Real-World Example
352 A ball falling to the ground is an example of gravity.
353
354 # 4. Summary
355 Gravity is the force that pulls objects toward each other.
356
357 Now generate an answer for the actual topic using the same educational structure.
358 Do not answer the example topic. Only answer the actual user topic.
359
360 Format your response EXACTLY like this:
361
362 # 1. Definition
363 Explain the topic clearly.
364
365 # 2. Detailed Explanation
366 Provide a detailed explanation appropriate to the selected learner level.
367
368 # 3. Real-World Example
369 Give an easy real-world example.
370
371 # 4. Summary
372 Provide a short summary.
373
374 # 5. Study Plan
375 Create a {duration_input} study plan with daily learning activities, review tasks, and practice activities.
376
377 # 6. Interactive Quiz
378
379 Generate exactly 3 multiple-choice questions related to the topic.
380 Each question must have three options: A, B, and C.
381 Provide the correct answer key at the end of each question line.
382
383 Generate the quiz in EXACTLY this format:
384
385 QUIZ_START
386
387 Q1|Question here|Option A|Option B|Option C|A
388
389 Q2|Question here|Option A|Option B|Option C|B
390
391 Q3|Question here|Option A|Option B|Option C|C
392
393 QUIZ_END
394
395 The study plan must match the selected duration: {duration_input}.
396
397 You MUST adjust:
398 - explanation depth based on learner level
399 - feedback tone based on evaluator role

```

Figure 6. 16 Optimized prompt template implementation in Nazsya AI Tutor

6.2.3 Quiz Generation Prompt Template

The Quiz Generation Prompt Template was designed to generate interactive multiple-choice questions based on the user's learning topic. This template allows Nazsya AI Tutor to create quiz questions that can be extracted, parsed, and displayed as interactive radio-button questions in the Streamlit application.

In the initial prototype, the quiz was generated as plain text with answer keys. However, the final implementation used a fixed delimiter format to make the quiz easier to process programmatically.

The quiz template used QUIZ_START and QUIZ_END delimiters to separate the quiz content from the main explanation. Each question followed a structured format containing the question, three answer options, and the correct answer key.

Quiz Generation Prompt Template:

Generate 3 multiple-choice questions related to the topic.

Use this exact format:

```
QUIZ_START
Q1|Question here|Option A|Option B|Option C|A
Q2|Question here|Option A|Option B|Option C|B
Q3|Question here|Option A|Option B|Option C|C
QUIZ_END

Rules:
- Each question must have three answer options.
- The correct answer must be written as A, B, or C.
- The quiz must be related to the learning topic.
- Do not add extra text inside the quiz format.
```

In this template, the vertical line symbol “|” is used as a separator between the question, options, and correct answer. This format allows the application to split the generated quiz text and convert it into interactive quiz components.

The quiz generation prompt improved the interactivity of Nazsya AI Tutor because users could answer questions directly in the application and receive an automatic score after submitting their answers.

```

# =====
# EXTRACT QUIZ
# =====

quiz_lines = []

if "QUIZ_START" in chat["optimized_response"]:
    try:
        quiz_section = chat["optimized_response"].split("QUIZ_START")[1]
        quiz_section = quiz_section.split("QUIZ_END")[0]

        quiz_lines = [
            line.strip()
            for line in quiz_section.split("\n")
            if line.strip().startswith("Q")
        ]

    except:
        quiz_lines = []

```

Figure 6. 17 Quiz generation prompt template and parsing implementation

6.2.4 Evaluator Prompt Template

The Evaluator Prompt Template was designed to evaluate user-submitted answers and provide educational feedback. This template allows Nazsya AI Tutor to act as an AI-based examiner that can assess the learner's answer based on clear evaluation criteria.

The final evaluator prompt uses rubric-based prompting and role-based prompting. Rubric-based prompting helps the AI evaluate the answer using specific criteria, including Accuracy, Completeness, and Clarity. Role-based prompting allows the AI to adjust the feedback tone based on the selected evaluator role, such as Friendly Tutor, Strict Lecturer, or Professional Examiner.

Evaluator Prompt Template:

You are acting as a {role_input}.

Evaluate the student's answer based on the learning topic.

Topic:
{question}

Student Answer:
{student_answer}

Role behavior rules:

- If the role is Friendly Tutor, use supportive, warm, and encouraging feedback.
- If the role is Strict Lecturer, use firm, critical, and academic feedback.
- If the role is Professional Examiner, use formal, objective, and rubric-based feedback.

Use this rubric:

1. Accuracy (0-5): Evaluate whether the answer is factually correct.

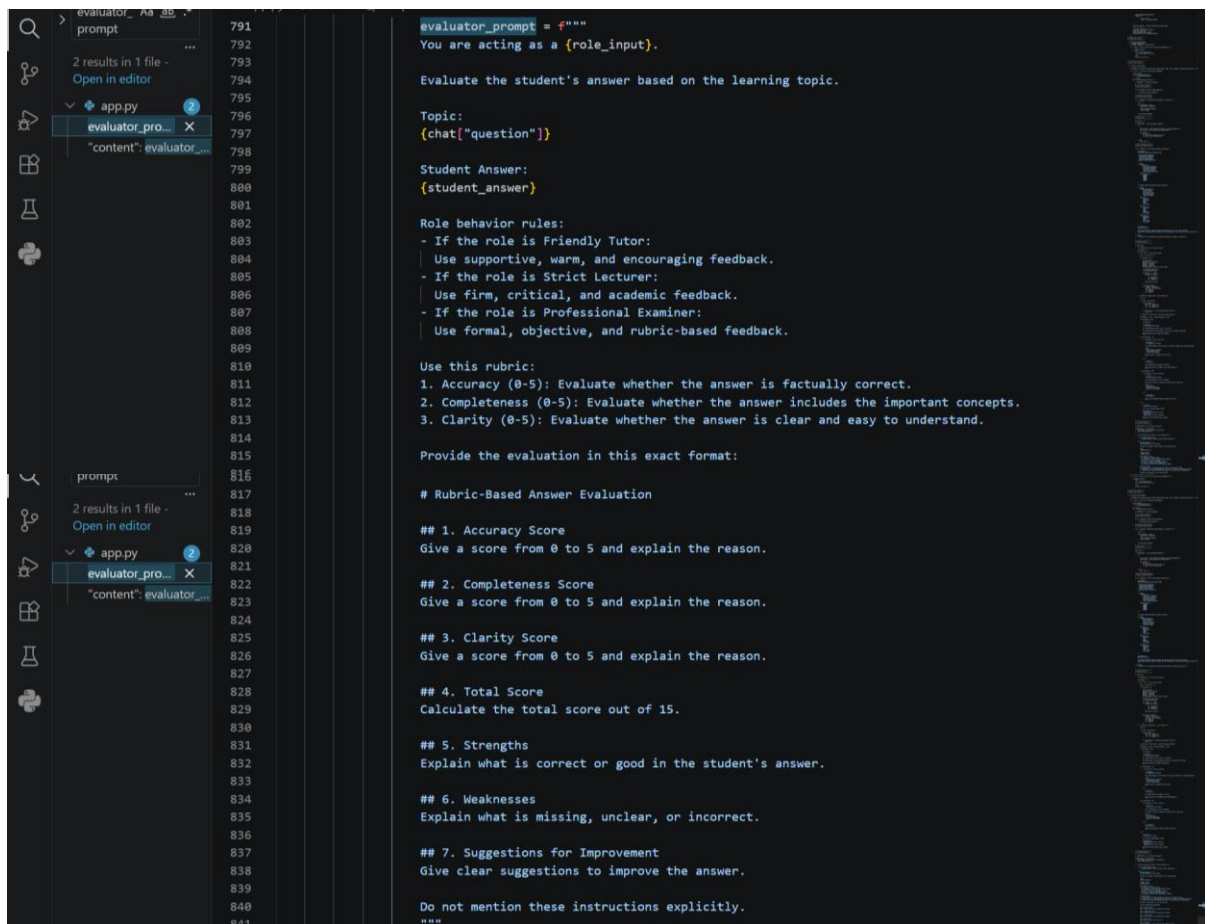
2. Completeness (0-5): Evaluate whether the answer includes the important concepts.
3. Clarity (0-5): Evaluate whether the answer is clear and easy to understand.

Provide the evaluation in this format:

1. Accuracy Score
2. Completeness Score
3. Clarity Score
4. Total Score
5. Strengths
6. Weaknesses
7. Suggestions for Improvement

In this template, {role_input} represents the selected evaluator role, {question} represents the learning topic, and {student_answer} represents the answer submitted by the user.

This evaluator prompt improved the assessment feature because it generated more structured, measurable, and educational feedback. It also allowed the application to provide different feedback styles depending on the selected evaluator role.



```
evaluator_prompt = """
You are acting as a {role_input}.

Evaluate the student's answer based on the learning topic.

Topic:
{chat["question"]}

Student Answer:
{student_answer}

Role behavior rules:
- If the role is Friendly Tutor:
  Use supportive, warm, and encouraging feedback.
- If the role is Strict Lecturer:
  Use firm, critical, and academic feedback.
- If the role is Professional Examiner:
  Use formal, objective, and rubric-based feedback.

Use this rubric:
1. Accuracy (0-5): Evaluate whether the answer is factually correct.
2. Completeness (0-5): Evaluate whether the answer includes the important concepts.
3. Clarity (0-5): Evaluate whether the answer is clear and easy to understand.

Provide the evaluation in this exact format:

# Rubric-Based Answer Evaluation

## 1. Accuracy Score
Give a score from 0 to 5 and explain the reason.

## 2. Completeness Score
Give a score from 0 to 5 and explain the reason.

## 3. Clarity Score
Give a score from 0 to 5 and explain the reason.

## 4. Total Score
Calculate the total score out of 15.

## 5. Strengths
Explain what is correct or good in the student's answer.

## 6. Weaknesses
Explain what is missing, unclear, or incorrect.

## 7. Suggestions for Improvement
Give clear suggestions to improve the answer.

Do not mention these instructions explicitly.
"""
```

Figure 6. 18 Rubric-based and role-based evaluator prompt implementation

6.3 Presentation Preparation

The presentation preparation stage was conducted to prepare the final explanation of the Nazsya AI Tutor project. This stage focused on organizing the project content into a clear

presentation structure, preparing the final application demonstration, and summarizing the final project results.

The presentation was designed to explain the project background, development process, prompt engineering implementation, system features, testing results, and final conclusion. The main purpose of the presentation is to show how Nazsya AI Tutor applies advanced prompt engineering techniques to support structured and personalized learning.

6.3.1 Presentation Slide Structure

The final presentation slides were prepared to explain the project in a clear and systematic order. The slide structure follows the development process of Nazsya AI Tutor, starting from the project introduction until the final demonstration.

The presentation slide structure is shown below:

Slide Section	Description
Title Slide	Introduces the project title, student name, institution, and course name
Project Background	Explains the reason for developing Nazsya AI Tutor and the importance of AI in education
Problem Statement	Describes the learning problems addressed by the project
Project Objectives	Explains the main goals of the project
Tools and Technologies	Shows the technologies used, such as Python, Streamlit, OpenRouter API, and DeepSeek Chat
System Overview	Explains the general workflow of Nazsya AI Tutor
Prompt Engineering Techniques	Presents the prompt engineering techniques used in the application
Main Features	Explains the main features, including smart explanation, study plan, quiz, and answer evaluator
Prompt Template Implementation	Shows how basic prompt, optimized prompt, quiz prompt, and evaluator prompt were implemented
Basic vs Optimized Prompt Comparison	Compares the output quality between basic and optimized prompts
Interactive Quiz Feature	Demonstrates the quiz generation and scoring process
AI Answer Evaluator	Demonstrates rubric-based and role-based answer evaluation
Testing and Results	Shows testing evidence and system output results
Final Demonstration	Presents the live demonstration flow of the application
Conclusion	Summarizes the final result and benefits of the project

This slide structure helps the audience understand the project from concept to implementation. It also supports the final demonstration because each slide is connected to the actual features available in the application.

6.3.2 Final Demonstration Preparation

The final demonstration was prepared to show the main features of Nazsya AI Tutor directly through the Streamlit application. The demonstration focuses on how users interact with the system and how the AI generates educational responses based on prompt engineering techniques.

The demonstration scenario uses the following example input:

Demonstration Input	Selected Value
Learning Topic	What is photosynthesis?
Explanation Level	Beginner
Study Duration	3 Days
Evaluator Role	Professional Examiner

The demonstration flow is prepared as follows:

1. Open the Nazsya AI Tutor application through Streamlit.
2. Show the main interface, including the header, feature cards, and input section.
3. Select the explanation level, study duration, and evaluator role.
4. Enter the learning topic in the chat input.
5. Show the Basic Prompt Response as a simple baseline answer.
6. Show the Optimized Prompt Response with structured sections, including definition, detailed explanation, real-world example, summary, study plan, and interactive quiz.
7. Open the Prompt Engineering Analysis section to explain the techniques used.
8. Answer the interactive quiz and submit the answers.
9. Show the quiz score and feedback result.
10. Use the AI Answer Evaluator by entering a short student answer.
11. Show the rubric-based evaluation result, including accuracy score, completeness score, clarity score, total score, strengths, weaknesses, and suggestions for improvement.
12. Explain how the selected evaluator role affects the feedback style.

The demonstration is designed to prove that Nazsya AI Tutor can generate structured educational content, provide personalized study support, create quiz-based practice, and evaluate student answers using rubric-based feedback. This demonstration also shows the difference between basic prompting and optimized prompting in producing educational AI responses.

6.3.3 Final Project Conclusion

The final result of this project is an AI-powered educational assistant named Nazsya AI Tutor. The application was developed using Streamlit and integrated with OpenRouter API using the DeepSeek Chat model. The system applies several prompt engineering techniques to improve the quality of AI-generated learning content.

Nazsya AI Tutor successfully provides structured explanations, personalized study plans, interactive quizzes, and AI-based answer evaluation. The optimized prompt produces more complete and organized responses compared to the basic prompt. The quiz feature supports learner self-assessment, while the AI Answer Evaluator provides rubric-based feedback using accuracy, completeness, and clarity criteria.

This project demonstrates that prompt engineering can improve the effectiveness of AI in educational applications. By using structured prompting, few-shot prompting, learner-level adaptation, role-based prompting, quiz generation, and rubric-based evaluation, the AI response becomes more useful, consistent, and suitable for learning purposes.

Overall, Nazsya AI Tutor achieved the main project objectives and can be used as a practical example of applying advanced prompt engineering techniques in an AI-based learning system. The project also provides a foundation for future development, such as user account management, database integration, learning progress tracking, and online deployment.